

AUTOMATED VERIFICATION OF EFFECTFUL
HIGHER-ORDER PROGRAMS

DARIUS FOO

NATIONAL UNIVERSITY OF SINGAPORE

2025

AUTOMATED VERIFICATION OF EFFECTFUL
HIGHER-ORDER PROGRAMS

Darius Foo

Bachelor of Computing (Honours) in Computer Science
National University of Singapore

A THESIS SUBMITTED
FOR THE DEGREE OF DOCTOR OF PHILOSOPHY
DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF SINGAPORE

2025

DECLARATION

I hereby declare that the thesis is my original work and it has been written by me in its entirety.

I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Darius

Darius Foo

29th December 2025

Acknowledgments

I would first like to thank my advisor, Dr. Chin Wei-Ngan, for his guidance and compassion. His support was instrumental to my journey, and I would not have made it this far without his encouragement and kindness. He led me to venture into the world of formal verification, and taught me the value of perseverance and never giving up.

I am deeply grateful to my PhD committee members, Dr. Ilya Sergey and Dr. Olivier Danvy, for reviewing my dissertation and giving me valuable feedback. Both had tremendous influence on me prior to their roles on the committee. Ilya's work was an inspiration to me from the beginning. I have always admired his commitment to rigour and machine-checked proof, and gained a lot of insight into the topics I would later work on from his courses on program logics and distributed systems. Olivier's work is foundational to much of the content of this dissertation, and his advice and example completely changed how I approach speaking and writing about my work.

I would also like to thank the faculty members I worked with in the course of my teaching appointments in the School of Computing: Dr. Martin Henz, Dr. Low Kok-Lim, Dr. Boyd Anderson, Dr. Ooi Wei Tsang, Dr. Adi Yoga Sidi Prabawa, and Dr. Damith Rajapakse. They are effective and passionate teachers. Some of them were my own lecturers in my undergraduate days, and their efforts directly shaped my path. CS1101S, in particular, was my introduction to functional programming. I enjoyed the course immensely and had no idea at that time that it would lead me down the road that I find myself on today. I would also like to acknowledge the late Dr. Stephane

Bressane. My first glimpse of the topics which would later underpin this dissertation was in his Discrete Structures course, and he continued to inspire me when I later worked with him in his Database Systems course, with his commitment to precision and clarity in his teaching.

To my collaborators, colleagues, and peers from NUS. My academic family: Yahui, Andreea, Wenhua, Adi, Cristina, Trung, Loc, and Asankhaya. My collaborators: Anh, Ivan, Tristan, Emily, Dan. My friends and colleagues: George, Yunjeong, Yasu, Koon Wen. Thank you for the ideas, the advice, the discussions, and the hard work. Working with you all was truly a privilege and made everything worthwhile. I would also like to acknowledge my fellow graduate tutors, including Bangjie, Alex, and Sherman. Thanks for making this journey with me.

To my friends in The Crescendos, Acappella Anonymous, and St. Joseph's Choir, thank you for enriching my life with music and fellowship. To my friends from NUS SoC, whom I began this journey with: Julia, Matthew, Hong Jin, WeiLin, Shi-Jie. Thanks for all the late-night suppers and conversations. Your friendship helped me through the toughest times.

Finally, I would like to thank my parents and my sister for supporting me unconditionally, and for always being there for me.

CONTENTS

Abstract	ix
1 Introduction	1
1.1 Program verification	1
1.2 Staged logic	2
1.3 Thesis statement & contributions	6
1.4 Publications	7
1.5 How to read this dissertation	8
2 Verifying Higher-order Imperative Programs	9
2.1 Introduction	9
2.1.1 Invariants in separation logic	13
2.1.2 Imperative effects in staged logic	17
2.1.3 Maintaining precision	20
2.2 Staged logic	21
2.2.1 Entailment	24
2.2.2 Reasoning about programs	26
2.3 Entailment checking	33
2.3.1 Rewriting and sequent rules	33
2.3.2 Simplification to a normal form	34
2.3.3 Biabduction	36
2.3.4 Proof search	37
2.4 Evaluation	39
3 Verifying Programs with Effect Handlers	43
3.1 Introduction	43
3.1.1 Reasoning about effect handlers	46
3.1.2 Effect handlers in staged logic	50
3.1.3 Exceptions and higher-order functions	52

3.2	Extensions to staged logic	57
3.3	Reduction	59
3.4	Another route to soundness	62
3.5	Evaluation	63
4	Verifying Programs with Delimited Continuations	65
4.1	Introduction	65
4.1.1	Reasoning with refinement	67
4.1.2	Beyond shift and reset	69
4.2	Extensions to staged logic	75
4.3	Reduction	78
4.4	Evaluation	79
5	Case studies	81
5.1	Landin’s Knot	81
5.2	Interpreting regular expressions	83
5.2.1	Verification	86
5.3	Program equivalence	94
5.3.1	Staged logic	97
6	Mechanising Staged Logic	102
6.1	A model using simple behavioural refinement	102
6.1.1	Staged formulae	102
6.1.2	Entailment	104
6.1.3	Avoiding impredicativity	105
6.1.4	Rewriting	106
6.1.5	Simplification and reduction	107
6.1.6	Limitations of the model	108
6.1.7	Evaluation	110
6.2	Towards a model using contextual refinement	110
6.2.1	A more expressive notion of refinement	111

6.2.2	Encoding in Rocq	113
6.2.3	Remaining work	113
6.3	Summary	114
7	Related work	116
7.1	Program logics	116
7.1.1	Hoare logics	116
7.1.2	Refinement	119
7.1.3	Monadic encodings of effects	121
7.1.4	Relational verification	122
7.2	Higher-order imperative programs	123
7.2.1	Specifications	123
7.2.2	Syntactic methods	125
7.2.3	Semantic methods	126
7.3	Effect handlers	128
7.3.1	In practice	128
7.3.2	Reasoning	128
7.4	Delimited continuations	130
7.4.1	In practice	130
7.4.2	Reasoning	130
8	Conclusion	134
8.1	Future work	134
	Bibliography	135
	Appendix A Higher-order functions specified in Cameleer	173

This dissertation describes *staged logic*, a new approach to the automated verification of effectful higher-order programs. Such programs are completely natural to write in most programming languages, yet are challenging to handle in verifiers, due to the need to reason simultaneously about the ordering and semantics of effects and higher-order control. Today, support for effectful higher-order programs is largely limited to interactive verifiers, each employing disparate, often semantic specification and verification approaches; many automated verifiers either handle them partially or not at all.

Staged logic offers a unified approach to effectful reasoning that is designed to bridge this gap. To support automation, it is syntactic in nature, based on refinement and rewriting. It is simple and general, and scales to complex language features, including effect handlers and delimited continuation operators. It serves as the basis for an automated tool, HEIFER, that verifies annotated OCaml programs via SMT, and is mechanised in Rocq. HEIFER has been used to verify a number of complex and challenging programs, demonstrating that staged logic can be effectively automated. This automation is achieved via a novel proof search procedure and a method of lemma synthesis for reasoning about control operators.

INTRODUCTION

1.1 Program verification

Bugs in software continue to plague the world, sometimes with catastrophic financial consequences. A recent example: a faulty driver deployed by the enterprise security firm CrowdStrike brought about the worst outage in history, rendering an estimated 8.5 million Windows devices across the world unable to boot [197]. The root cause was a memory error due to a parser mishandling invalid data [204]. This is just another example in a *software crisis* that has been in effect since the advent of software engineering [169], and which shows no signs of abating.

Ensuring correctness There is hope, however: there are well-understood ways to ensure that programs work correctly. These can be used to tame unruly software and ensure that it is free of bugs. Two families of approaches [48] in wide use are *testing* and *static analysis*.

Testing, in its modern forms such as fuzzing and symbolic execution, is accessible and inexpensive, scaling to practical systems. It is, however, *underapproximate* in nature, guaranteeing only that a program works correctly in known configurations that have been reached, which may cover only a small portion of an extremely large space of possibilities.

Static analysis seeks to prove properties of a fixed form, and do so automatically and scalably. It has seen huge success; type systems are “arguably the most widely-deployed instance of formal methods in software engineering” [153], and abstract interpretation is relied on in safety-critical domains [44]. The tradeoff is that static analyses suffer from false positives when overapproximations prove too coarse-grained [155], and false negatives when approximations are not possible [140]. In other words, they do

not provide guarantees beyond the fixed (abstract) domains they operate in, which are suitable for proving only simple properties, such as lack of memory errors or overflows.

Verifying programs A third family of approaches, *deductive verification*, is in a sense the only verification method which scales to arbitrarily complex programs *and* properties: hallmark examples include the OS kernels Cer- tikOS [95] and seL4 [124], the C compiler CompCert [138], the TLS stack in Project Everest [38], and, most recently, the Amazon Web Services (AWS) authorization engine [53]. The tradeoff, however, is that deductive verifica- tion scales mainly with human effort and expertise, which is expensive [97]; automation may alleviate some of the burden of constructing proofs, but some amount of handwritten specification is fundamentally required. It is thus used in situations where failure is too costly, testing too incomplete, and approximation too imprecise.

In this work, I seek a way to broaden the applicability of deductive verifi- cation to the features found in modern languages, with the goal of lowering the cost of using it in real-world settings.

1.2 Staged logic

Program logics Deductive verification of programs is carried out using *program logics*, formal systems which view programs as mathematical ob- jects, and are used to establish judgments that they satisfy their *specifications*, which are typically given as logical statements.

Hoare logic is the basis for modern program logics. Its defining feature is the *Hoare triple* $\{P\} e \{Q\}$, a ternary relation between propositions P and Q in some “underlying logic” and a program e , which means roughly that, given a state satisfying the precondition P , evaluation of the program e in it will result in a state satisfying Q . Depending on the choice of logic and

the assertions used, a wide variety of properties can be proved about the behaviours of programs.

Since the inception of Hoare logic, research has focused on scaling it to the features found in real programming languages. *Separation logic*, perhaps its most influential extension, is a substructural logic of resources which allows reasoning about heap-manipulating behaviors, aliasing, fine-grained concurrency, and a myriad of other things which may be modelled as resources.

Higher-order effectful programs One combination of language features that has historically been challenging to support is the interaction of higher-order functions with effects. The HOPE workshop, a regular feature at ICFP for the past 12 years, is dedicated to this problem.

While there has been much progress on it, support for this combination of features remains spotty in today’s verifiers, which are overwhelmingly based on Hoare logic. Amongst automated tools, many do not support verification of such programs at all [137, 108], build it into their logics primitively [206], or use encodings for partial support [75, 182].

Amongst interactive tools, which support varying subsets of effects, there is also no *consensus* on a uniform means of *specification*, which ranges from invariants [42] (Section 2.1) to protocols [72] (Section 3.1), non-context-local triples [198] (Section 4.1), and monads [142, 193, 81] (Section 7.1.2).

To solve these problems, I propose to rethink both the means of specification and verification, which I argue fundamentally makes these problems awkward to solve. I describe a simple refinement-based program logic, *staged logic*. It has first-class support for the combination of higher-order functions and arbitrary effects, and provides a uniform means of specification for both. Due to its elementary nature, it is more generally applicable: in the following chapters, I demonstrate that it is well-suited to serve as the basis of both an

automated verification tool and a foundational encoding in a proof assistant.

Staged logic Staged logic¹ is a logic for precisely specifying programs. A high-level overview of its syntax is given in Fig. 1.1. It is an untyped lambda calculus, extended with specification constructs (with propositions σ for describing program states) and logical connectives.

$\varphi ::= \mathbf{req} \sigma \mid \mathbf{ens} r. \sigma$	Specifications
$\mid \lambda x. \varphi \mid \varphi_1 \varphi_2 \mid \varphi_1; r. \varphi_2 \mid \dots$	Program elements
$\mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \exists x. \varphi \mid \forall x. \varphi$	Logical connectives

Fig. 1.1. Syntax of staged logic

Staged logic is intended to serve as the “underlying logic” of a Hoare triple $\{\varphi'\}e\{\varphi\}$. The precondition is generalised to a *history* φ' , while the postcondition is now a *behaviour* φ . Intuitively, the triple means: φ' is a description of some historical behaviour which occurred prior to the execution of e , and φ describes the composite behaviour of φ' and e . This setup is shown graphically in Fig. 1.2, where s_0, s_1, s_2 are successive program states.

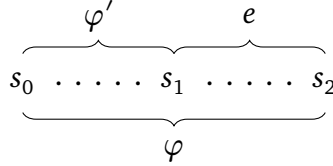


Fig. 1.2. A staged logic triple, graphically

When the meaning is clear from the context, we use “staged logic” to refer to both the assertion language of Fig. 1.1, and its use in a program logic via triples.

¹The name *staged logic* was chosen in homage to Gherghina et al. [92], where the term “staged formulae” first appeared. There, it was a means of controlling operational aspects of how verification conditions were dispatched to provers: “Apart from better sharing, this also allows verification to be carried out incrementally over multiple (smaller) stages, instead of a single (larger) stage.” The process of proving a staged logic refinement also occurs piecemeal in a similar way, as the proof search decomposes a refinement according to the structure of formulae. The partial evaluation around unknowns in staged logic (Section 2.3.2) is also similar to the same mechanism in multi-stage programming, though this was not the original intent behind the choice of name. In particular, it allows verification to be “staged”, carried out incrementally as more information becomes available.

Staged logic generalises Hoare logic: every Hoare logic derivation can be injected into staged logic by defining the regular Hoare triple as follows.

$$\{P\}e\{r.Q\} \triangleq \{\mathbf{ens}\ P\}e\{\mathbf{ens}\ r.Q\}$$

The *specification assertion* $e :: \varphi$ is useful when there is no interesting history to speak of, which gives us another way to define the Hoare triple. In terms of Fig. 1.2, this situation is illustrated by $s_0 = s_1$ and $\varphi' = \mathbf{empty}$, where **empty** is the unit of sequencing.

$$\begin{aligned} e :: \varphi &\triangleq \{\mathbf{empty}\}e\{\varphi\} \\ \{P\}e\{r.Q\} &\triangleq e :: \mathbf{req}\ P; \mathbf{ens}\ r.Q \end{aligned}$$

As staged logic is a program logic, it admits a consequence rule.

$$\begin{array}{c} \text{CONSEQ} \\ \frac{\{ \varphi_3 \} e \{ r. \varphi_4 \} \quad \varphi_1 \sqsubseteq \varphi_3 \quad \varphi_4 \sqsubseteq \varphi_2}{\{ \varphi_1 \} e \{ r. \varphi_2 \}} \end{array}$$

The *refinement* or *entailment* relation $\sqsubseteq$ used in the premises is where much of the interesting work lies when constructing proofs. It enables the use of *rewriting* for reasoning about programs. This, together with the ability to represent programs precisely in the logic, is key to the main advantage of staged logic, which is the ability to switch between compositional reasoning and rewriting as needed. It smoothens the discontinuity that one encounters with language features which are difficult to reason about compositionally. The resulting proofs are simpler, which in turn leaves the door open to automation.

An extended example justifying these claims and exploring their implications will be covered shortly, in Chapter 2.

1.3 Thesis statement & contributions

The thesis I defend is captured by the following statement.

Staged logic is an effective and sound approach to the automated verification of effectful, higher-order programs.

By “effectful”, I mean using *computational effects* [146]. This includes primitive mutable state (Chapter 2), effect handlers (Chapter 3), as well as delimited control operators (Chapter 4).

By “effective”, I mean that staged logic enables automated verification of the aforementioned class of programs. This is supported by the proof search algorithm described in Section 2.3, its implementation (Section 2.3.4), and experimental evaluation (Sections 2.4, 3.5 and 4.4).

By “sound”, I mean that staged logic is compatible with an encoding in a foundational proof assistant (Chapter 6). The soundness of the metatheory is thus checked.

The novelty of my work is justified by Chapter 7, which provides an extensive survey of the landscape of verifiers for such programs and situates staged logic in it.

Contributions The main contributions of this thesis may be summarised as follows.

I propose *staged logic*, a refinement-based approach to modularly specifying and verifying higher-order effectful programs. It improves the state of the art with simpler specifications, not requiring intermediate abstractions, such as protocols, while being equally capable at verification.

I demonstrate its effectiveness in automated verification by formulating a proof search algorithm and implementing it in a new SMT-based verifier, HEIFER².

²Higher-order Effectful Imperative Function Entailments & Reasoning

I justify its soundness by developing an encoding for it in Rocq and implementing a set of tactics for proving staged logic entailments interactively.

Finally, I investigate its expressiveness by demonstrating how it uniformly handles three distinct flavours of effectful higher-order program, hitherto treated in very disparate ways.

1.4 Publications

This dissertation is based on work described in the following publications.

- Darius Foo, Yahui Song, and Wei-Ngan Chin. Staged specification logic for verifying higher-order imperative programs. In *FM 2024: Formal Methods - 17th International Symposium on Formal Methods, Milan, Italy, Sept 9-13, 2024. Proceedings*, Lecture Notes in Computer Science. Springer, 2024
- Yahui Song, Darius Foo, and Wei-Ngan Chin. Specification and verification for unrestricted algebraic effects and handling. *Proc. ACM Program. Lang.*, (ICFP), 2024
- Darius Foo, Yahui Song, and Wei-Ngan Chin, “A Staged Hoare Logic for Imperative Shift/Reset Programs” (under submission)

In the course of my PhD, I worked on a number of other projects. For posterity, publications from them are listed below in chronological order. While they are not described further in my dissertation, they directly or indirectly inspired it in many ways. In particular, the experience working on the APLAS 2022 paper made it clear to me and my collaborators that the problem of reasoning about the temporal behaviors of programs containing algebraic effects was far deeper than we had anticipated, and worthy of a more thorough treatment.

- Darius Foo and Wei-Ngan Chin, “Tracing OCaml Programs” (OCaml 2022) [[85](#)]
- Yahui Song, Darius Foo, and Wei-Ngan Chin, “Automated Temporal Verification for Algebraic Effects” (APLAS 2022) [[185](#)]
- Darius Foo, Andreea Costea, and Wei-Ngan Chin, “Protocol Conformance with Choreographic PlusCal” (TASE 2023) [[86](#)]
- Yahui Song, Darius Foo, and Wei-Ngan Chin, “Specifying and Verifying Future Conditions” (SAS 2025; to appear)

1.5 How to read this dissertation

For an intuitive look at the key ideas and a first pass through examples, see: [Section 2.1](#) (an overview of staged logic), [Section 3.1](#) (effect handlers), [Section 4.1](#) (delimited continuations), [Chapter 5](#) (case studies).

[Chapter 7](#) provides a literature review of this corner of the field of deductive verification.

Finally, the dissertation can be read from beginning to end, for an extended presentation of the logic, interspersed with examples, with features explained as they are introduced.

VERIFYING HIGHER-ORDER IMPERATIVE PROGRAMS

2.1 Introduction

The use of effectful higher-order functions is widespread in everyday programming. It is thus important to support reasoning about such programs in automated tools, if program verification is to be applied more widely.

Yet, today, support for such function in tools is spotty - there is no straightforward answer to the question of how one can easily work with such programs in an automated verification setting.

Consider how we would verify the following OCaml program, which computes the sum of a list of integers `xs` using the higher-order function `foldr`.

```
let foldr f xs acc =
  match xs with
  | [] -> acc
  | x :: t -> f x (foldr f t acc)

foldr (fun x t -> x + t) xs 0
```

This is a purely functional program, and so can be verified in two distinct ways by automated (SMT-based) tools.

Approach 1 We can view `foldr` as a mathematical function, and its arguments as mathematical objects. This view allows us to embed it in a logic, such as the first-order logic of a state-of-the-art automated verifier, like Why3.

```
function foldr_pure
  (f: 'a -> 'b -> 'b) (xs: list 'a) (acc: 'b) :
  'b =
  match xs with
  | Nil -> acc
  | Cons x t -> f x (foldr_pure f t acc)
end
```

$$\{ true \} foldr (\lambda x t. x + t) xs 0 \{ r. r = sum xs \}$$

Fig. 2.1. A Hoare triple specifying a use of *foldr*

We can then state and prove properties of it directly, using standard relations, as well as mathematical functions like *sum*.

```
foldr_pure (fun x t -> x + t) xs 0 = sum xs
foldr_pure (fun x t -> x + t) xs 0 >= 0
```

The function argument can be axiomatised with an uninterpreted function, and the Why3 tactic *induction_ty_lex* produces an induction hypothesis (from the only structurally inductive argument and the form of the goal). The proof is automated from that point on.

Equational reasoning, often used on functional programs, is a special case of this approach, which may in general use other relations.

Approach 2 We can also view *foldr* as a program and verify the call to it using Hoare logic. For that, we would construct a derivation of the Hoare triple in Fig. 2.1.

As the top-level construct is an application, we would need a specification for *foldr*. Having written one, we would prove that the body of *foldr* satisfies it before using it to verify this call.

Looking at the body of *foldr*, the call to *f* in the recursive case immediately seems problematic: nothing is known about *f*, so we cannot proceed. We must somehow strengthen the precondition of *foldr* to say something about how *f* and the result of *foldr* are related.

A standard solution is to use an invariant (Fig. 2.2), which we parameterise the specification of *foldr* over, using a Why3 ghost parameter. *inv* is a property relating the suffix of the input list *xs* traversed to the accumulator *acc* of *foldr*. In the precondition, we require that the invariant holds initially, when the suffix traversed is empty and the ac-

```

let rec foldr (ghost inv : list 'a -> 'b -> bool)
  (f: 'a -> 'b -> 'b) (xs: list 'a) (acc: 'b) : 'b
  variant { xs }
  requires { inv Nil acc }
  requires { forall acc x ys.
    inv ys acc -> inv (Cons x ys) (f x acc) }
  ensures { inv xs result }
= match xs with
| Nil -> acc
| Cons x t -> f x (foldr inv f t acc)
end

```

Fig. 2.2. A specification for *foldr* in Why3, using an invariant

cumulator is at its initial value *acc*, and that it is preserved across the execution of *f*: assuming that the invariant holds of the current suffix *ys* and accumulator, it holds on the extension of the suffix by the element *x* that will be given to *f*, and the result of *f*. Finally, the post-condition ensures that the invariant holds of the result and the entire list.

Note, crucially, that *f* appears as an argument to *inv*, in a *logical assertion* – it is treated here as a mathematical function, even though the use of Hoare logic suggests that *foldr* and it are programs.

Proving that *foldr* satisfies this specification is simple. We know that the invariant holds inductively, and since *f* preserves it, it must hold at the end of the recursive case.

With this specification, the Hoare triple from earlier can be proved. We state it in the form of a Why3 lemma (Fig. 2.3). Note that we must provide a concrete invariant (or ghost argument) for this particular call to *foldr*, which says that the intermediate results of *foldr* on the suffixes of the input list are the sums of the elements of those lists.

```

let lemma foldr_sum (xs : list int) : int
  ensures { result = sum xs }
= foldr (fun suff r -> r = sum suff)
  (fun x t -> x + t) xs 0

```

Fig. 2.3. An invariant for *foldr*, given to prove a Why3 lemma

```

let l = ref 0 in
foldr (fun x () -> l := x + !l) xs (); !l

```

Fig. 2.4. A use of *foldr* involving a closure

Tradeoffs Comparing the two approaches, we see that the second is considerably less automated than the first. Not only did we have to write a specification for *foldr*, which took creativity, we had to supply an invariant, an arbitrary, problem-specific semantic property, which the induction hypothesis of *foldr_pure* already hinted at.

The tradeoff is that the use of a program logic enables compositional reasoning, instead of requiring direct proofs about the entire program at once. Furthermore, Approach 2 is able to handle programs with imperative behaviours, as we will see shortly.

Imperative programs Now, consider the use of *foldr* in Fig. 2.4. It computes the sum of the list, as before, but it does so using a closure, whose imperative *effect* on a captured mutable reference accumulates the numbers in the list.

This is a perfectly natural OCaml program to write, but perhaps surprisingly, this small change leads to a feature cliff: it can no longer be verified by Why3.

Why? Consider how the two approaches from before are affected. They differed in whether *foldr* was treated as a program or mathematical function. However, in both cases, the argument to *foldr*, now a closure, was treated in the latter way and embedded directly in a first-order logic formula; in

Approach 2, an invariant as used to describe its result, but both had no way to talk about its effect.

This problem extends beyond Why3, to other mature automated verifiers [168, Section 2]: for example, Dafny also cannot verify this program for the same reasons. There is no consistent way to verify this in Viper-based verifiers, though variants for specific languages contain extensions which are handle it (e.g. Prusti, though it targets Rust and relies on the guarantees of Rust’s ownership types).

2.1.1 Invariants in separation logic

One might wonder: how do interactive tools, to which more expressive logics are available, handle this example? Indeed, we can look there for inspiration – separation logics such as Iris can indeed be used to verify this example, using Approach 2.

The insight is that *inv* is now a separation logic predicate, i.e. a value of type $(\text{list } 'a \rightarrow 'b \rightarrow \text{hprop})$, where $\text{hprop} := (\text{heap} \rightarrow \text{bool})$, which allows it to represent the state-changing effect of the closure *f*.

This leads to the following specification, in more Iris-like syntax.

$$\begin{aligned} & \forall \text{Inv } f \text{ xs } \text{acc}. \\ & \left\{ \begin{array}{l} \text{Inv } [] \text{ acc} * \\ \forall x \text{ accys}. \{ \text{Inv } \text{ys } \text{acc} \} f \text{ x } \text{acc} \{ r. \text{Inv } (x::\text{ys}) r \} \} \\ \text{foldr } f \text{ xs } \text{acc} \\ \{ \text{result}. \text{Inv } \text{xs } \text{result} \} \end{array} \right\} \end{aligned}$$

Fig. 2.5. A simple specification for *foldr* in separation logic

As before, *Inv* is a parameter, represented now by a higher-order quantification. This time, the fact that *f* preserves the invariant is represented by a *nested Hoare triple*, which is necessary because *f* is a program, not a mathematical function. The proof that the body of *foldr* satisfies this specification is otherwise very similar.

$$\forall l a. \{ l \mapsto a \} \text{foldr } (...) \text{ xs } () \{ r. l \mapsto (a + \text{sum xs}) * [r = ()] \}$$

Fig. 2.6. A separation logic specification for the call to *foldr* in Fig. 2.4

On to the use of *foldr*. We specify the call using a separation logic triple (Fig. 2.6), which states that initially there is some location with value *a*; we quantify over precisely which location in memory it is, and its initial value. The specification guarantees that the location has its value incremented by the sum of the elements of *xs*. $[-]$ injects a first-order proposition into separation logic.

To prove this triple, we would use the triple in Fig. 2.5 with the invariant $(\lambda \text{ suff } r. l \mapsto (\text{sum suff}) * [r = ()])$, which is similar to the one given in Fig. 2.3.

The one-spec-per-use problem The use of separation logic solves the immediate problem. However, this general approach has a rather severe shortcoming, which only becomes apparent once variations of the particular use of *foldr* from above are considered. The authors of the Iris Lecture Notes [42] recognise this:

Different clients may instantiate *foldr* with some very different functions, hence it can be hard to give a specification for *f* that is reasonable and general enough to support all these choices. In particular knowing when one has found a good and provable specification can be difficult in itself.

In other words, even though the specification as written is very general, it is not sufficient to verify all uses of *foldr*. I refer to this as *the one-spec-per-use problem*. It is a fundamental issue with *the use of semantic properties of the underlying logic to represent effects*.

I illustrate this point by the following two examples.

1. The following illustrative variation is also from the Iris Lecture Notes.

$$\begin{aligned}
& \forall \text{Inv } P \text{ } f \text{ } xs \text{ } acc. \\
& \left\{ \begin{array}{l} [all \ P \ xs] * \text{Inv } [] \ acc * \\ \forall x \ acc \ ys. \{ [Px] * \text{Inv } ys \ acc \} \ f \ x \ acc \ \{ r. \text{Inv } (x::ys) \ r \} \end{array} \right\} \\
& \quad \text{foldr } f \text{ } xs \text{ } acc \\
& \quad \{ result. \text{Inv } xs \ result \}
\end{aligned}$$

Fig. 2.7. A specification for *foldr* with a stronger precondition

This specification is similar to the one in Fig. 2.5, but is also parameterised over a unary predicate P , a precondition for f . P holds individually of each element x of the list. Thus, to rely on P in f , we must require that *all* $P \ xs$ holds in the precondition of *foldr*.

This captures the pattern of what one should do given a function argument that is partial; we would propagate the precondition outwards, to that of *foldr*.

While this example is highly illustrative, it also shows the issue with the approach. The choice of the (unary) form of P is rather arbitrary, and constrains the kinds of properties one can use.

- Suppose one wanted to rely on a property concerning consecutive elements of the list. One would have to write a new specification, with a property parameterised over xs .
- What if one passed an argument such as $(\lambda x \ t. \text{assert } x \geq t; x+t)$? This would require a property relating each element and suffix-sum. Again, neither P or Inv would be sufficient.

Making all these choices upfront would produce a specification parameterised over all sorts of extraneous predicates, which is also untenable.

The point is that this approach requires one to commit to a parametrisation of a fixed form, which clearly cannot work for all cases.

2. What if we passed the argument $(\lambda x \ t. \text{raise } E)$? This approach clearly cannot cope with this, as the invariant cannot be maintained – the ef-

fect of this function cannot be described in terms of program states. This highlights the fact that the invariant over resources is only *a representation of a specific kind of effect*. It does not generalise to other kinds of effects because it does not faithfully describe effects themselves.

To recap, our options today for automatically verifying effectful higher-order programs are as follows, with the problems presented in the order they were introduced. Neither is ideal.

Approach 1 Less compositional, and ultimately inapplicable, as we are not working with pure functions.

Approach 2 Less automated, one-spec-per-use problem, requires underlying logic that can represent effects.

Towards a solution Backtracking a little, suppose we allowed ourselves to imagine a logic where Approach 1 could apply. This is the motivation for staged logic. It is a simple, unifying viewpoint on the problem which combines the advantages of Approaches 1 and 2, while solving their problems:

- It admits a formulation as a program logic, retaining the compositionality of Approach 2.
- Like Approach 1, an intermediate specification for `foldr` is not required, and a standard induction hypothesis is used instead of an invariant.
- The one-spec-per-use problem is solved by not requiring one to commit to a parametrisation, or any other kind of representation or proxy for describing higher-order functions and effects. This is done by maintaining a precise account of effects and their ordering.
- As it is itself a direct means of describing effects, the underlying logic is not used or required to represent effects. For example, there is no

need for an underlying separation logic for reasoning about programs that use exceptions, but no mutable state.

In the rest of this chapter, I make these points concrete and illustrate how staged logic does these things in the setting of programs with imperative effects (with exceptions covered later, in [Section 3.1.3](#)). Subsequent chapters generalise this to arbitrary control effects and provide a more formal treatment.

2.1.2 Imperative effects in staged logic

We start with the language of staged logic from [Fig. 1.1](#), which allows us to both *model* and *specify* programs. In the former view, it is a (higher-order) intermediate verification language [[149](#)], where one may mix specifications and programs. In the latter, it is a logic, in which one may use quantifiers and other standard logical connectives, assert and assume arbitrary propositions, and talk about programs using fragments of syntax.

Our approach to proving the triple in [Fig. 2.6](#) will be as follows.

1. We model *foldr* (and its argument *f*) in staged logic, such that

$$f :: g \qquad \text{foldr } f \, xs \, a :: \text{foldr}(g, xs, a)$$

both hold by construction.

2. We prove the entailment

$$\text{foldr}(g, xs, ()) \sqsubseteq \mathbf{req} \, l \mapsto a; \mathbf{ens} \, r. l \mapsto (\text{sum } xs + a) * [r = ()]$$

with the specification also given in staged logic.

3. We use the rule of consequence to piece both of these together into a proof of the assertion:

$$\text{foldr } f \, xs \, a :: \mathbf{req} \, l \mapsto a; \mathbf{ens} \, r. l \mapsto (\text{sum } xs + a) * [r = ()]$$

which is equivalent to the triple in Fig. 2.6.

First, we model the *foldr* program (Fig. 2.8). This is a direct translation of the constructs of the program to the corresponding features in the logic. We use disjunction to model the branches of the **match** and existentials for the destructured variables. An intuitive view of **ens** is that it allows us to assume conditions on values; we use it both for the assumptions on the form of the **match** scrutinee in each branch, and to describe what the result of each branch should be, via the binder *res*. Calls to recursive and unknown functions are represented directly, in their sequential order. Binding is written $\varphi_1; r. \varphi_2$; in the syntax of staged logic, binders are always followed by a period.

<pre> let foldr f init xs = match xs with [] => init h :: t => f h (foldr f init t) </pre>	$ \begin{aligned} \text{foldr}(f, \text{init}, xs) &\triangleq \\ &\mathbf{ens} \text{ res}. xs = [] \wedge \text{res} = \text{init} \\ &\vee \exists h \ t. \mathbf{ens} \text{ xs} = h :: t; \\ &\quad \text{foldr}(f, \text{init}, t); r. \\ &\quad f(h, r) \end{aligned} $
(a) <i>foldr</i>	(b) <i>foldr</i>

Fig. 2.8. *foldr* program and formula

Next, we prove the entailment:

$$\forall l \ a. \text{foldr}(g, xs, ()) \sqsubseteq \mathbf{req} \ l \mapsto a; \mathbf{ens} \ r. l \mapsto (\text{sum } xs + a) * [r = ()]$$

Interestingly, the proof can be completely automated.

Proof. We proceed by induction on the structure of *xs*. Unfolding *foldr* and focusing on the inductive case, we get the following entailment.

$$\begin{aligned}
 &\mathbf{ens} \ (xs = h :: t); \text{foldr}(f, (), t); r. f(h, r) \\
 &\sqsubseteq \mathbf{req} \ x \mapsto a; \mathbf{ens} \ \text{res}. x \mapsto (\text{sum } xs + a) * [\text{res} = ()]
 \end{aligned}$$

Rewriting the call to *foldr* using the induction hypothesis and unfolding

f , we get a lengthy sequence of **req** and **ens**.

$$\begin{array}{l} \mathbf{ens} (xs = h :: t); \\ \forall a_1, \mathbf{req} x \mapsto a_1; \mathbf{ens} r. (x \mapsto (\text{sum } t + a_1) * [r = ()]); r. \\ \underline{\forall z, \mathbf{req} x \mapsto z; \mathbf{ens} res. x \mapsto (h + z) * [res = ()]} \\ \sqsubseteq \mathbf{req} x \mapsto a; \mathbf{ens} res. x \mapsto (\text{sum } xs + a) * [res = ()] \end{array}$$

It is not as complex as it looks: it is merely a description of all the state changes to x which occur along this program path. The underlined portion is notable because it can be subjected to simplification. Intuitively, when on the left side of an entailment, an **ens** can be understood as a separation logic *assumption*, producing a piece of heap, while **req** can be seen as an *obligation*, consuming a piece of heap. This view lets us “cancel” them; the mechanism which enables this is *biabduction* [50].

$$\frac{H_A * H_1 \vdash H_2 * H_F}{\mathbf{ens} H_1; \mathbf{req} H_2 \sqsubseteq \mathbf{req} H_A; \mathbf{ens} H_F} \text{BIAB}$$

We can view this simplification of the formula representing the program as a kind of symbolic execution, analogous to what separation logic verifiers typically do. **req** and **ens** can be seen as an *internalisation* of the operational behaviour of heap entailment.

What remains to be proved, then, is the following.

$$\begin{array}{l} \forall a_1, \mathbf{req} x \mapsto a_1; \mathbf{ens} res. x \mapsto (h + (\text{sum } t + a_1)) * \\ [res = () \wedge xs = h :: t] \\ \sqsubseteq \mathbf{req} x \mapsto a; \mathbf{ens} res. x \mapsto (\text{sum } xs + a) * [res = ()] \end{array}$$

Now, we see that both sides of the goal consist of a single pair of **req** and **ens**. This allows it to be reduced to separation logic entailment via the following two rules.

$$\frac{H_2 \vdash H_1}{\mathbf{req} H_1 \sqsubseteq \mathbf{req} H_2} \qquad \frac{Q_1 \vdash Q_2}{\mathbf{ens} Q_1 \sqsubseteq \mathbf{ens} Q_2}$$

The two entailments follow.

$$\begin{array}{l} \forall a_1. x \mapsto a_1 \vdash x \mapsto a \\ x \mapsto (h + (\text{sum } t + a)) * \\ \quad [res = () \wedge xs = h :: t] \\ \vdash x \mapsto (\text{sum } xs + a) * [res = ()] \end{array}$$

The one on the right can be reduced to the following first-order obligation, which is easily dispatched by an SMT solver with standard list and arithmetic theories.

$$\begin{array}{l} res = () \wedge xs = h :: t \\ \Rightarrow h + (\text{sum } t + a) = \text{sum } xs + a \wedge res = () \end{array}$$

□

2.1.3 Maintaining precision

What exactly enabled the automation? The crucial difference from the standard separation logic approach in [Section 2.1.1](#) is that we were not forced to come up with an abstraction for the behaviour of *foldr* immediately, in the form of its invariant-parameterised specification. Instead, we simply left the parts which were difficult to handle *in the specification*, until we knew enough (i.e. saw the definition of the closure argument) that proving something meaningful was easy. In other words, we retained a precise specification of the essential portions – the ordering of effectful higher-order calls – while abstracting away things like the concrete state changes which occurred.

The insight from this example is simply that some language features are inherently challenging to specify modularly, because they depend on things external to the program being specified. While feature- or even problem-specific insight can be employed to write down an adequate specification, such a specification will certainly be lossy.

I argue that it is simpler and more productive to sometimes allow the extent of a modular treatment for a particularly tricky language feature to be: to let it be represented in the specification, then to handle it (in)equationally in the logic at a later point. This results in a smoothening of the discontinuity

in what a state-based logic can represent: proofs become simpler, which in turn enables automation.

In the rest of this chapter, we provide a more formal treatment of what we have seen so far, by fixing a syntax and semantics for staged logic, then describing the ingredients needed for a proof search procedure which implements the automation from the previous section.

In the following chapters, we apply this methodology to the verification of more kinds of effectful programs.

For a first pass through this dissertation, it is possible to jump to [Section 3.1](#) (effect handlers), then [Section 4.1](#) (delimited continuations) to get a sense of the breadth of the framework.

2.2 Staged logic

$\varphi ::= \mathbf{req} \sigma \varphi \mid \mathbf{ens} r. \sigma$	Specifications
$\mid f(v, r) \mid \varphi_1; \varphi_2$	Programs
$\mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \exists x. \varphi \mid \forall x. \varphi$	Logical connectives

Fig. 2.9. Syntax of staged logic

Syntax The syntax of staged logic is given in [Fig. 2.9](#)¹. Its formulae consist firstly of **req** and **ens** constructs, which are best thought of as similar to the corresponding annotations in standard Hoare logic verifiers. Something unusual here is that they are internalised in the logic, i.e. used to construct propositions, and not merely delimiters for different portions of a Hoare triple. σ is a state formula whose models are program states. Henceforth we take it to be a separation logic heap formula H , i.e. $\sigma ::= H$.

¹There are some differences from the syntax given in [Fig. 1.1](#), which is the staged logic that we have by the end of the dissertation. Functions are represented with a relational output parameter and sequencing is used instead of binding; this is explained in [Section 3.2](#). Staged logic technically is a lambda calculus, but we use a function-pointer presentation of functions here, to remain closer to the implementation and the original paper [87]. The lambda calculus presentation is explored in [Section 6.2](#).

Informally, **req** $\sigma \varphi$ may be understood as an obligation to prove that some program state satisfies σ , and **ens** σ as an assumption that we have a program state satisfying σ , though like their annotation counterparts, their roles flip depending on which side of an call site (or entailment) they are on. We will make this precise shortly.

The next two constructs are what allow effectful behaviour to be represented natively in the logic: we can talk directly about a call to an unknown, possibly-effectful function f with argument v and result r , and also retain the sequential order of such unknown calls in specifications.

The remaining ones are standard logical connectives lifted to specifications and programs.

The following abbreviations are also useful:

$$\begin{aligned}
\mathbf{req} H; \varphi &\triangleq \mathbf{req} H \varphi & \mathbf{ret} v &\triangleq \mathbf{ens} r. [r=v] \\
\mathbf{ens} H &\triangleq \mathbf{ens} r. [r=()] * H & \mathbf{assert} H &\triangleq \mathbf{req} H; \mathbf{ens} H \\
\mathbf{empty} &\triangleq \mathbf{ens} \mathbf{true} & \perp &\triangleq \mathbf{ens} \mathbf{false} & \top &\triangleq \mathbf{req} \mathbf{false}; \mathbf{empty}
\end{aligned}$$

Semantics Staged logic admits computational behaviour, so its models are conceptually traces: sequences of program states. As we work in separation logic, we identify a program state with a heap, which is a finite partial map from locations ℓ to values v . We represent traces intensionally as an initial heap, a final heap, and a result value, and a staged formula φ relating them. Finally, because φ may contain uses of unknown functions, we augment the model with a *specification environment* $\mathcal{E}$, a finite partial map from variables to staged formula functions $\lambda v r. \varphi$, to give them interpretations.

The semantics of a staged formula can thus be given via the *satisfies* relation $\mathcal{E}, h_1, h_2, v \models \varphi$, defined in Fig. 2.10. It is similar to a nondeterministic big-step semantics containing logical connectives.

SREQ shows that, as mentioned earlier, a **req** represents the obligation to prove that the “current” program state h_1 satisfies H . In separation logic

terms, this means the initial heap h_1 can be partitioned into two heaplets h_p and h_r , such that h_p satisfies H (and subsequently vanishes), and the remaining piece h_r may serve as the initial heap for satisfying φ . Dually, **SENS** shows that we may assume a piece of heap h_3 satisfying Q into existence and add it to the initial heap h_1 . These internalise the operational meaning of heap entailment in symbolic execution-style separation logic verification, similar to the *exhale* and *inhale* operations from Viper [149]².

Next are functions and sequencing. The semantics of an unknown function call is simply that of its interpretation in $\mathcal{E}$ in **SFN**. Sequencing in **SSEQ** is similar to relational composition of *satisfies* assertions on heaps, discarding the intermediate result v_1 .

The remaining rules lift the standard connectives to staged logic.

$\mathcal{E}, h, h, v \models \varphi$		
$\frac{\text{SREQ} \quad \forall h_p h_r. \left(\begin{array}{l} h_1 = h_p \circ h_r \wedge h_p \models H \Rightarrow \\ \mathcal{E}, h_r, h_2, v \models \varphi \end{array} \right)}{\mathcal{E}, h_1, h_2, v \models \mathbf{req} \ H \ \varphi}$	$\frac{\text{SENS} \quad \begin{array}{l} h_3, v \models Q[v/r] \\ h_2 = h_1 \circ h_3 \end{array}}{\mathcal{E}, h_1, h_2, v \models \mathbf{ens} \ r. Q}$	
$\frac{\text{SFN} \quad \begin{array}{l} \mathcal{E}(f) = g \quad \mathcal{E}, h_1, h_2, r \models g(v, r) \end{array}}{\mathcal{E}, h_1, h_2, r \models f(v, r)}$	$\frac{\text{SSEQ} \quad \begin{array}{l} \mathcal{E}, h_1, h_3, v_1 \models \varphi_1 \\ \mathcal{E}, h_3, h_2, v \models \varphi_2 \end{array}}{\mathcal{E}, h_1, h_2, v \models \varphi_1; \varphi_2}$	
$\frac{\text{SEXISTS} \quad \exists a. (\mathcal{E}, h_1, h_2, v \models \varphi)}{\mathcal{E}, h_1, h_2, v \models \exists a. \varphi}$	$\frac{\text{SFORALL} \quad \forall a. (\mathcal{E}, h_1, h_2, v \models \varphi)}{\mathcal{E}, h_1, h_2, v \models \forall a. \varphi}$	
$\frac{\text{SDISJL} \quad \mathcal{E}, h_1, h_2, v \models \varphi_1}{\mathcal{E}, h_1, h_2, v \models \varphi_1 \vee \varphi_2}$	$\frac{\text{SDISJR} \quad \mathcal{E}, h_1, h_2, v \models \varphi_2}{\mathcal{E}, h_1, h_2, v \models \varphi_1 \vee \varphi_2}$	$\frac{\text{SCONJ} \quad \begin{array}{l} \mathcal{E}, h_1, h_2, v \models \varphi_1 \\ \mathcal{E}, h_1, h_2, v \models \varphi_2 \end{array}}{\mathcal{E}, h_1, h_2, v \models \varphi_1 \wedge \varphi_2}$

Fig. 2.10. Semantics of staged logic

²They are only similar insofar as they produce and consume pieces of heap. **req** and **ens** flip in meaning depending on which side of an entailment they are on, whereas *exhale* and *inhale* are ghost operations with a fixed meaning – in other words, **req** is translated to *either inhale or exhale* depending on the side of an entailment it occurs on.

2.2.1 Entailment

To support reasoning at the level of staged formulae, with higher-level induction and rewriting principles instead of with pieces of heaps, we introduce an entailment relation $\sqsubseteq$. This is also the refinement relation, if we think of a formula as a program. We define it intensionally, as a direct lifting of implication to staged logic.

Definition 2.1 (Entailment and equivalence).

$$\begin{aligned}\varphi_1 \sqsubseteq \varphi_2 &\triangleq \forall \mathcal{E} h_1 h_2 R. (\mathcal{E}, h_1, h_2, R \models \varphi_1 \Rightarrow \mathcal{E}, h_1, h_2, R \models \varphi_2) \\ \varphi_1 \equiv \varphi_2 &\triangleq \varphi_1 \sqsubseteq \varphi_2 \wedge \varphi_2 \sqsubseteq \varphi_1\end{aligned}$$

This semantic definition is useful to justify the soundness of the entailment rules in this section in this section and in [Section 2.3.2](#). In [Section 2.3.4](#), we give an alternative, syntactic definition suitable for automated proof.

Properties of entailment Entailment is reflexive and transitive, and equivalence is an equivalence relation. Using both, we can now state properties of the formulae of staged logic.

Many standard properties apply to lifted connectives: for example, disjunction is associative, commutative, idempotent, and has $\perp$ as an identity element and $\top$ as annihilator. We omit discussion of these and focus only on properties of the new constructs, in [Fig. 2.11](#).

$\top$ and $\perp$ function as one would expect as the staged analogues of *true* and *false*. [NSEQBOTL](#) shows that when combined with sequencing, $\perp$ has its usual interpretation as “dead code” or “unreachability”, allowing anything after it to be ignored.

Important properties of **req** and **ens** are that they are contravariant and covariant, respectively, results familiar from the theory of behavioural subtyping [131]. [NREQPUREL](#) and [NENSPURER](#) further illustrate the duality of

$$\boxed{\varphi \sqsubseteq \varphi}$$

NBOT	NTOP	NSEQBOTL	REQCONTRA
$\frac{}{\perp \sqsubseteq \varphi}$	$\frac{}{\varphi \sqsubseteq \top}$	$\frac{}{\perp; \varphi \equiv \perp}$	$\frac{H_2 \vdash H_1 \quad \varphi_1 \sqsubseteq \varphi_2}{\mathbf{req} H_1 \varphi_1 \sqsubseteq \mathbf{req} H_2 \varphi_2}$
ENSCOV	NREQPUREL	NENSPURER	
$\frac{H_1 \vdash H_2}{\mathbf{ens} H_1 \sqsubseteq \mathbf{ens} H_2}$	$\frac{P}{\mathbf{req} [P] \varphi \equiv \varphi}$	$\frac{P}{\mathbf{empty} \equiv \mathbf{ens} [P]}$	
NSEQEMPL	NSEQEMPR	NSEQASSOC	
$\frac{}{\mathbf{empty}; \varphi \equiv \varphi}$	$\frac{\varphi[()]}{\varphi; \mathbf{empty} \equiv \varphi}$	$\frac{}{(\varphi_1; \varphi_2); \varphi_3 \equiv \varphi_1; (\varphi_2; \varphi_3)}$	
NDISJSEQDISTR		NFORALLDISTR	
$\frac{}{\varphi_1 \vee (\varphi_2; \varphi_3) \equiv (\varphi_1 \vee \varphi_2); (\varphi_1 \vee \varphi_3)}$		$\frac{}{\varphi_1; \forall x. \varphi_2 \sqsubseteq \forall x. \varphi_1; \varphi_2}$	
NFORALLDISTR		REQREASSOC	
$\frac{}{(\forall x. \varphi_2); \varphi_1 \sqsubseteq \forall x. \varphi_2; \varphi_1}$		$\frac{}{(\mathbf{req} H \varphi_1); \varphi_2 \sqsubseteq \mathbf{req} H (\varphi_1; \varphi_2)}$	

Fig. 2.11. Select properties of entailment

req and **ens**, showing that to introduce **req** (resp. **ens**) on the left (resp. right), one must be able to prove the condition P ; with **req** (resp. **ens**) on the *right* (resp. left), one *assumes* P . **empty** is the left identity of sequencing, and its right as well only if φ has the unit value as its result. Sequencing is associative, and disjunction distributes over it. Quantifiers distribute over sequencing and vice versa; note that these are not equivalences because φ_1 may depend on x . Finally, **REQREASSOC** shows that **req** may always be rewritten into right-associative form, i.e. where anything sequenced after it may be “moved under” the **req**, a key property which ensures that the nesting of **req** and sequencing does not matter³. Note that the converse of this last property is not true, as φ_2 may be dependent on an assumption in H .

Theorem 2.2.1 (Soundness of Entailment Rules). *Given an entailment $\varphi_a \sqsubseteq \varphi_c$, the consequent overapproximates the antecedent, i.e. if $\mathcal{E}, h_1, h_2, R \models \varphi_a$,*

³This ensures that the abbreviation for **req** defined at the start of [Section 2.2](#) does not cause any loss of expressiveness.

then $\mathcal{E}, h_1, h_2, R \models \varphi_c$.

Proof. See [Section 6.1.2](#). □

2.2.2 Reasoning about programs

In this section, we take a short detour to formalise the relation between staged logic and programs. To continue the progression towards a proof search procedure, skip ahead to [Section 2.3](#).

$$e ::= x \mid v \mid \text{let } x=e_1 \text{ in } e_2 \mid \lambda x.e \mid \lambda f x.e \mid e_1 e_2 \mid \text{assert } e \mid \\ \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid \text{ref } e \mid e_1 := e_2 \mid !e$$

Fig. 2.12. Core language

Core language We begin with the definition of an ML-like core language ([Fig. 2.12](#)), with higher-order functions and primitive state. Expressions are assumed to be in A-normal form (ANF); sequencing and control over evaluation order may be achieved using *let*-bindings, which define immutable variables. Mutation may occur through heap-allocated *refs*. *assert* allows proofs of program properties to be carried out at arbitrary points.

We assume a standard set of values v , including the unit value $()$, integers i , strings s , boolean values *true* and *false*, tuples (v_1, v_2) , lists built using constructors *nil* and $::$, recursive and non-recursive lambda expressions $\lambda f x.e$ and $\lambda x.e$, and *locations* ℓ .

A big-step reduction relation $E, h_1, e \rightsquigarrow h_2, v$ ([Fig. 2.13](#)) is used to define the semantics of programs. As before ([Section 1.2](#)), program states are represented by heaps, and *program environments* E are the program analogue of specification environments, mapping variables to function values.

let-bindings are handled by (capture-avoiding) substitution in [BIGLET](#), avoiding the need for a separate store. Function application is handled by [BIGAPPFUN](#) and [BIGAPPFIX](#) if functions are known; if they are unknown, i.e.

$\boxed{E, h, e \rightsquigarrow h, v}$		
BIGVAL	BIGFUN	BIGFIX
$\frac{}{E, h, v \rightsquigarrow h, v}$	$\frac{}{E, h, (\lambda x. e) \rightsquigarrow h, (\lambda x. e)}$	$\frac{}{E, h, (\lambda f x. e) \rightsquigarrow h, (\lambda f x. e)}$
BIGASSERT		BIGAPPFUN
$\frac{}{E, h, (\text{assert true}) \rightsquigarrow h, ()}$		$\frac{E, h_1, e[v/x] \rightsquigarrow h_2, R}{E, h_1, (\lambda x. e) v \rightsquigarrow h_2, R}$
BIGAPPUNK	BIGAPPFIX	
$\frac{E, h_1, (E(f) v) \rightsquigarrow h_2, R}{E, h_1, (f v) \rightsquigarrow h_2, R}$	$\frac{E, h_1, e[v/x][(\lambda f x. e)/f] \rightsquigarrow h_2, R}{E, h_1, (\lambda f x. e) v \rightsquigarrow h_2, R}$	
BIGLET		BIGREF
$\frac{E, h_1, e_1 \rightsquigarrow h_3, v \quad E, h_3, e_2[v/x] \rightsquigarrow h_2, R}{E, h_1, (\text{let } x=e_1 \text{ in } e_2) \rightsquigarrow h_2, R}$		$\frac{\ell \notin \text{dom}(h_1) \quad h_2 = h_1[\ell \mapsto v]}{E, h_1, (\text{ref } v) \rightsquigarrow h_2, \ell}$
BIGIFT		BIGDEREF
$\frac{E, h_1, e_1 \rightsquigarrow h_2, R}{E, h_1, (\text{if true then } e_1 \text{ else } e_2) \rightsquigarrow h_2, R}$		$\frac{\ell \in \text{dom}(h)}{E, h, (!\ell) \rightsquigarrow h, h(\ell)}$
BIGIFF	BIGASSIGN	
$\frac{E, h_1, e_2 \rightsquigarrow h_2, R}{E, h_1, (\text{if false then } e_1 \text{ else } e_2) \rightsquigarrow h_2, R}$	$\frac{\ell \in \text{dom}(h)}{E, h, (\ell := v) \rightsquigarrow h[\ell \mapsto v], R}$	

Fig. 2.13. Big-step semantics

a variable f , which would otherwise be stuck, the rule **BIGAPPUNK** applies. Due to the use of ANF and substitution to handle *let*-bindings, this situation only occurs when a function *without a prior definition* is applied. Hence, environments enable talking about *open* programs by deferring their closure to the point where an environment is provided. We say more on this subject shortly, when we discuss the soundness of staged logic.

Specification assertions We relate a program and a staged formula via a *specification assertion*, a binary relation $e ::: \varphi$, named by analogy to a *type assertion* $e : \tau$.

A specification assertion may be thought of as a generalization of the Hoare triple – a triple $\{ P \} e \{ Q \}$ is equivalent in meaning to $e ::: \mathbf{req} P; \mathbf{ens} Q$.

However, there are many staged formulae which cannot directly be written as Hoare triples, at least without further parametrisation at the meta-logical level; a simple example is the *open* program $e; f(v, r) ::= \mathbf{req} P; \mathbf{ens} Q; f(v, r)$, where an unknown function f is called after e . An equivalent specification in Hoare logic would require quantifying over some invariant of f , or properties representing (approximations of) its pre- and postconditions, as mentioned in [Section 2.1](#). In this view, staged logic can be seen as a means of importing some syntactic reasoning into the language of specifications.

[Fig. 2.14](#) contains mostly syntax-directed rules for not just reasoning about programs, but *deriving* the staged logic form of a program. Focusing on the latter reading gives us a means of *verifying* that a program conforms to a specification, via the rule of consequence [FCONSEQ](#), the only rule that is not syntax-directed: to verify $e ::= \varphi_2$, it suffices to derive a specification φ_1 for e , then check the entailment $\varphi_1 \sqsubseteq \varphi_2$.

The [FVAL](#) rule relates a value v to $\mathbf{ret} v$. In [FLET](#), we first derive a specification φ_1 for the expression e_1 being bound, and, given that *its result is* v – denoted by $\varphi[v] \triangleq \forall \mathcal{E} h_1 h_2 v_1. \mathcal{E}, h_1, h_2, v_1 \models \varphi \Rightarrow v_1 = v$ – we substitute v into e_2 , derive its specification, then simply sequence the (effects of the) two specifications. Given the use of ANF, *let*-bindings are the only place at which effects are sequenced. The use of substitution again allows us to avoid having stores as part of the model.

The next three rules concern function application. [FAPPFUN](#) and [FAPPFIX](#) are standard rules for handling the application of possibly-recursive, *known* lambda expressions, which occur in this form due to the use of ANF and substitution. Something to note is that despite their naive-looking definitions, which appear to require verifying same function repeatedly at every call to it with different arguments – quite against the spirit of modular verification! – if a function has had a prior specification assertion $\forall x. (e ::= \varphi)$ proven (quantified over parameters occurring in *both* e and φ), we may simply in-

$e :: \varphi$

$$\begin{array}{c}
\text{FCONSEQ} \quad \frac{e :: \varphi_1 \quad \varphi_1 \sqsubseteq \varphi_2}{e :: \varphi_2} \qquad \text{FVAL} \quad \frac{}{v :: \mathbf{ret} \ v} \qquad \text{FLET} \quad \frac{e_1 :: \varphi_1 \quad \varphi_1[v] \quad e_2[v/x] :: \varphi_2}{(\mathbf{let} \ x=e_1 \ \mathbf{in} \ e_2) :: \varphi_1; \varphi_2} \\
\\
\text{FAPPFUN} \quad \frac{e[v/x] :: \varphi}{(\lambda x. e) \ v :: \varphi} \qquad \text{FAPPFIX} \quad \frac{e[v/x][(\lambda f x. e)/f] :: \varphi}{(\lambda f x. e) \ v :: \varphi} \qquad \text{FAPPUNK} \quad \frac{}{f \ v :: \exists r. f(v, r)} \\
\\
\text{FIF} \quad \frac{e_1 :: \varphi_1 \quad e_2 :: \varphi_2}{(\mathbf{if} \ b \ \mathbf{then} \ e_1 \ \mathbf{else} \ e_2) :: (\mathbf{ens} \ [b = \mathbf{true}]; \varphi_1) \vee (\mathbf{ens} \ [b = \mathbf{false}]; \varphi_2)} \qquad \text{FDEREF} \quad \frac{}{!x :: \forall v. \mathbf{req} \ x \mapsto v \quad \mathbf{ens} \ r. [r = v] * x \mapsto v} \\
\\
\text{FASSIGN} \quad \frac{}{(\ell := v) :: \mathbf{req} \ x \mapsto v \quad \mathbf{ens} \ x \mapsto \ell} \qquad \text{FREF} \quad \frac{}{(\mathbf{ref} \ v) :: \exists \ell. \mathbf{ens} \ r. [r = \ell] * \ell \mapsto v} \\
\\
\text{FASSERT} \quad \frac{}{\mathbf{assert} \ v :: \mathbf{req} \ [v = \mathbf{true}]}
\end{array}$$

Fig. 2.14. Specification assertions

stantiate it and use it to satisfy the premise.

The next rule, **FAPPUNK**, is completely trivial, yet perhaps the most illuminating as to the design of staged logic. If there is *no known specification* for a function f – indicated at different levels by the absence of a premise in the rule, and a variable being left in application position after substitution – we may simply leave it in the specification and continue. This neatly deals with the possibility that f is a function parameter, unknown, and/or unspecified. Utilizing the actual specification of f , if it is eventually provided, then becomes a concern of entailment proving.

The **FIF** rule models conditional branching as disjunction, with assumptions on the value of the conditional in both disjuncts. The **FDEREF**, **FASSIGN**, **FREF** rules specify heap-manipulating primitives. The **FASSERT** rule gives assertions the interpretation of requiring properties. Crucially, it cannot change

the heap due to the use of a pure predicate. Properties of heap data can still be stated by first reading them into variables.

Semantics In order to state and prove the soundness of the specification assertion rules, we must first give specification assertions a semantics. We begin with a basic (partial correctness) definition reminiscent of the semantics of Hoare triples, for now parameterized over program and specification environments.

Definition 2.2 (Validity under environments). *A specification assertion $E, \mathcal{E} \vdash e :: \varphi$ is valid under environments E and $\mathcal{E}$ iff for all h_1, h_2, v , if e evaluates to v under E, h_1, h_2 , then φ is satisfied by $\mathcal{E}, h_1, h_2, v$.*

$$E, \mathcal{E} \vdash e :: \varphi \triangleq \forall h_1 h_2 v.$$

$$(E, h_1, e \rightsquigarrow h_2, v) \Rightarrow (\mathcal{E}, h_1, h_2, v \models \varphi)$$

Intuitively, this says that given an execution of a program e , given intensionally as a big-step semantics relating some initial and final configurations, we must be able to show that the configurations are a model of the formula φ . In other words, every behaviour of e corresponds to one of φ , and the behaviors of φ overapproximate those of e .

The next step is to supply appropriate environments. We first need an assumption over the contents of the input environments E (mapping names to programs) and $\mathcal{E}$ (mapping names to specifications) such that executions that refer to something in an environment also correspond. The solution is to require that the environments are *compatible*, in the sense that their contents are also related by valid specification assertions.

Definition 2.3 (Environment compatibility).

$$\begin{aligned} \text{compatible}(E, \mathcal{E}) &\triangleq \\ &\forall x e f a. E(f) = (\lambda x. e) \Rightarrow \\ &\exists g. \mathcal{E}(f) = g \wedge \forall r. (E, \mathcal{E} \vdash e[a/x] :: g(a, r)) \end{aligned}$$

Finally, we close the definition by quantifying over environments.

Definition 2.4 (Specification assertion validity).

$$e ::: \varphi \triangleq \forall E \mathcal{E}. \text{compatible}(E, \mathcal{E}) \Rightarrow (E, \mathcal{E} \vdash e ::: \varphi)$$

With validity defined, we can state and prove the soundness of the system of rules.

Theorem 2.1 (Soundness of specification assertions). *If $e ::: \varphi$ is derivable using the rules of Fig. 2.14, then it is valid.*

Proof. By induction over the derivation of $e ::: \varphi$. □

History triples Staged logic [87] originally used a *history triple*, a ternary relation between two staged formulae and a program $\{ \varphi_h \} e \{ \varphi \}$, as a direct analogue to the Hoare triple. History triples add to specification assertions $e ::: \varphi$ the *history* φ_h of a program. We describe them in this section for posterity and completeness.

The rules are given in Fig. 2.15. There are two new structural rules. **HHISTFRAME** is analogous to the frame rule from separation logic (which still applies and is used internally, within heap assertions in **req** and **ens**, but only for program states), but for *program behaviour*. **HCONSEQ** is similar to **FCONSEQ**, but is contravariant in the history. **HAPPFUN** and **HAPPFIX** are very similar to their earlier counterparts, simply recursing on e , while **HVAL**, **HASSERT**, **HDEREF**, **HASSIGN**, **HREF**, and **HAPPUNK** are practically the same, the only difference being that they *append* some description of their effect to the history. The only rules that are stated differently are **HIF** and **HLET**. In **HIF**, we instead express the constraints on the condition as a precondition in the history. This is functionally the same thanks to **NREQPUREL** and **NENSPURER**, but results in a more succinct rule. In **HLET**, we use φ_1 as the history of e_2 instead, leaving sequencing implicit.

$\boxed{\{\varphi\}e\{\varphi\}}$	
$\frac{\text{HHISTFRAME} \quad \{\varphi_h\}e\{\varphi\}}{\{\varphi_f; \varphi_h\}e\{\varphi_f; \varphi\}}$	$\frac{\text{HCONSEQ} \quad \varphi_1 \sqsubseteq \varphi_3 \quad \varphi_4 \sqsubseteq \varphi_2 \quad \{\varphi_3\}e\{\varphi_4\}}{\{\varphi_1\}e\{\varphi_2\}}$
$\frac{\text{HVAL}}{\{\varphi_h\}v\{\varphi_h; \mathbf{ens} \ r.[r = n]\}}$	$\frac{\text{HASSERT}}{\{\varphi_h\}v\{\varphi_h; \mathbf{req} \ [v = \text{true}]\}}$
$\frac{\text{HDEREF}}{\{\varphi_h\}!x\{\varphi_h; \forall v. \mathbf{req} \ x \mapsto v \quad \mathbf{ens} \ r.[r = v] * x \mapsto v\}}$	$\frac{\text{HASSIGN}}{\{\varphi_h\}x := v\{\varphi_h; \mathbf{req} \ x \mapsto v_0 \quad \mathbf{ens} \ r.[r = v] * x \mapsto v\}}$
$\frac{\text{HREF}}{\{\varphi_h\}ref \ v\{\varphi_h; \exists \ell. \mathbf{ens} \ r.[r = \ell] * \ell \mapsto v\}}$	$\frac{\text{HIF} \quad \{\varphi_h; \mathbf{req} \ [b = \text{true}]\}e_1\{\varphi_1\} \quad \{\varphi_h; \mathbf{req} \ [b = \text{false}]\}e_2\{\varphi_2\}}{\{\varphi_h\}v\{\varphi_1 \vee \varphi_2\}}$
$\frac{\text{HLET} \quad \{\varphi_h\}e_1\{\varphi_1\} \quad \varphi_1[v] \quad \{\varphi_1\}e_2[v/x]\{\varphi_2\}}{\{\varphi_h\}let \ x=e_1 \ in \ e_2\{\varphi_2\}}$	$\frac{\text{HAPPFUN} \quad \{\varphi_h\}e[v/x]\{\varphi\}}{\{\varphi_h\}(\lambda x. e) \ v\{\varphi\}}$
$\frac{\text{HAPPFIX} \quad \{\varphi_h\}e[v/x][(\lambda f x. e)/f]\{\varphi\}}{\{\varphi_h\}(\lambda f x. e) \ v\{\varphi\}}$	$\frac{\text{HAPPUNK}}{\{\varphi_h\}f \ v\{\varphi_h; \exists r. f(v, r)\}}$

Fig. 2.15. History triples

Relation to specification assertions A specification assertion is simply a history triple with an empty history.

Lemma 2.1 (Empty history). $e ::: \varphi \Leftrightarrow \{\mathbf{empty}\}e\{\varphi\}$

As the history describes program behaviour that has already occurred, its result does not contribute to any staged formula φ derived from e , as [HRES](#) shows. This fact may be used to derive [HAPPEND](#), which shows that every specification assertion can be turned into an append-only history triple.

Soundness of history triples then directly follows from this and [Theorem 2.1](#).

$$\begin{array}{c}
\text{HRES} \\
\frac{\{\varphi_h; \mathbf{empty}\} e \{\varphi\}}{\{\varphi_h\} e \{\varphi\}} \\
\\
\text{HAPPEND} \\
\frac{e ::: \varphi}{\{\varphi_h\} e \{\varphi_h; \varphi\}}
\end{array}$$

The remaining triples, including the structural rules, are proved sound by semantic reasoning. To that end, we give a semantics for history triples, then state the soundness theorem.

Definition 2.5 (Validity of history triples).

$$\begin{aligned}
\{\varphi_h\} e \{\varphi\} &\triangleq \forall E \mathcal{E} h_0 h_1 h_2 \vee R. \\
&\mathcal{E}, h_0, h_1, R \models \varphi_h \Rightarrow \text{compatible}(E, \mathcal{E}) \Rightarrow \\
&(E, h_1, e \rightsquigarrow h_2, R) \Rightarrow (\mathcal{E}, h_0, h_2, R \models \varphi)
\end{aligned}$$

To explain the definition simply, the history φ_h constrains execution from some *pre-initial* state h_0 to the program's initial state h_1 . The program then executes from h_1 to h_2 . The description of the program φ includes the history, so it constrains execution from h_0 to h_2 , and the definition asserts that e and φ are related in the same way as in [Definition 2.2](#).

Theorem 2.2 (Soundness of history triples). *Every history triple derivable using the rules of [Fig. 2.15](#) is valid.* □

2.3 Entailment checking

We return in this section to the subject of checking entailments $\varphi_1 \sqsubseteq \varphi_2$. We first cover the key issues and principles required for constructing proofs, then cover the ideas required for automated proof search.

2.3.1 Rewriting and sequent rules

The first idea required is that there are really two flavours of entailment rules. We saw some properties of entailment earlier, in [Fig. 2.11](#), and most

are *rewriting rules*, which have $\sqsubseteq$ only “below the line”. For example, many of the earlier steps in [Section 2.1.2](#) went by rewriting, using rules such as [BIAB](#).

The other kind of rule is a *sequent rule* – it has exactly one $\sqsubseteq$ premise, and can be seen as a *transformation* of the entire sequent, usually to lift things into the metalogic. An example is [REQCONTRA](#), which simplifies the proof state by reducing a matching **req** prefix to a separation logic entailment. More examples are shown in [Fig. 2.16](#).

$$\begin{array}{ccc}
 \text{EFORALLR} & \text{EFORALLL} & \text{EREQR} \\
 \frac{\forall x. (\varphi_1 \sqsubseteq \varphi_2)}{\varphi_1 \sqsubseteq \forall x. \varphi_2} & \frac{\exists x. (\varphi_1 \sqsubseteq \varphi_2)}{\forall x. \varphi_1 \sqsubseteq \varphi_2} & \frac{\text{ens } H; \varphi_1 \sqsubseteq \varphi_2}{\varphi_1 \sqsubseteq \mathbf{req} \ H \ \varphi_2}
 \end{array}$$

Fig. 2.16. Examples of sequent rules

While in theory some sequent rules can also be used for rewriting, in practice it is useful to consider them separately. Rewriting rules must be applicable regardless of context – we discuss how to guarantee this later, in [Section 6.1.4](#) – while sequent rules apply to goals of a fixed form.

Finally, rewriting and sequent rules work in tandem. For example, to lift a quantifier from staged logic into the metalogic, we must technically iterate [NFORALLDISTR](#) to move it to the left before [EFORALLR](#) can lift it out.

2.3.2 Simplification to a normal form

Many rewriting rules can always be fruitfully applied, and iterated until they no longer change the goal. We call them *simplification rules*, denoted $\varphi_1 \rightsquigarrow \varphi_2$. Semantically these are just entailments, but are *oriented*, intended to be used on the antecedent in a left-to-right direction. The whole system of rules is manually ensured to be confluent and terminating.

The important rules are given in [Fig. 2.17](#). Generally, they aim to widen the scope of quantifiers, associate sequences and **req** to the right, distribute

$$\boxed{\varphi \rightsquigarrow \varphi}$$

$$\begin{array}{ll}
\varphi; (\forall x. \varphi_1) \rightsquigarrow \forall x. \varphi; \varphi_1 & \varphi; (\exists x. \varphi_1) \rightsquigarrow \exists x. \varphi; \varphi_1 \\
(\forall x. \varphi); \varphi_1 \rightsquigarrow \forall x. \varphi; \varphi_1 & (\exists x. \varphi); \varphi_1 \rightsquigarrow \exists x. \varphi; \varphi_1 \\
(\varphi_1; \varphi_2); \varphi_3 \rightsquigarrow \varphi_1; \varphi_2; \varphi_3 & (\mathbf{req} \varphi \varphi_1); \varphi_2 \rightsquigarrow \mathbf{req} \varphi (\varphi_1; \varphi_2) \\
\varphi; (\varphi_1 \vee \varphi_2) \rightsquigarrow \varphi; \varphi_1 \vee \varphi; \varphi_2 & (\varphi_1 \vee \varphi_2); \varphi \rightsquigarrow \varphi_1; \varphi \vee \varphi_2; \varphi \\
\mathbf{ens} H * [P] \rightsquigarrow \mathbf{ens} [P]; \mathbf{ens} H & \mathbf{ens} [P_1]; \mathbf{ens} [P_2] \rightsquigarrow \mathbf{ens} [P_1 \wedge P_2] \\
\mathbf{ens} H_1; \mathbf{ens} H_2 \rightsquigarrow \mathbf{ens} (H_1 * H_2) & \mathbf{req} H_1; \mathbf{req} H_2 \varphi \rightsquigarrow \mathbf{req} (H_1 * H_2) \varphi
\end{array}$$

Fig. 2.17. Select simplification rules

all connectives over disjunction, and merge **ens** and **req** as much as possible while moving pure **ens** assumptions to the left. The goal of simplification is reach a sort of disjunctive normal form of sequences, where the head connective of each sequence in the antecedent φ_a can guide the proof search.

$$\bigvee (\mathbf{ens} H;)? \varphi_a \sqsubseteq \varphi$$

Notably, the normal form has a **ens** H prefix in the antecedent. This is useful to contain separation logic assertions, which cannot be directly lifted to the metalogic, by e.g. rules like **NREQPUREL**; in this sense it is similar in role to the *spatial context* [126] in Iris. The rules are designed to manipulate and maintain this context. For example, **EREQR** normalises **req** assumptions into this form.

The original paper on staged logic [87] identified the following stronger normal form and used it to define an entailment checking procedure.

$$\bigvee (\mathbf{req} H \mathbf{ens} Q; f(v, r);)^* \mathbf{req} H \mathbf{ens} Q \sqsubseteq \varphi$$

While it is achievable in the current setting, it no longer is with the more general kinds of effects described in subsequent chapters. We thus mention it here only for posterity.

$$\frac{\text{NTRANSPOSE} \quad H_A * H_1 \vdash H_2 * H_F}{\text{ens } H_1; \text{req } H_2 \varphi \sqsubseteq \text{req } H_A (\text{ens } H_F; \varphi)}$$

Fig. 2.18. Biabduction

2.3.3 Biabduction

A key rewriting rule is **NTRANSPOSE** (Fig. 2.18), presented earlier informally as **BIAB**. Intuitively, it allows an **ens** to be “pushed through” a **req**, a situation commonly encountered in the verification conditions that arise from function calls in programs – imagine H_1 as describing the state before a function call, and H_2 as the precondition of the function. This can be done by solving a *biabduction* [50] problem, a generalization of heap entailment which infers an *antiframe* H_A in addition to the frame H_F . [Example 2.1](#) demonstrates this, with the inferred antiframe and frame shown.

Example 2.1 (Biabduction).

$$\frac{[i = 3] * (x \mapsto 3 * y \mapsto 2) \vdash (x \mapsto i) * (y \mapsto 2)}{\text{ens } (x \mapsto 3 * y \mapsto 2); \text{req } (x \mapsto i) \varphi \sqsubseteq \text{req } [i = 3] (\text{ens } y \mapsto 2; \varphi)}$$

The resulting frame and antiframe may be simpler due to the common parts (the points-to on x in the example) having been eliminated. Either way, swapping the **req** and **ens** will bring them into closer proximity with other **req** and **ens**, allowing them to be merged. When iterated, this effectively *symbolically executes* a sequence of separation logic state transitions.

NTRANSPOSE is not an equivalence, as solutions to (bi)abduction problems are not unique [51]. In principle, this may lead to backtracking during proof search. In practice, the best solution can usually be found for symbolic heaps.

$$\boxed{\varphi \sqsubseteq \varphi}$$

$$\begin{array}{c}
\text{EBASEENS} \quad \frac{D_1 \vdash D_2}{\mathbf{ens} D_1 \sqsubseteq \mathbf{ens} D_2} \quad \text{EBASEREQ} \quad \frac{D_2 \vdash D_1}{\mathbf{req} D_1 \sqsubseteq \mathbf{req} D_2} \quad \text{ESEQ} \quad \frac{\varphi_1 \sqsubseteq \varphi_3 \quad \varphi_2 \sqsubseteq \varphi_4}{\varphi_1; \varphi_2 \sqsubseteq \varphi_3; \varphi_4} \\
\\
\text{EPUREENS} \quad \frac{\pi \Rightarrow (\varphi_1 \sqsubseteq \varphi_2)}{\mathbf{ens} \pi; \varphi_1 \sqsubseteq \varphi_2} \quad \text{EPUREREQ} \quad \frac{\pi \Rightarrow (\varphi_1 \sqsubseteq \varphi_2)}{\varphi_1 \sqsubseteq \mathbf{req} \pi; \varphi_2} \quad \text{ECANCELFN} \quad \frac{x_1=x_2 \quad (r_1=r_2 \Rightarrow \varphi_1 \sqsubseteq \varphi_2)}{f(x_1, r_1); \varphi_1 \sqsubseteq f(x_2, r_2); \varphi_2} \\
\\
\text{EWFIND} \quad \frac{f(x, r) \sqsubseteq \varphi_f \quad (\forall x_0 r_0. x_0 \leq x \Rightarrow f(x_0, r_0) \sqsubseteq \varphi[x_0/x][r_0/r]) \Rightarrow \varphi_f \sqsubseteq \varphi}{f(x, r) \sqsubseteq \varphi} \quad \text{EFORALLL} \quad \frac{\exists x. (\varphi_1 \sqsubseteq \varphi_2)}{(\forall x. \varphi_1) \sqsubseteq \varphi_2} \\
\\
\text{EDISJL} \quad \frac{\varphi_1 \sqsubseteq \varphi_3 \quad \varphi_2 \sqsubseteq \varphi_3}{(\varphi_1 \vee \varphi_2) \sqsubseteq \varphi_3} \quad \text{EDISJR} \quad \frac{\varphi_3 \sqsubseteq \varphi_i \quad i \in \{1, 2\}}{\varphi_3 \sqsubseteq (\varphi_1 \vee \varphi_2)} \quad \text{EFORALLR} \quad \frac{\forall x. (\varphi_1 \sqsubseteq \varphi_2)}{\varphi_1 \sqsubseteq (\forall x. \varphi_2)}
\end{array}$$

Fig. 2.19. Essential sequent rules

2.3.4 Proof search

We now have all the ingredients required to define a syntax-directed entailment checking procedure. This procedure cannot be complete, as φ may contain arbitrary propositions. We thus focus on what is needed to handle a practical fragment.

Algorithm 1: Entailment proof search

Function *search* () : unit tactic

 unfold » simpl » reduction » lemmas » entl;

end

Function *entl* () : unit tactic

 base <|> lift_pure <|> biab <|> cancel <|> induction <|>
 quantifiers <|> disj <|> fail;

end

The approach in this section extends and generalises the one we proposed originally [87], lifting the requirement for a normal form, and simplifying the argument for its soundness, by the use of only standard entailment and rewriting instead of a syntactic sequent for propagating frames.

[Algorithm 1](#) is structured as a pair of mutually-recursive functions in a tactic monad $'a \text{ tactic}$, a function of type $pstate \rightarrow ('a \times pstate) \text{ iter}$. $pstate$ is the *proof state*, which contains things like assumptions, the current goal, and so on; it may be thought of as abstract for our purposes. $iter$ is some abstract type of lazy list, allowing nondeterministic search. The search tree is constructed by the interleaved execution of the two functions, where a call to *search* occurs at every node, which performs simplification, unfolding of nonrecursive definitions, simplification ([Section 2.3.2](#)), and use of lemmas, followed by branching using the *alternative* constructor $<|>$; each of the *entl* cases applies one of the entailment rules ([Fig. 2.19](#)) before recursively calling *search* with a simplified goal.

The rules [EBASEENS](#) and [EBASEREQ](#) serve as the base cases of this process, applying when the goal can be reduced to a (covariant or contravariant) separation logic entailment. Pure assumptions may be lifted to the context by [EPUREENS](#) and [EPUREREQ](#). Functions may be handled by unfolding, or in an uninterpreted manner by [ECANCELFN](#) – where, if their inputs are equal, they are simply removed on both sides. If they are recursive, well-founded induction may apply ([EWFIND](#)), where an induction hypothesis is constructed and made available in the context of the new goal. Recursive functions must otherwise be handled by user-provided lemmas. [ESEQ](#) is applied modulo associativity of sequencing. [EDISJL](#), [EDISJR](#), [EFORALLL](#), and [EFORALLR](#) lift logical connectives to the metalogic; dual ones apply for conjunction and existentials. Heuristics are used to find constants for instantiating staged logic quantifiers; they mainly apply to quantifiers produced by [Section 2.2.2](#). Quantifiers within embedded separation logic assertions are left where they are and given to the SMT solver, giving users some control over how they are to be solved.

This proof search algorithm is implemented in a prototype tool named

HEIFER⁴, a push-button verification tool which takes programs written in a subset of OCaml, annotated with specifications and lemmas, and verifies them automatically. For programs without annotations, it verifies them in a greybox manner by applying the rules of [Section 2.2.2](#) to derive specifications.

2.4 Evaluation

To evaluate the effectiveness of HEIFER’s approach, we verified a suite of programs involving higher-order functions and closures ([Table 2.1](#)). As the focus of our work is to explore a new program logic and subsumption-based verification methodology (rather than to verify existing programs), the benchmarks are small in size, and are meant to illustrate the style of specification and give a taste of the potential for automation. We thus selected programs that demonstrated interesting and varied use cases for imperative higher-order functions. A few were taken from the literature and adapted as necessary to our setting, with care taken to remain faithful to the spirit of each example. Those without references were created by us.

Table 2.1
Comparison with Cameleer and Prusti

Benchmark	Heifer				Cameleer [158]			Prusti [206]		
	LoC	LoS	T	T_p	LoC	LoS	T	LoC	LoS	T
map	13	11	0.66	0.58	10	45	1.25	N/A		
map_closure	18	7	1.06	0.77		✗		N/A		
fold	23	12	1.06	0.87	21	48	8.08	N/A		
fold_closure	23	12	1.25	0.89		✗		N/A		
iter	11	4	0.40	0.32		✗		N/A		
compose	3	1	0.11	0.09	2	6	0.05	N/A		
compose_closure	23	4	0.44	0.32		✗		✗		
closure [191]	27	5	0.37	0.27		✗		13	11	6.75
closure_list	7	1	0.15	0.09		✗		N/A		
applyN	6	1	0.19	0.17	12	13	0.37	N/A		
blameassgn [84]	14	6	0.31	0.28		✗		13	9	6.24
counter	16	4	0.24	0.18		✗		11	7	6.37
lambda	13	5	0.25	0.22		✗		N/A		
	197	73			45	112		37	27	

⁴Higher-order Effectful Imperative Function Entailments and Reasoning

Table 2.1 provides an overview of the benchmark suite. The first two sub-columns show the size of each program (LoC) and the number of lines of user-provided specifications (LoS) required. The next two give the total wall-clock time taken T (in seconds) to verify all functions in each program against the provided specifications, and the amount of time T_p spent in external provers. Programs known to be natively inexpressible in each prover are marked with “X”. Programs that could not be verified for technical reasons and are not known to be inexpressible are marked with “N/A”.

The next column shows the same programs verified using Cameleer, a state-of-the-art deductive verifier [158]. Cameleer serves as a good baseline for several reasons: it is representative of the Hoare logic approach to automated verification, and, like HEIFER, targets a subset of OCaml. The most significant differences between Cameleer and HEIFER are that Cameleer does not support *effectful* higher-order functions and is intended to be used via the Why3 IDE in a semi-interactive way (allowing tactic-like *proof transformations*, used in the above programs) rather than in a push-button manner like HEIFER.

The last column shows results for Prusti [206]. Despite the major difference in Rust’s ownership type system, we compare against Prusti because of its state-of-the-art support for mutable closures, highlighting caveats below. While we were able to reproduce the claims made in Prusti’s OOPSLA 2021 artifact [10], we were not able to verify many of our own benchmark programs due to two technical reasons, namely lacking support for Rust’s `impl Trait` (to return closures) and ML-like cons lists (which caused timeouts and crashes). Support for closures is also not yet in mainline Prusti [12]. Nevertheless, we verified the programs we could use for the artifact.

User annotations required. Significantly less specification than code is required in Heifer, with an average LoS/LoC ratio of 0.37. This is helped by

two things: the ability to specify unknown functions in specifications, and the use of biabduction, which allows the specifications of many functions to be automatically derived, requiring only auxiliary lemmas to be provided. In contrast, Cameleer’s ratio is 2.49, due to the need to adequately summarise the behaviors of the function arguments and accompany these summaries with invariants and auxiliary lemmas. Two examples illustrating this are detailed in [Appendix A](#). Prusti’s ratio is 0.73, but a caveat is that in the programs for it, only closure reasoning was used, without lemmas or summarization.

Proof automation for HEIFER The time taken by external provers (column T_p of [Table 2.1](#)) is low, indicating that proof automation is feasible for the small (but representative) higher-order programs we used in our benchmark. The remaining time $T - T_p$ is spent on the other proof steps required to reduce programs to staged formulae ([Section 2.2.2](#)), simplification ([Section 2.3.2](#)), reduction to a pure verification condition, as well as the application of structural rules, induction, etc. ([Section 2.3](#)).

Expressiveness HEIFER is able to express many programs that Cameleer cannot, particularly closure-manipulating ones. This accounts for the ✗ rows in [Table 2.1](#). While some of these can be verified with Prusti, unlike stages, Prusti’s call descriptions do not capture ordering [[75](#), [10](#)]; an explicit limitation as shown by the ✗ rows in Prusti’s column. Prusti is able to use history invariants and the ownership of the Rust type system, but this difference is more than mitigated in HEIFER with the adoption of an expressive staged logic with spatial heap state, more appropriate for the weaker (but more general) type system of OCaml.

Implementation HEIFER is agnostic to the SMT solver used, relying on both Z3 [[70](#)] and Why3 [[83](#)] (with Z3, Alt-Ergo [[139](#)], and CVC4 [[29](#)] as backends) and switching between them dynamically, depending on the needs of

queries. Why3 is more fully-featured, with an extensive standard library, while Z3 is faster for the theories it supports natively.

Experimental setup All experiments were performed on macOS using a 2.3 GHz Quad-Core Intel Core i7 CPU with 16 GB of RAM. Why3 1.7.0 was used, with SMT solvers Z3 4.12.2, CVC4 1.8, and Alt-Ergo 2.5.2. The Prusti artifact, a Docker image, was run using Moby 25.0.1.

Data availability All benchmark programs and the source code of HEIFER are available on GitHub⁵.

⁵<https://github.com/hipsleek/Heifer>

VERIFYING PROGRAMS WITH EFFECT HANDLERS

3.1 Introduction

Algebraic effects were introduced by Plotkin and Power [163] as a means of reasoning about computational effects *algebraically*, using equations relating different operations. *Effect handlers* [164] later enabled providing user-defined interpretations for effects. They proved to be not just elegant but practical, eventually finding their way into OCaml 5.0 [179] as the main means of structuring concurrency.

This chapter begins with a gentle introduction to effect handlers and related concepts. I then survey briefly how verification tools treat them today before covering how they are treated in staged logic.

<pre> type _ Effect.t += E: string -> int t let client () = a (); let x : int = perform (E "?") in b () </pre>	<pre> let main () = match client () with effect (E y), k -> c (); let z = continue k 42 in e () v -> d (); v </pre>
(a) Client	(b) Handler

Fig. 3.1. Effect handlers

Fig. 3.1 illustrates how an effect handler works in OCaml. To help explicate the control flow of the example, functions are called in the order a, b, c, d, e, and variables are bound in the order x, y, z.

We start on the left, with the declaration of an effect E. Like an exception type, it extends an extensible variant `Effect.t`. It has an optional input type to the left of the arrow, and an output type to the right.

Below it, we have a program which *performs* effects, using the primitive **perform**. We refer to this program as the *client*. At this point, *E* is *uninterpreted*: we don't know what it does, only that it accepts a string from the client and returns to it an integer.

On the right is the entry point of the program, *main*, which invokes the client under a **match**, the syntax for an effect handler. Operationally, an effect handler may be seen as a generalisation of an exception handler which also gets access to a *delimited continuation* *k*. The **perform** thus causes a jump to the *effect case* of the handler, where *y* in the handler is bound to "?", and *k* is bound to (a reification of) **let** *x* : *int* = $\square$ **in** *b* (). It should be clear now why *k* is called a delimited continuation: it consists of the rest of the computation of *client* following the **perform**, delimited by the extent of the **match**, and it can be *resumed* by plugging some value into the $\square$.

This particular handler resumes the continuation using **continue**, calling functions *c* and *e* before and after. Notably, the continuation is only resumed once, and is thus called *one-shot*. Control goes back to the client, where *x* is bound to 42. *b* is then invoked, and at some point after, the client returns its result.

Another notable point is that the handler remains installed over resumption of continuations, handling subsequent effects that arise from their invocation; it is a *deep* handler. Because of this, the value the client returns is then bound to *v* in the second, *value case* of the **match**.

Finally, when the value case returns, its value is bound to *z* in the first case of the match, and *e* is executed. The intuition for this is that **continue** may be thought of as similar to a function call in the way it uses the call stack. A resumption of a continuation thus later returns to the point of resumption, and continues with whatever comes after.

Uses of effects Effect handlers are compelling because they are a structured means of programming with (delimited) continuations, which in turn provides a way to extend a language with user-defined computational effects. Effect handlers also allow abstraction, by decoupling an effectful program from its denotation: in Fig. 3.1, the meaning of E only became known given a handler, and one could imagine it being invoked under different handlers for e.g. testing, instrumentation, or simply begin given an alternative semantics.

Deep and shallow handlers In Fig. 3.1, we used a *deep* handler, which continues to apply to resumptions of continuations. It is also possible to work with *shallow handlers*, which handle only one effect and require a new handler for each resumption. This enables easier expression of some kinds of programs, including dynamic sequences of handled effects [6] and generators [9]. Semantically, a deep handler is a fold over a computation tree of trees, while a shallow handler is a case split [98].

Shallow handlers are supported in OCaml through the library functions `continue_with`, where a handler is associated with a resumption rather with a computation, and `fiber`, to lift a computation into a continuation, to start the process.

In our examples, we focus mostly on deep handlers, as they are the default in OCaml [1]. However, we also demonstrate that our work is applicable to shallow handlers, discussing provisions for them as needed. It is not difficult in principle to support reasoning for both, as they are inter-derivable [98], so one can be taken as primitive and the other as derived. Prior work has taken shallow handlers as primitive, as defining a deep handler in terms of a shallow one is easier [71].

Zero-shot, one-shot, & multishot continuations In Fig. 3.1, the handler used k in a manner that made it a *one-shot* continuation. It is of course also

possible to write handlers which do not use the continuation at all – making them *zero-shot*, and essentially turning the effect into an exception – or use them multiple times, making them *multishot*. The latter are interesting because they admit both new use cases – such as nondeterminism, probabilistic programming [123], a Unix-style fork [136], or continuation-based web servers [145] – as well as new challenges, which we go into shortly.

3.1.1 Reasoning about effect handlers

To illustrate the current paradigm for specifying and verifying effect-handling programs, consider the following example, which has been discussed in multiple papers [181, 72].

<pre> type _ Effect.t += XCHG: int -> int Effect.t let exchange n = perform (XCHG n) let client () = exchange (exchange 42) </pre>	<pre> let main () = let p = ref 0 in try client () with effect XCHG n, k -> let old_p = !p in p := n; continue k old_p </pre>
(a) Client	(b) Handler

Fig. 3.2. XCHG effect

The XCHG effect abstracts away the content of a mutable reference and offers clients a single operation on it: a client may update its value, receiving a reply containing the old value. The implementation of the handler (Fig. 3.2, right) expresses this directly. The **try** syntax abbreviates a **match** with an identity value case.

A client which exchanges twice (Fig. 3.2, left) should naturally get back the value they sent, causing no overall change to the state of the system. That seems simple enough to confirm intuitively for this program, but how can we specify and verify this in general?

Consider what we will require to verify the client modularly: eventually we will need a specification for the **perform** of XCHG. The same goes for the handler and its use of **continue** on the continuation *k*. Furthermore, we will need a way to treat the handler in a way that expresses its deep behaviour.

de Vilhena and Pottier [72]’s insight is that the specifications of the continuation and the **perform** are dual, and can be expressed in the form of a *protocol*: essentially a contract for **performing** a particular effect which both client and handler adhere to. It “describes functionality on which the client can rely, and which the handler must implement” [71]. Soares and Pereira [181] augment this with the idea of a *handler invariant*¹, which a deep handler both preserves and ensures as it recursively handles effects.

```

protocol XCHG x:
  ensures !p = x && reply = old !p
  modifies p

let exchange n = perform (XCHG n)
(*@ ensures !p = n && result = old !p
   performs XCHG *)

try client () with ...
(*@ try_ensures !p = old !p && result = 42 *)

```

Fig. 3.3. Cameleer specifications for XCHG

Fig. 3.3 shows the specifications needed in GOSPEL, which Soares and Pereira use in their automated verification setting. First is the protocol, which pertains to a particular effect. Here it consists of a relational postcondition that allows the client to know what **performing** XCHG will do. Verification of exchange is immediate given the protocol, and verification of `client` is easy, by composing the postcondition twice.

Next, we verify `main`, focusing on the effect case first. Due to the afore-

¹de Vilhena and Pottier originated the idea, but present it in more general terms as a combination of two features: (1) a guarded-recursive assumption containing the specification of a *deep-handler*, (2) with the postcondition universally quantified, allowing a user to instantiate it.

mentioned duality, the postcondition of the protocol is really the *precondition* of the continuation: we must check that the handler body prior to the resumption actually implements the functionality specified by the protocol, that it sets the reference to the correct value, etc. Correspondingly, the precondition of the protocol can be assumed to hold at the start of the handler body.

Moving on, what should the postcondition of the continuation be? The answer is that it should be the *handler invariant*. Seeing as the effects of the continuation are recursively handled, we can assume the handler invariant after the resumption and must re-establish it before the end of the handler body, much like the use of the postcondition of a recursive call. Because the resumption is the last thing in the handler body, this is immediate.

Finally, the (elided, identity) value case simply involves checking that the postcondition of `client` entails the handler invariant. This is also immediate.

Multishot continuations and context-locality The protocol-based solution is elegant, and adequate for stock OCaml. However, OCaml may be extended [5] with multishot continuations. It has been noted that they enable some applications and make others notably simpler, and that more research is required to understand how best to support them [61].

The protocol approach (and more generally, Hoare logic) is notoriously difficult to extend to this setting. The problems may be seen in the following example from de Vilhena [71], in Fig. 3.4.

```
let call f b =
  b := 0;
  f ();
  b := !b + 1;
  assert (!b = 1)
```

Fig. 3.4. A problematic example with multishot continuations

Assuming we have a specification $\{true\} f() \{r. r=()\}$ for f , it seems

obvious on first reading that the assertion cannot fail. The *bind rule* allows us to compose the specification of f with triples about the following statements to derive a triple about the whole program. The *frame rule* tells us that f does not have access to b , so the value of b must still be 0 after the call to f .

With multishot continuations, however, it is not possible to guarantee that the assertion holds. The reason is that f may capture its continuation and resume it multiple times, so 0, 1, and 2 are all possible values of b .

How does this conflict with the rules we normally take for granted?

One reason is that we cannot reason *context-locally* about f , i.e. reason about f and its context independently as the standard bind rule promises, at least without considering its effects. This is because f may capture its context and use it in an unrestricted way. Timany and Birkedal [198] noted this problem, which de Vilhena and Pottier [72] fixed with a weakest precondition predicate augmented with a context-local description of effects.

Another reason is that even with this fix, the frame rule also no longer applies because of how continuations can capture resources [72]. This problem has shown up many times in the literature, with different solutions. Delbianco and Nanevski [74] use a separation logic without a general frame rule; programs which preserve other parts of the heap must explicitly quantify over it in their specifications. Timany and Birkedal [198] distinguish *context-local* from *non-context-local* triples; the former allow local reasoning when there are no control operators, and the latter allow global reasoning essentially based on the operational semantics. de Vilhena [71] restricts the frame rule so that when an effect is performed, resources disappear and cannot be framed around uses of effects.

A theme common to all prior solutions is that at some point, local reasoning simply has to be restricted, with a possible fallback to global reasoning. With this in mind, the position we take is that it is fundamentally challenging to invent abstractions for modular reasoning about multishot continuations,

and we cannot truly have it without a first-class treatment of effects, i.e. *in the logic*. The solution we present next is guided by this intuition.

3.1.2 Effect handlers in staged logic

The difficulty of coming up with an abstraction might remind the reader of the issue addressed in the previous chapter (Section 2.1), and in a sense the problem and solution are the same.

The idea is to model effects and handlers directly in the logic, stating specifications using refinement. We may then reason about them using standard principles, such as rewriting and induction. Crucially, this retains context-locality: it is a modular approach in that one can always state a specification about a piece of code, and combine two specifications, with the tradeoff being that this statement may be without much abstraction at first. Nevertheless, abstraction can always be recovered when it is easier to do so.

To illustrate things concretely, we first explain how to verify the program in Fig. 3.4, then the one in Fig. 3.2. We end the section by concluding the narrative of Section 2.1 regarding exceptions.

First, the program in Fig. 3.4. We need a specification for `call`. For ease of presentation, we replace the call to the unknown function `f` with an effect `E`, to focus on the problematic multishot case, and define **assert** $D \triangleq \mathbf{req} D; \mathbf{ens} r. D \wedge r = ()$.

$$call \triangleq \forall b. \mathbf{ens} b \mapsto 0; E; \forall v. \mathbf{req} b \mapsto v; \mathbf{ens} b \mapsto v+1; \mathbf{assert} b \mapsto 1$$

The specification describes the state changes that occur on either side of the effect.

We would like to prove functional correctness: the assertion fails or does not depending on the kind of handler used and how it resumes the continuation, and if it does not fail, what the result should be.

This can be stated as the following three refinements.

$$\mathbf{try\ call\ with}\ E, k \rightarrow -1 \sqsubseteq \forall b. \mathbf{ens}\ r. b \mapsto 0 \wedge r = -1$$

$$\mathbf{try\ call\ with}\ E, k \rightarrow k() \sqsubseteq \forall b. \mathbf{ens}\ r. b \mapsto 1 \wedge r = ()$$

$$\mathbf{try\ call\ with}\ E, k \rightarrow k(); k() \sqsubseteq \mathbf{req\ false}$$

These entailments are simple enough to be automatically proved by symbolic execution. The proof for the one-shot case is as follows. Each step is annotated with (informal) justification for how to reach it from the previous step.

$$\begin{aligned} & \mathbf{try\ call\ with}\ E, k \rightarrow k() \sqsubseteq \forall b. \mathbf{ens}\ r. b \mapsto 1 \wedge r = () \\ \sqsubseteq & \mathbf{try}(\forall b. \mathbf{ens}\ b \mapsto 0; E; \dots) \mathbf{with}\ E, k \rightarrow k() && \text{Unfold} \\ \sqsubseteq & \forall b. \mathbf{ens}\ b \mapsto 0; \mathbf{try}(E; \dots) \mathbf{with}\ E, k \rightarrow k() && \text{Float} \\ \sqsubseteq & \forall b. \mathbf{ens}\ b \mapsto 0; \forall v. \mathbf{req}\ b \mapsto v; \mathbf{ens}\ b \mapsto v+1; \mathbf{assert}\ b \mapsto 1 && \text{Handle} \\ \sqsubseteq & \forall b. \mathbf{ens}\ b \mapsto 0+1; \mathbf{assert}\ b \mapsto 1 && \text{Biab} \\ \sqsubseteq & \forall b. \mathbf{ens}\ b \mapsto 1 \wedge r = 0 \quad \square && \text{Biab} \end{aligned}$$

Float refers to the idea of letting pure prefixes of programs *float* out of handlers, to reduce their scope; Handle refers to simplification driven by the operational semantics of handlers; and Biab refers to the use of biabduction to simplify sequences of **req** and **ens**.

This example demonstrates the ability to reason simply and precisely about what happens with each kind of handler. As a protocol is not needed, there is no need to commit to an abstraction of the semantics for a particular handler, or try to abstract over all of them; the effect remains uninterpreted, as it was in the program, until a particular handler is encountered. In a similar way as in [Section 2.1](#), *maintaining precision* has once again simplified our proof effort, opening the door to automation.

We close this section with a proof for χCHG , which also goes by symbolic execution.

We first model the client and handler.

$$\begin{aligned}
client &\triangleq XCHG(42); r. XCHG(r) \\
main &\triangleq \forall p. \mathbf{ens} \, p \mapsto 0; \mathbf{try} \, client \mathbf{with} XCHG(n), k \rightarrow \\
&\quad (\forall o. \mathbf{req} \, p \mapsto o; \mathbf{ens} \, p \mapsto n; k(o))
\end{aligned}$$

Notably, we have added a *bind* construct $\varphi_1; r. \varphi_2$ to the logic, which is essential for working with continuations (Section 3.2).

Then we prove the following entailment about their composition:

$$main \sqsubseteq \forall p. \mathbf{ens} \, r. p \mapsto 0 \wedge r=42$$

which is to say that the result is the original argument to the client, and the state of p remains unchanged.

The proof is as follows.

$$\begin{aligned}
&\mathbf{ens} \, p \mapsto 0; \mathbf{try} \, client \mathbf{with} XCHG(n), k \rightarrow \dots \\
\sqsubseteq &\mathbf{ens} \, p \mapsto 0; && \text{Handle} \\
&\mathbf{ens} \, k=(\lambda x. \mathbf{try} \, XCHG(x) \mathbf{with} XCHG(n), k \rightarrow \dots) \\
&\forall o. \mathbf{req} \, p \mapsto o; \mathbf{ens} \, p \mapsto 42; k(o) \\
\sqsubseteq &\mathbf{ens} \, k=(\lambda x. \mathbf{try} \, XCHG(x) \mathbf{with} XCHG(n), k \rightarrow \dots); && \text{Biab} \\
&\mathbf{ens} \, p \mapsto 42; k(0) \\
\sqsubseteq &\mathbf{ens} \, p \mapsto 42; \mathbf{try} \, XCHG(0) \mathbf{with} XCHG(n), k \rightarrow \dots && \text{Simplify} \\
\sqsubseteq &\mathbf{ens} \, p \mapsto 42; \forall o. \mathbf{req} \, p \mapsto o; \mathbf{ens} \, p \mapsto n; k(o) && \text{Handle} \\
\sqsubseteq &\mathbf{ens} \, r. p \mapsto 0 \wedge r=42 \quad \square && \text{Biab}
\end{aligned}$$

The reasoning goes very operationally, but at a high level of abstraction, using lemmas to take larger steps.

3.1.3 Exceptions and higher-order functions

In Section 2.1.1, we saw the issues with existing approaches for reasoning about higher-order functions and effects, with separation logic invariants providing only a partial solution for imperative behaviour, without an answer for other effects. In Section 2.1.2, staged logic was presented as an alternative, but for imperative effects only. Now that we have seen how effects are treated in staged logic, we conclude the earlier thread with an example illustrating reasoning about exceptions.

```

foldr (fun x t ->
  let r = x + t in
  if r < b
  then r
  else raise Failed) xs 0

```

Fig. 3.5. A use of foldr with exceptions

$$\begin{aligned}
&\{b > 0 \wedge \text{sum } xs < b\} \text{ foldr } \dots \text{ xs } 0 \{r. r = \text{sum } xs\} \\
&\{b > 0 \wedge \text{sum } xs \geq b\} \text{ foldr } \dots \text{ xs } 0 \{r. \text{Failed}; \mathbf{req} \text{ false}\}
\end{aligned}$$

Fig. 3.6. Two specifications for foldr

Consider the `foldr` function again. Previously, we proved a functional correctness result for using it compute the sum of a list of integers. Here, we add partiality to the mix: we assume some bound b which the sum should not exceed. When exceeded, `foldr` raises an exception to stop the computation².

For efficiency, we would like to stop as soon as b is exceeded, so we invoke `foldr` as in Fig. 3.5, checking each intermediate sum. For simplicity, we assume xs is a list of natural numbers, and $b > 0$.

How can we specify this function? Two triples are given in Fig. 3.6, for the different cases of whether the bound is exceeded. *Failed* is an effect that we use to model an exception. Having it in the specification ensures that *Failed* is raised. Furthermore, we follow it with the obligation to *prove false*, to express that *Failed* itself must *guarantee false*; in other words, there can be no meaningful behaviour after it.

As we will work in staged logic, we will reuse the *foldr* formula from Fig. 2.8; crucially, we will not have to write a new intermediate specification. We will also write the final specification using *intersection*, i.e. conjunction

²It is of course possible to model partiality using the option monad, which would allow the use of a standard logic. The tradeoff is the need for a (syntactically close but ultimately) different definition of `foldr` using binds, which is really a different program.

lifted to staged logic. The goal we would like to prove is thus:

$$\begin{aligned}
& \text{foldr}(g, \text{init}, xs) \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } xs < b; \mathbf{ens} \ r. r = \text{sum } xs \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } xs \geq b; \text{Failed}; \mathbf{req} \ \text{false} \\
& g \triangleq \lambda x \ t. \mathbf{ens} \ (x + t < b); \mathbf{ret} \ (x + t) \vee \mathbf{ens} \ (x + t \geq b); \text{Failed}
\end{aligned}$$

The proof goes by induction on xs .

$$\begin{aligned}
\text{Base case} \quad & \mathbf{ens} \ xs = []; \mathbf{ret} \ 0 \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } xs < b; \mathbf{ens} \ r. r = \text{sum } xs \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } xs \geq b; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

Simplifying,

$$\begin{aligned}
& \mathbf{ret} \ 0 \\
& \sqsubseteq \mathbf{req} \ b > 0; \mathbf{ens} \ r. r = 0 \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge b \leq 0; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

The second case is absurd, and the first is immediate.

$$\begin{aligned}
\text{Inductive case} \quad & \exists h \ t. \mathbf{ens} \ xs = h :: t; \text{foldr}(g, \text{init}, t); r. g(h, r) \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } xs < b; \mathbf{ens} \ r. r = \text{sum } xs \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } xs \geq b; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

Eliminating existentials and simplifying,

$$\begin{aligned}
& \text{foldr}(g, \text{init}, t); r. g(h, r) \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) < b; \mathbf{ens} \ r. r = \text{sum } (h :: t) \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) \geq b; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

We use the induction hypothesis.

$$\begin{aligned}
& (\mathbf{req} \ \text{sum } t < b; \mathbf{ens} \ r. r = \text{sum } t \wedge \mathbf{req} \ \text{sum } t \geq b; \text{Failed}); r. g(h, r) \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) < b; \mathbf{ens} \ r. r = \text{sum } (h :: t) \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) \geq b; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

Distributing sequencing over conjunction,

$$\begin{aligned}
& \mathbf{req} \ \text{sum } t < b; \mathbf{ens} \ r. r = \text{sum } t; r. g(h, r) \\
& \quad \wedge \mathbf{req} \ \text{sum } t \geq b; \text{Failed}; r. g(h, r) \\
& \sqsubseteq \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) < b; \mathbf{ens} \ r. r = \text{sum } (h :: t) \\
& \quad \wedge \mathbf{req} \ b > 0 \wedge \text{sum } (h :: t) \geq b; \text{Failed}; \mathbf{req} \ \text{false}
\end{aligned}$$

At this point we can split the goal and prove the two conjuncts independently,

which requires us to how to arrive at the two outcomes: either the sum of the list, or failure.

Case: sum of list The right conjunct on the left of the entailment clearly cannot contribute to the final outcome, so we drop it.

$$\begin{aligned} & \mathbf{req} \ (sum \ t < b); \mathbf{ens} \ r.r = sum \ t; r. \ g(h, r) \\ \sqsubseteq & \ \mathbf{req} \ (sum \ h::t < b); \mathbf{ens} \ r.r = sum \ (h::t) \end{aligned}$$

We can finally use the definition of g .

$$\begin{aligned} & \mathbf{req} \ (sum \ t < b); \mathbf{ens} \ r.r = sum \ t; r. \\ & (\mathbf{ens} \ (h + r < b); \mathbf{ret} \ (h + r) \vee \mathbf{ens} \ (h + r \geq b); \mathbf{Failed}) \\ \sqsubseteq & \ \mathbf{req} \ (sum \ h::t < b); \mathbf{ens} \ r.r = sum \ (h::t) \end{aligned}$$

After we simplify, we see that there two disjuncts.

$$\begin{aligned} & \mathbf{req} \ (sum \ t < b); \\ & (\mathbf{ens} \ (h + sum \ t < b); \mathbf{ret} \ (h + sum \ t) \vee \mathbf{ens} \ (h + sum \ t \geq b); \mathbf{Failed}) \\ \sqsubseteq & \ \mathbf{req} \ (sum \ h::t < b); \mathbf{ens} \ r.r = sum \ (h::t) \end{aligned}$$

We use the following lemma to eliminate the **reqs**.

Lemma 3.1. *Sum monotonic* $sum \ h::t < b \Rightarrow sum \ t < b$

The first case is immediate. The second case is absurd.

Case: failure It is possible for both conjuncts on the left of the entailment to result in failure: if a previous intermediate sum (from the recursive call) exceeded the bound, or if the current intermediate sum will exceed it (from g). We thus keep both cases.

$$\begin{aligned} & \mathbf{req} \ sum \ t < b; \mathbf{ens} \ r.r = sum \ t; r. \ g(h, r) \\ & \wedge \mathbf{req} \ sum \ t \geq b; \mathbf{Failed}; r. \ g(h, r) \\ \sqsubseteq & \ \mathbf{req} \ b > 0 \wedge sum \ (h::t) \geq b; \mathbf{Failed}; \mathbf{req} \ false \end{aligned}$$

[Lemma 3.1](#)

In order to know which case we are in, we need to know if $sum \ t \geq b$.

We thus proceed by case analysis.

Case: $sum \ t \geq b$ Failure must have come from the recursive call. We

thus drop the left conjunct.

$$\begin{aligned} & \mathbf{req} \text{ sum } t \geq b; \mathbf{Failed}; r. g(h, r) \\ \sqsubseteq & \mathbf{req} b > 0 \wedge \text{sum } (h::t) \geq b; \mathbf{Failed}; \mathbf{req} \text{ false} \end{aligned}$$

Eliminating **reqs**,

$$\begin{aligned} & \mathbf{Failed}; r. g(h, r) \\ \sqsubseteq & \mathbf{Failed}; \mathbf{req} \text{ false} \end{aligned}$$

This is immediate due to the **req false**.

Case: $\text{sum } t < b$

$$\begin{aligned} & \mathbf{req} \text{ sum } t < b; \mathbf{ens} r. r = \text{sum } t; r. g(h, r) \\ \sqsubseteq & \mathbf{req} b > 0 \wedge \text{sum } (h::t) \geq b; \mathbf{Failed}; \mathbf{req} \text{ false} \end{aligned}$$

Failure must have come from g . We eliminate the right conjunct, then examine the two disjuncts of g .

$$\begin{aligned} & \mathbf{req} \text{ sum } t < b; \mathbf{ens} r. r = \text{sum } t; r. (\mathbf{ens} (h + r < b); \mathbf{ret} (h + r) \\ & \quad \vee \mathbf{ens} (h + r \geq b); \mathbf{Failed}) \\ \sqsubseteq & \mathbf{req} b > 0 \wedge \text{sum } (h::t) \geq b; \mathbf{Failed}; \mathbf{req} \text{ false} \end{aligned}$$

Simplifying,

$$\begin{aligned} & \mathbf{ens} (h + \text{sum } t < b); \mathbf{ret} (h + \text{sum } t) \\ & \quad \vee \mathbf{ens} (h + \text{sum } t \geq b); \mathbf{Failed} \\ \sqsubseteq & \mathbf{req} b > 0 \wedge \text{sum } (h::t) \geq b; \mathbf{Failed}; \mathbf{req} \text{ false} \end{aligned}$$

The first case is absurd, and the second has the *Failed* effect that we need. □

Next, we cover the extensions to staged logic required to realise the reasoning in the above sections. A first-time reader may wish to skip ahead to [Section 4.1](#) to see how these techniques apply more generally to delimited continuation operators.

3.2 Extensions to staged logic

$$\begin{aligned}
\varphi &::= \mathbf{req} \ \sigma \ \varphi \mid \mathbf{ens} \ r. \sigma \\
&\mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \exists x. \varphi \mid \forall x. \varphi \\
&\mid \varphi_1; r. \varphi_2 \mid f(v) \mid \epsilon(v) \mid \mathbf{match}^\delta \varphi \mathbf{with} \mathcal{H} \\
\delta &::= s? \\
\mathcal{H} &::= x \rightarrow \varphi \mid \epsilon(v), k \rightarrow \varphi \mid \mathcal{H}
\end{aligned}$$

Fig. 3.7. Staged logic for effect handlers

We first extend the syntax of staged logic to be able to model effect handlers. The first two rows of Fig. 3.7, for specifications and logical connectives, are the same as in Fig. 2.9; we only extend the syntax with forms to model the new program elements.

Existential problems The first change is that we have generalised sequencing to *binding*. The notation $\varphi_1; r. \varphi_2$ alludes to this, denoting that the result of φ_1 is named r in φ_2 . The reason for this is that existentials are no longer adequate for modelling the results of programs using multishot continuations, which can return multiple times and thereby have *multiple results*; in contrast, an existential can be instantiated to exactly one result value. We resolve this by modelling the continuations of programs explicitly, allowing them to be taken multiple times with different values.

With `bind` in the language, we no longer have to write functions in relational form to be able to talk about their results, and so use the standard applicative form, bringing staged logic closer to the language of programs. This is more general – the relational form we used up to this point can be reinterpreted as $f(v, r); \varphi \triangleq f(v); r. \varphi$.

Effect handlers Next are effects $\epsilon(v)$ and handlers, which are imported as-is from the language of programs. The reason is that unknown functions can appear in the scrutinees of matches, and knowing nothing else about them,

we may have no choice but to leave them (and their context) in specifications, sans abstraction, in keeping with our proposed methodology.

Handlers may be shallow (indicated by the superscript s) or deep (no superscript, and the default). **try** can be interpreted as the following abbreviation:

$$\mathbf{try}^\delta \varphi_1 \mathbf{with} \epsilon(v), k \rightarrow \varphi_2 \triangleq \mathbf{match}^\delta \varphi_1 \mathbf{with} x \rightarrow \mathbf{ret} x \mid \epsilon(v), k \rightarrow \varphi_2$$

$$\boxed{\mathcal{E}, h, h, R \models \varphi}$$

$$\begin{array}{c}
R ::= v \mid \epsilon(v, \lambda x. \varphi_k) \\
\\
\begin{array}{ccc}
\text{SEFF} & \text{SSEQ} & \text{SBINDVAL} \\
\frac{R = \epsilon(v, \lambda r. \mathbf{ret} r)}{\mathcal{E}, h, h, R \models \epsilon(v)} & \frac{\mathcal{E}, h_1, h_3, R \models \varphi_1 \quad \mathcal{E}, h_3, h_2, R \models \varphi_2}{\mathcal{E}, h_1, h_2, R \models \varphi_1; \varphi_2} & \frac{\mathcal{E}, h_1, h_3, v \models \varphi_1 \quad \mathcal{E}, h_3, h_2, R \models \varphi_2[v/r]}{\mathcal{E}, h_1, h_2, R \models \varphi_1; r. \varphi_2} \\
\text{SBINDEFF} & \text{SUNHANDLED} & \\
\frac{\mathcal{E}, h_1, h_2, \epsilon(v, \lambda x. \varphi_k) \models \varphi_1 \quad R = \epsilon(v, \lambda x. \varphi_k; r. \varphi_2)}{\mathcal{E}, h_1, h_2, R \models \varphi_1; r. \varphi_2} & \frac{\mathcal{E}, h_1, h_2, \epsilon(v, \lambda x. \varphi_k) \models \varphi \quad \epsilon \notin \mathcal{H} \quad R = \epsilon(v, \lambda x. \mathbf{match} \varphi_k \mathbf{with} \mathcal{H})}{\mathcal{E}, h_1, h_2, R \models \mathbf{match} \varphi \mathbf{with} \mathcal{H}} & \\
\text{SHANDLE} & \text{SHANDLEVAL} & \\
\frac{\mathcal{E}, h_1, h_3, \epsilon(v, \lambda x. \varphi_k) \models \varphi \quad \epsilon(x), k \rightarrow \varphi_b \in \mathcal{H} \quad \varphi'_k = \mathbf{match} \varphi_k \mathbf{with} \mathcal{H} \quad \mathcal{E}[k := \lambda x. \varphi'_k], h_3, h_2, R \models \varphi_b[v/x]}{\mathcal{E}, h_1, h_2, R \models \mathbf{match} \varphi \mathbf{with} \mathcal{H}} & \frac{\mathcal{E}, h_1, h_3, v \models \varphi \quad x \rightarrow \varphi_b \in \mathcal{H} \quad \mathcal{E}, h_3, h_2, R \models \varphi_b[v/x]}{\mathcal{E}, h_1, h_2, R \models \mathbf{match} \varphi \mathbf{with} \mathcal{H}} & \\
\text{SHANDLES} & & \\
\frac{\mathcal{E}, h_1, h_3, \epsilon(v, \lambda x. \varphi_k) \models \varphi \quad \epsilon(x), k \rightarrow \varphi_b \in \mathcal{H} \quad \mathcal{E}[k := \lambda x. \varphi'_k], h_3, h_2, R \models \varphi_b[v/x]}{\mathcal{E}, h_1, h_2, R \models \mathbf{match}^s \varphi \mathbf{with} \mathcal{H}} & &
\end{array}
\end{array}$$

Fig. 3.8. Extended semantics of staged logic, on top of Fig. 2.10

Semantics The extensions to the semantics are shown in Fig. 3.8. First, we add the notion of an *outcome* R , inspired by the operational view of effects as a generalisation of exceptions – a formula φ may reduce to a value v , or a “unhandled” effect $\epsilon(v, \lambda x. \varphi_k)$, containing a payload v and a continuation. The latter allows us to operationalise the construction of the delimited

continuation that is conceptually built up around the effect as it “unwinds the stack”. **SEFF** shows how this process begins – an unhandled effect on its own simply stands for itself, and has an intermediate identity continuation. The **SSEQ** rule from before is superseded by two new rules, **SBINDVAL** and **SBINDEFF**, which distinguish the outcome of φ_1 . The former is simply a generalisation of the original rule to binding. The latter picks up after **SEFF**: given an unhandled effect, it extends its continuation with the *current* continuation, which is made explicit by the bind. The next two rules show what happens when an unhandled effect finally makes it to a handler. If the effect remains unhandled, as in **SUNHANDLED**, the enclosing handler continues to apply to the continuation; this is true even for shallow handlers, which vanish only after a particular effect is handled. If the effect is handled, however (**SHANDLE**), execution continues with the appropriate branch of the handler, with the continuation bound to the intermediate continuation of ϵ , which is at this point is completely known and delimited. The continuation is placed under an identical handler, implementing the semantics of deep handlers.

Shallow handlers **SHANDLES** is a variation of **SHANDLE** which implements the semantics of shallow handlers directly: the only difference is that the intermediate continuation φ_k is not placed under an identical handler. A strength of the operational approach we take is that extensions to new control operators are very direct.

3.3 Reduction

With effect handlers in the language of staged logic, situations may arise where an effect occurs immediately inside a handler, and there is nothing to do but to handle the effect symbolically, by unfolding the body of the corresponding case. This can be seen as a form of simplification, but driven by the semantics of program constructs, rather the laws relating logical connectives

in Section 2.3.2. Biabduction (Section 2.3.3) also falls into this category, if we see it as the symbolic execution of state-changing operations. We refer to this specific kind of simplification as *reduction*.

$$\begin{aligned}
& \mathbf{ret} \ v; r. \varphi \rightsquigarrow \varphi[v/r] \\
& \varphi; r. \mathbf{ret} \ r \rightsquigarrow \varphi \\
& \mathbf{match} \ \varphi_1 \vee \varphi_2 \mathbf{with} \ \mathcal{H} \rightsquigarrow (\mathbf{match} \ \varphi_1 \mathbf{with} \ \mathcal{H}) \vee \\
& \quad (\mathbf{match} \ \varphi_2 \mathbf{with} \ \mathcal{H})
\end{aligned}$$

Fig. 3.9. New simplification rules

First, some prerequisites. Most of the rules of Fig. 2.17 can be generalised directly to **bind**. We add a few more in Fig. 3.9, as well as a rule relating handlers and disjunction.

$$\begin{aligned}
& \mathbf{match}^\delta \ \varphi_1 \# r. \varphi_2 \mathbf{with} \ x \rightarrow \varphi_3 \mid \epsilon(v), k \rightarrow \varphi_4 \triangleq \\
& \mathbf{match}^\delta \ \varphi_1 \mathbf{with} \ x \rightarrow \varphi_3 \# r. \varphi_2 \mid \epsilon(v), k \rightarrow \varphi_4
\end{aligned}$$

Fig. 3.10. **match#**

Next, we introduce a handler variant called **match#** (Fig. 3.10). Intuitively, all it does is expose the continuation of a handler φ_2 as a parameter. Note that the effects of φ_2 *cannot* be handled by this particular handler; **match#** is hence useful for delimiting the effects of the scrutinee which have and have not been handled by this handler, which gives us a way to keep track of the progress being made in reduction.

We are now ready to tackle the reduction rules of Fig. 3.11. The overall strategy of reduction is to minimise the scope of handlers, by moving things out of them, and by handling as many effects as we can statically. Any effects or effectful program fragments that we cannot treat this way – i.e. when none of these rules apply and we are “stuck” – will have to be reasoned about using induction or a user-provided lemma.

The first rule, **RNORM** applies when φ is *effect-free* with respect to a handler $\mathcal{H}$, denoted $\varphi \setminus \mathcal{H}$, which means that it does not perform any effects

$$\boxed{\varphi \rightsquigarrow \varphi}$$

$$\begin{array}{c}
\text{RNORM} \\
\frac{x \rightarrow \varphi_b \in \mathcal{H} \quad \varphi \setminus \mathcal{H}}{\text{match } \varphi \text{ with } \mathcal{H} \rightsquigarrow \varphi; x. \varphi_b} \\
\\
\text{RDEEP} \\
\frac{\text{match } \varphi_2 \text{ with } \mathcal{H} \rightsquigarrow \varphi_c}{\text{match } \varphi_1; r. \varphi_2 \text{ with } \mathcal{H} \rightsquigarrow} \\
\text{match } \varphi_1 \# r. \varphi_c \text{ with } \mathcal{H} \\
\\
\text{RSHALLOW} \\
\frac{\text{match}^s \varphi_1; r. \varphi_2 \text{ with } \mathcal{H} \rightsquigarrow}{\text{match } \varphi_1 \# r. \varphi_2 \text{ with } \mathcal{H}} \\
\\
\text{RHANDLE} \\
\frac{\epsilon(x), k \rightarrow \varphi_b \in \mathcal{H}}{\text{match } \epsilon(v) \# r. \varphi_c \text{ with } \mathcal{H} \rightsquigarrow \varphi_b[\varphi_c/k][v/x]}
\end{array}$$

Fig. 3.11. Reduction rules for effect handlers

handled by $\mathcal{H}$.

$$\varphi \setminus \mathcal{H} \triangleq \forall \epsilon. \epsilon \in \mathcal{H} \Rightarrow \mathcal{E}, h_1, h_2, R \models \varphi \Rightarrow \neg(\exists v x \varphi_k. R = \epsilon(v, \lambda x. \varphi_k))$$

In that case, we can eliminate the handler by composing its value case with φ . **RSKIP** relates bind and handlers – it allows moving an effect-free prefix of the scrutinee out of the handler, reducing its scope. Effects unhandled by $\mathcal{H}$ will also be taken care of here.

If **RSKIP** cannot be applied, i.e. φ_1 will have its effects handled by $\mathcal{H}$, we have no choice but to reason operationally; the next two rules introduce **match#**. **RDEEP** reduces a deep handler from *right to left*, recursively reducing φ_2 . This gives us φ_c , which we move it into the continuation, as its effects will no longer be handled by this handler. Shallow handlers will only handle one effect, so in **RSHALLOW**, we do not recursively reduce φ_2 .

This process reaches a turning point when we have handled everything to the right, and moved everything to the left out. If the head of scrutinee is something like an unknown function, we will be stuck and require a lemma. Otherwise, if it is an effect, we can handle it by continuing with the body of the appropriate case and binding φ_c as the continuation in it, eliminating

match#.

Theorem 3.3.1 (Soundness of Reduction Rules).

$$\varphi_1 \leadsto \varphi_2 \text{ iff } \mathcal{E}, h_1, h_2, R \models \varphi_1 \Rightarrow \mathcal{E}, h_1, h_2, R \models \varphi_2.$$

Proof. See [Section 6.1.5](#). □

3.4 Another route to soundness

In [Section 2.2.2](#), we showed that staged logic was a sound way of reasoning about programs. To do this, we defined the programming language that staged logic was intended to model, then defined specification assertions and history triples, which relate programs and staged logic formulae using their semantics. This was justifiable because there was some semantic distance between the two languages, with state changes modelled by separation logic **req** and **ens**, higher-order functions modelled by the specification environment, etc.

However, the additions to staged logic in this chapter were direct equivalents of the corresponding program constructs, in order to be able to treat them operationally using refinement and keep specifications precise. Moreover, reasoning about continuations necessitated a bind operator, making staged logic much closer to a programming language.

Hence, instead of just repeating the approach for the previous chapter, we take a different route: we redefine specification assertions once and for all so that they can accommodate future operational additions to staged logic.

The idea is essentially to take staged logic as the programming language, adding new constructs concretely, possibly in an uninterpreted way, with refinement to axiomatise definitions. With this view, staged logic becomes more of an extensible intermediate verification language. Entailment then becomes the workhorse for everything, even specification assertions.

Definition 3.1 (Concrete staged formula). A concrete staged formula Φ^e for a given language of programs e extends φ with direct equivalents for all the constructs in e . For example, given a construct *if* x *then* e_1 *else* e_2 , the equivalent *if* x *then* Φ_1^e *else* Φ_2^e would be present in Φ^e , with the same semantics.

We assume a function *translate*, which maps a program e to the corresponding construct in Φ^e . It is sound by construction, as all of the constructs of e are present in Φ^e , with identical semantics.

Theorem 3.1 (*translate* is sound). Given that the program executes safely in a given model, i.e., $h_1, e \rightsquigarrow h_2, R$, we can construct a corresponding derivation in the semantics of *translate*(e), i.e., $\mathcal{E}, h_1, h_2, R \models \text{translate}(e)$.

Proof. By construction. □

Finally, we define specification asserts and history triples as follows.

$$\begin{aligned} \{\varphi_h\} e \{\varphi\} &\triangleq \varphi_h; \text{translate}(e) \sqsubseteq \varphi \\ e &::: \varphi \triangleq \text{translate}(e) \sqsubseteq \varphi \end{aligned}$$

[Theorems 2.1](#) and [2.2](#) then follow easily from the soundness of entailment and *translate*.

3.5 Evaluation

To evaluate our proposed approach to reasoning about effect handlers, we implemented it in HEIFER and verified a suite of benchmark programs. This section reports on the results, which are shown in [Table 3.1](#).

Program 1 is taken from the *Memory Cell with Exchange* example from de Vilhena and Pottier [72] and extended with additional stateful operations. Programs 3 and 7 are from the benchmarks of the *multicont*³ library, which provides practical examples for multishot continuations. Program 5 revises

³<https://github.com/dhil/ocaml-multicont>

program 3 by changing the handler to be shallow. Programs 4 and 6 demonstrate the use of lemmas for left-recursive functions in both deep and shallow handlers. Program 2 is a new example.

Table 3.1
Experimental Results

#	Program	Ind	MultiS	ImpureC	HO	LoC	LoS	Total(s)	Z3(s)
1	State monad	✗	✗	✓	✗	126	16	8.54	6.21
2	Inductive sum	✓	✗	✓	✗	41	11	1.68	1.28
3	Flip-N (Deep Right Rec)	✓	✓	✓	✗	39	10	2.09	1.52
4	Flip-N (Deep Left Rec)	✓	✓	✓	✗	45	13	2.03	1.53
5	Flip-N (Shallow Right Rec)	✗	✓	✓	✗	37	11	5.08	3.18
6	Flip-N (Shallow Left Rec)	✓	✓	✓	✗	64	23	6.75	4.26
7	McCarthy’s amb operator	✓	✓	✓	✓	109	45	7.71	5.34
						461	129	33.88	23.32

In Table 3.1, columns **LoC** and **LoS** record the lines of code and lines of specification, respectively. The column **Total** records the total verification time taken in seconds. These numbers include the time taken by **Z3**, which is shown separately in the next column. Although verification is mostly automated, we show that the verification is non-trivial by labeling the programs with features: **Ind** indicates whether the proof is inductive, **MultiS** indicates if the program uses multishot continuations, **ImpureC** indicates if there are impure (heap-manipulating) continuations. **HO** indicates if the example is higher-order, i.e., function inputs or outputs are of function type.

Given 461 lines of code in total, 129 lines of specification are required, resulting in an average LoS/LoC ratio of 0.27.

The average verification time is in the order of seconds, with 68.8% of verification time taken by SMT solving. These demonstrate that the approach is lightweight, and feasible for guiding the design of an automated verification tool.

VERIFYING PROGRAMS WITH DELIMITED CONTINUATIONS

4.1 Introduction

While effects handlers have attracted much recent attention due to their inclusion in OCaml 5, they represent only one set of design choices – the literature contains many more delimited continuation operators with different characteristics. In this chapter, I generalise the framework of staged logic to handle them.

We begin with a gentle introduction to the *shift* and *reset* operators and the use of refinement for reasoning about them, before moving to the other operators in the family.

`4 + reset (1 + shift k (k 2 + k 3))`

Fig. 4.1. A simple example using shift/reset

Fig. 4.1 demonstrates the use of shift and reset with a small program in an ML-like syntax. The **shift** operator captures its *continuation* – the remaining computation that its result will be used in – and *reifies* it as a function value, which will then be bound to *k* in its body (*k* 2 + *k* 3). Here, the continuation of **shift** is 1 + □, with □ denoting a *hole* where the result of the **shift** would go. The outer expression 4 + □ is not included due to the presence of **reset**, which *delimits* the extent of the continuation which is captured. **reset** has no other operational behaviour, and its result is simply that of its body once the latter is *shift-free* – if it reduces to a value with no further shifts.

How does this program execute? The continuation 1 + □ is *reified* into a function value ($\lambda x. 1 + x$), and is bound to *k* in (*k* 2 + *k* 3), which is executed next; it uses the continuation twice. The result of the program

will thus be $4 + ((\underline{1+2}) + (\underline{1+3})) = 11$, where uses of the continuation are underlined. Note that in general, the continuation could contain an arbitrary (and dynamic) amount of computation, as we will see in the append example.

Specifying and reasoning about programs using control operators compositionally is challenging, due to their fundamentally *non-context-local* [198] nature: an arbitrary expression e may contain a control operator, causing reasoning about e to depend on its continuation. This necessitates an approach such as *non-context-local* triples $\{P\} C[e] \{Q\}$ [198]. We defer a survey of approaches to Section 7.4.2.

Answer type modification The ability to manipulate the continuation of an expression gives rise to some subtle issues, one of which is known as *answer type modification* [66] (ATM).

```
let percent_d () =
  shift k (fun n -> k (string_of_int n))

(reset (percent_d () ^ "2")) 1
```

Fig. 4.2. *printf* using answer-type modification

Consider the program in Fig. 4.2. It is an encoding of a call to the classic *printf* function with the format string "%d2", here represented as a delimited program which uses **shift**. Its result is "12".

At a glance, the program appears rather odd: the result of the **reset** appears to be a string, the result of a string concatenation (^); this is referred to as its *answer type*. However, the result of **reset** is applied to the integer 1. Somehow, the types “inside” and “outside” the **reset** are allowed to differ, even though, as we saw in the last example, **reset** has no operational behaviour besides delimiting the extent of **shifts**, and is the identity when its body is a value.

The explanation for this phenomenon is that the type of the **shift** body

does not necessarily have to agree with the result type of the continuation. When these are different, the program is said to have *modified the answer type* of the **reset**.

Initially, the answer type of `percent_d` is `string`, as it is in a context which accepts a string. However, the result of the **shift** body is a function (`fun n -> ...`), which is said to *modify the answer type* to `int -> string`. This function accepts an integer before producing a string using the continuation `k`, which is then the final result of the whole expression.

As this example demonstrates, ATM is useful: being able to repeatedly change the answer type allows us to accept a variable number of arguments depending on the expressions appearing in the **reset**, among other things.

ATM complicates reasoning about programs, due to the need to know about possible answer type changes in **reset** bodies in order to verify them. Again, more approaches are covered in [Section 7.4.2](#); we first proceed to our approach, which works even in the presence of ATM.

4.1.1 Reasoning with refinement

We demonstrate the use of staged logic in the verification of a recursive program with **shift/reset** and ATM.

```
let append_sh xs =
  match xs with
  | [] -> shift k k
  | x :: xs1 -> x :: append_sh xs1

let append_delim xs ys =
  reset (append_sh xs) ys
```

Fig. 4.3. `append_delim` [\[64\]](#)

`append_delim` in [Fig. 4.3](#) uses ATM to implement a *difference list* [\[105\]](#): a function that builds a list by appending its argument to it. It does this using the `shift k k` idiom, which changes the answer type from a list (as the *cons*

branch suggests) to a function. The continuation is in effect a difference list: it includes all the deferred operations of the recursive calls to `append_sh`, which will build the list once a suitable tail is provided in `ys`.

This program is elegant, and it is easy to convince oneself informally that it implements the standard list append function. How can we prove this formally?

First, we write the specification as a refinement.

$$\langle \text{append_sh}(xs) \rangle; f. f(ys) \sqsubseteq \mathbf{ens} \ r. r = xs ++ ys$$

Like before, we assume a staged logic with constructs **shift** $k. \varphi$ and $\langle \varphi \rangle$, and the ability to symbolically execute them.

A straightforward induction on the structure of `xs` does not work. We eventually see that the inductive case requires the following generalisation, over a prefix of the input `zs`; the original goal then follows from instantiating `zs` with `[]`.

$$\langle \text{append_sh}(xs); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) \sqsubseteq \mathbf{ret} \ (zs ++ xs) ++ ys$$

This lemma *can* be proved by induction on `xs`.

Base case $\langle \text{append_sh}([]); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) \sqsubseteq \mathbf{ret} \ (zs ++ []) ++ ys$

$$\begin{aligned} & \langle \text{append_sh}([]); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) \\ \sqsubseteq & \langle (\mathbf{shift} \ k. k); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) && \text{Unfold} \\ \sqsubseteq & \langle \mathbf{ret} \ (\lambda r. \langle \mathbf{ret} \ zs ++ r \rangle) \rangle; f. f(ys) && \text{Reduction}^1 \\ \sqsubseteq & \mathbf{ret} \ (\lambda r. \mathbf{ret} \ zs ++ r); f. f(ys) && \text{Pure} \\ \sqsubseteq & \mathbf{ret} \ zs ++ ys && \text{Simplify} \\ \sqsubseteq & \mathbf{ret} \ (zs ++ []) ++ ys && \text{Lem. ++} \quad \square \end{aligned}$$

Inductive case $\langle \text{append_sh}(x_1 :: xs_2); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) \sqsubseteq \mathbf{ret} \ (zs ++ (x_1 :: xs_2)) ++ ys$

$$\begin{aligned} & \langle (\text{append_sh}(xs_2); r_1. \mathbf{ret} \ x_1 :: r_1); r. \mathbf{ret} \ zs ++ r \rangle; f. f(ys) \\ \sqsubseteq & \langle \text{append_sh}(xs_2); r_1. \mathbf{ret} \ zs ++ (x_1 :: r_1) \rangle; f. f(ys) && \text{Reassoc. bind} \\ \sqsubseteq & \langle \text{append_sh}(xs_2); r_1. \mathbf{ret} \ (zs ++ [x_1]) :: r_1 \rangle; f. f(ys) && \text{Lem. cons, ++} \\ \sqsubseteq & \mathbf{ret} \ (((zs ++ [x_1]) ++ xs_2) ++ ys) && \text{Ind. hyp.} \\ \sqsubseteq & \mathbf{ret} \ ((zs ++ (x_1 :: xs_2)) ++ ys) && \text{Lem. cons, ++} \quad \square \end{aligned}$$

Reduction¹ (used in the base case) eliminates **shifts** from formulae via symbolic execution, similar to Handle in [Section 3.1.2](#). The result of the shift body is k , the continuation itself, which prepends zs to its result.

The proofs presented here harken back to the classical use of (monadic) equational reasoning [\[99, 13\]](#), with two key differences. The first is that with refinement, we have the added freedom to use specifications or abstractions of the pieces involved. The second is that this approach is itself a refinement of the one from [Chapter 2](#), so stateful and higher-order programs, as well as automation, are supported naturally.

4.1.2 Beyond shift and reset

Next, I generalise the framework so that it may be applied to other continuation operators, in a similar spirit as Ishio and Asai [\[107\]](#). Dybvig et al. [\[80\]](#) present a taxonomy of such operators. Their semantics are given in [Fig. 4.4](#). On the left is the notation used by Dybvig et al., while the name of the operator used appears in each rule.

$$\begin{array}{ll}
\langle v \rangle \Longrightarrow v & \\
+\mathcal{F}+ & \langle E[\mathbf{shift} \ k. \ e] \rangle \Longrightarrow \langle (\lambda k. e) (\lambda v. \langle E[v] \rangle) \rangle \quad [67] \\
-\mathcal{F}+ & \langle E[\mathbf{shift}_0 \ k. \ e] \rangle \Longrightarrow (\lambda k. e) (\lambda v. \langle E[v] \rangle) \quad [66] \\
+\mathcal{F}- & \langle E[\mathbf{control} \ k. \ e] \rangle \Longrightarrow \langle (\lambda k. e) (\lambda v. E[v]) \rangle \quad [82] \\
-\mathcal{F}- & \langle E[\mathbf{control}_0 \ k. \ e] \rangle \Longrightarrow (\lambda k. e) (\lambda v. E[v]) \quad [96]
\end{array}$$

Fig. 4.4. Reduction semantics of various control operators [\[80, 8\]](#)

The operators differ operationally in whether they leave a delimiter around the continuation (indicated by $+$ on the right) and the operator body e ($+$ on the left). Intuitively, these differences allow for more “dynamic” [\[41\]](#) behaviours, along distinct axes: **control** is able to access the bodies of parallel operators, while **shift**₀ is able to “escape” to enclosing delimiters [\[144\]](#).

Assuming **control** in the logic ([Section 4.2](#)), we illustrate how reasoning with it is as natural as with shift and reset, using an interesting example originally from Biernacki and Danvy [\[40\]](#).

```

let visit xs =
  match xs with
  | [] -> []
  | x :: xs1 -> visit (shift k (x :: k xs))

let traverse xs = reset (visit xs)

```

Fig. 4.5. traverse [40, 39]

The function `traverse` (Fig. 4.5) copies a given list. Interestingly, if the **control** operator were instead used in `visit`, `traverse` *reverses* the list.

How does this happen? Let us first consider how the programs execute on a small input to for some intuition (Fig. 4.6). The key difference is in whether and how the control operators access the cons operations; each **shift** does not access them, and so extends the list in “well-behaved” manner, whereas **control** does access them and nests them one level deeper, resulting in the reversal.

Now, to prove these two facts, we start with specifications (Fig. 4.7): pure functions *copy* and *reverse*. The latter is implemented using an accumulator.

That using **shift** copies the list is easy to prove by induction.

$$\langle \text{visit}(xs) \rangle \sqsubseteq \text{copy}(xs)$$

The base case is trivial (both sides are identical after unfolding).

Inductive case $\langle \text{visit}(x :: xs_1) \rangle \sqsubseteq \text{copy}(x :: xs_1)$

$$\begin{aligned}
& \langle \text{visit}(x :: xs_1) \rangle \\
& \sqsubseteq \langle (\mathbf{shift} \ k. \ k(xs_1); r. \mathbf{ret} \ x :: r); r_1. \text{visit}(r_1) \rangle && \text{Unfold} \\
& \sqsubseteq \mathbf{ens} \ k = (\lambda r_1. \langle \text{visit}(r_1) \rangle); \langle k(xs_1); r. \mathbf{ret} \ x :: r \rangle && \text{Reduction}^2 \\
& \sqsubseteq \langle \langle \text{visit}(xs_1) \rangle; r. \mathbf{ret} \ x :: r \rangle && \text{Simplify} \\
& \sqsubseteq \langle \text{copy}(xs_1); r. \mathbf{ret} \ x :: r \rangle && \text{Ind. hyp.} \\
& \sqsubseteq \text{copy}(xs_1); r. \mathbf{ret} \ x :: r \quad \square && \text{Pure}
\end{aligned}$$

The statement using **control** is trickier. *visit'* is the same as *visit* except

```

reset (visit [1; 2; 3])
= reset (visit (shift k (1 :: k [2; 3])))
= reset (1 :: reset (visit [2; 3]))
= reset (1 :: reset (visit (shift k (2 :: k [3]))))
= reset (1 :: reset (2 :: reset (visit [3])))
= reset (1 :: reset (2 :: reset (visit (shift k (3 :: k [])))))
= reset (1 :: reset (2 :: reset (3 :: reset (visit []))))
= reset (1 :: reset (2 :: reset (3 :: reset ([]))))

reset (visit' [1; 2; 3])
= reset (visit' (control k (1 :: k [2; 3])))
= reset (1 :: visit' [2; 3])
= reset (1 :: visit' (control k (2 :: k [3])))
= reset (2 :: (fun x -> 1 :: visit' x) [3])
= reset (2 :: 1 :: visit' [3])
= reset (2 :: 1 :: visit' (control k (3 :: k [])))
= reset (3 :: (fun x -> 2 :: 1 :: visit' x) [])
= reset (3 :: 2 :: 1 :: visit' [])
= reset (3 :: 2 :: 1 :: [])

```

Fig. 4.6. shift to traverse, and control to reverse

for using **control** instead of **shift**.

$$\langle \text{visit}'(xs) \rangle \sqsubseteq \text{rev}(xs, [])$$

In the inductive case, we eventually get stuck in the following state, where we can't appeal to the induction hypothesis.

$$\langle \text{visit}'(xs_1); r. \text{ret } x :: r \rangle$$

The key idea is that we must generalise the entailment over the *continuation* of visit' . In general, nontrivial generalisation may be required [99].

Suppose we did that for the left side of the entailment. Now, what is the appropriate generalisation of rev over k ?

$$\langle \text{visit}'(xs); r. k(r) \rangle \sqsubseteq \dots$$

A naive idea would be to apply the CPS transformation to rev , resulting in

$$\begin{aligned}
\text{copy}(xs) = & \\
& \mathbf{ens} \text{ } xs = []; \mathbf{ret} [] \\
& \vee \exists x \text{ } xs_1. \mathbf{ens} \text{ } xs = x :: xs_1; \text{copy}(xs_1); r_1. \mathbf{ret} x :: r_1 \\
\\
\text{rev}(xs, acc) = & \\
& \mathbf{ens} \text{ } xs = []; \mathbf{ret} acc \\
& \vee \exists x \text{ } xs_1. \mathbf{ens} \text{ } xs = x :: xs_1; \text{rev}(xs_1, x :: acc)
\end{aligned}$$

Fig. 4.7. Specifications for *copy* and *rev*

a definition with signature $\text{rev}'(xs, acc, k)$. While this works just as well in staged logic as it does for programs, it does not help us: looking at the state where we got stuck, we see that the continuation that we are generalising over is involved in building the reversed list. In other words, we need a version of *rev* which performs the accumulation *using* the continuation, i.e. $\text{rev}_1(xs, k)$. Coming up with this reformulation is not straightforward, as it is further complicated by **control**.

The good news is that this definition can be *synthesised* using just a small extension of the machinery we have presented so far.

Lemma synthesis The technique of lemma synthesis we propose is an exploratory one which discovers intermediate lemmas to prove a goal.

Suppose we have an entailment $\varphi \sqsubseteq \varphi_c$ to be proved. We find that direct inductive proof does not work, and that we need to come up with a lemma φ' as an intermediate step towards the goal. We know that this lemma is a generalisation of the goal and can write down its signature, but do not know its definition, nor the two proofs demonstrating that it is indeed an intermediate step. This situation is shown as the following incomplete proof tree.

$$\frac{
\frac{
\frac{[\dots]}{\varphi \sqsubseteq [\varphi']}
}{\varphi \sqsubseteq [\varphi']}
\quad
\frac{\vdots}{\varphi' \sqsubseteq \varphi_c}
}{\varphi \sqsubseteq \varphi_c}$$

The idea is that we can discover a candidate for φ' by taking it as uninterpreted, and searching for a proof of $\varphi \sqsubseteq \varphi'$ as per normal. When we inevitably get stuck, we proceed by interpreting the pending goal(s) as constraints on the definition of φ' , making $\varphi \sqsubseteq \varphi'$ true by construction. Having synthesised φ' , we may then recursively continue trying to prove the goal φ_c .

We illustrate this technique by using it to solve the *visit'* example. The statement we use to direct the search is as follows, where rev_1 is uninterpreted.

$$\langle visit'(xs); r. k(r) \rangle \sqsubseteq rev_1(xs, k)$$

Base case $\langle visit'([]); r. k(r) \rangle \sqsubseteq rev_1([], k)$

$$\begin{aligned} & \langle visit'([]); r. k(r) \rangle \\ & \sqsubseteq \langle \mathbf{ret} []; r. k(r) \rangle && \text{Unfold} \\ & \sqsubseteq k([]) \quad \square && \text{Simplify} \end{aligned}$$

Inductive case $\langle visit'(x :: xs_1); r. k(r) \rangle \sqsubseteq rev_1(x :: xs_1, k)$

$$\begin{aligned} & \langle visit'(x :: xs_1); r. k(r) \rangle \\ & \sqsubseteq \langle (\mathbf{control} \ k_1. k_1(xs_1); r. \mathbf{ret} \ x :: r); r_1. visit'(r_1); r. k(r) \rangle && \text{Unfold} \\ & \sqsubseteq \mathbf{ens} \ k_1 = (\lambda r_1. visit'(r_1); ; r. k(r)); \langle k_1(xs_1); r. \mathbf{ret} \ x :: r \rangle && \text{Reduction}^3 \\ & \sqsubseteq \langle visit'(xs_1); r_1. k(r_1); r. \mathbf{ret} \ x :: r \rangle && \text{Simplify} \\ & \sqsubseteq rev_1(xs_1, \lambda r_1. k(r_1); r. \mathbf{ret} \ x :: r) \quad \square && \text{Ind. hyp.} \end{aligned}$$

This is as far as we can get in both proofs. Notably, the proof search is exactly what would have been done given a goal with an interpretation that we could direct the search towards. Unlike [Reduction²](#), [Reduction³](#) does not leave a delimiter in the continuation, according to the semantics of **control** ([Fig. 4.4](#)).

The final unproved goals together give us an inductive definition for rev_1 ([Fig. 4.8](#)), by assembling together the two cases under the assumptions on xs in each case.

Interestingly, due to the use of reduction, both **control** and delimiter have been eliminated, resulting in a *continuation-composing* program that by con-

$$\begin{aligned}
rev_1(xs, k) = & \\
& \mathbf{ens} \ xs=[]; k([]) \\
& \vee \exists x \ xs_1. \mathbf{ens} \ xs=x :: xs_1; rev_1(xs_1, \lambda r_1. k(r_1)); r. \mathbf{ret} \ x :: r)
\end{aligned}$$

Fig. 4.8. Synthesised definition for rev_1

struction (over)approximates $visit'$. This program is not strictly in CPS (as the continuation is not invoked in tail position, fixing its answer type to a list) and is not derivable through only the CPS transformation. However, it is the definition, and the intermediate step, that we need.

Now, it remains to prove the final subgoal:

$$rev_1(xs, k) \sqsubseteq rev(xs, acc)$$

Our next problem is that there is no relation between k and acc ; the induction step will thus give us no information about how acc changes across it. We need another generalisation, guided by intuition as to what acc is.

acc is the suffix of the reversed list that is extended as rev traverses the original list. The continuation of rev_1 builds a similar suffix as it descends, but in closures, with k finally being applied to the empty list at the end. The generalisation is thus the following statement: given that the continuation is finally applied to the empty list, acc is everything that will be *prepended* to the empty list, i.e. the suffix.

$$rev_1(xs, \lambda y. acc ++ y) \sqsubseteq rev(xs, acc)$$

This statement can now be proved automatically.

Base case $rev([], \lambda y. \mathbf{ret} \ acc ++ y) \sqsubseteq rev(xs, acc)$

$$\begin{aligned}
& rev([], \lambda y. \mathbf{ret} \ acc ++ y) && \text{Unfold} \\
\sqsubseteq & \mathbf{ens} \ xs=[]; \mathbf{ret} \ acc && \text{Simplify} \\
\sqsubseteq & rev(xs, acc) \quad \square && \text{Fold}
\end{aligned}$$

Inductive case $\text{rev}(x :: xs_1, \lambda y. \mathbf{ret} \text{ acc} ++ y) \sqsubseteq \text{rev}(xs, \text{acc})$

$$\begin{aligned}
& \text{rev}(x :: xs_1, \lambda y. \mathbf{ret} \text{ acc} ++ y) \\
& \sqsubseteq \text{rev}(xs_1, \lambda y. \mathbf{ret} \text{ acc} ++ y; r_1. \mathbf{ret} x :: r_1) && \text{Unfold} \\
& \sqsubseteq \text{rev}(xs_1, \lambda y. \mathbf{ret} x :: (\text{acc} ++ y)) && \text{Simplify} \\
& \sqsubseteq \text{rev}(xs_1, \lambda y. \mathbf{ret} (x :: \text{acc}) ++ y) && \text{Lem. cons, ++} \\
& \sqsubseteq \text{rev}(x :: xs_1, \text{acc}) \quad \square && \text{Ind. hyp.}
\end{aligned}$$

Taking a step back, *how* did the proof work? When we generalised *visit'* over the continuation, lemma synthesis took us from a **control**-using program into a CPS program, which we then related to the final direct-style program via a second generalisation – a surprisingly systematic approach given its heuristic nature. Interestingly, it did not go through the standard CPS transformation, which for **control** is more involved [113].

Finally, given the proofs for both *copy* and *rev*, where does the difference in behaviour that we saw in Fig. 4.6 show up? Consider the steps *after* Reduction² and Reduction³. The $\mathbf{ret} x :: r$ portion is the deferred operation that may or may not be accessed, depending on which control operator is used. In the case of Reduction², it is not accessed, reflected by how we can simply rewrite (a delimited) *visit* on its own. In Reduction³, however, we had to generalise to be able to talk about what follows *visit'*, or *what it accesses*. Moreover, this became part of the inductive argument, pointing to how *visit'* repeatedly accesses the deferred operation, to nest it deeper and reverse the list.

In the next section, we cover the extensions to staged logic required to realise the reasoning above.

4.2 Extensions to staged logic

We follow the technical development of Chapter 3, extending staged logic with delimited continuation operators. As before, we define their semantics directly, instead of encoding them as effects, for a simpler and more direct

treatment.

$$\begin{aligned}
\varphi &::= \mathbf{req} \ \sigma \ \varphi \mid \mathbf{ens} \ r. \sigma \\
&\mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \exists x. \varphi \mid \forall x. \varphi \\
&\mid \varphi_1; r. \varphi_2 \mid f(v) \\
&\mid \mathbf{shift}_n^c \ k. \varphi \ (\lambda x. \varphi_k) \mid \mathbf{control}_n^c \ k. \varphi \ (\lambda x. \varphi_k) \mid \langle \varphi \rangle \\
n &::= 0? \\
\mathbf{id} &\triangleq \lambda x. \mathbf{ret} \ x \\
\mathbf{shift}_n \ k. \varphi &\triangleq \mathbf{shift}_n^c \ k. \varphi \ \mathbf{id} \\
\mathbf{control}_n \ k. \varphi &\triangleq \mathbf{control}_n^c \ k. \varphi \ \mathbf{id}
\end{aligned}$$

Fig. 4.9. Staged logic with delimited control operators

Syntax We add both **shift** and **control**, with both zero and nonzero variants, and a single delimiter (Fig. 4.9). We define the control operators in terms of more primitive constructors **shift**^c and **control**^c, which carry an additional argument for an *intermediate continuation*, which is the identity continuation **id** by default. This is similar to the continuation we used in the **match**# in Section 3.3, but here it is associated with the control operator rather than the delimiter. This allows us to perform reduction in a more intuitive “forward” manner, driven by the control operator rather than the delimiter. It is also possible to associate the continuation with the delimiter by giving it a value case, i.e. the *dollar* operator [144], and in that case we would take the same approach to reduction as in the previous chapter.

Semantics We use a semantics inspired by the exception-like semantics of effects given in Section 3.2 for control operators. It is similar to the “bubble-up semantics” of resumable exceptions proposed by Kiselyov [119]. Again, the construction of the delimited continuation is operationalised in it.

The semantics (Fig. 4.10) starts with a definition of outcomes: values, or “unhandled” control operators; the latter directly correspond to **shift**^c and **control**^c. The rules for them, **SSHIFT** and **SCONTROL**, are hence quite trivial and serve as base cases. Unhandled control operators are technically ill-formed programs, but we treat them as we do effects, giving them a seman-

$$\boxed{\mathcal{E}, h, h, R \models \varphi}$$

$$R ::= v \mid \mathbf{shift}_n(\lambda k. \varphi_b, \lambda x. \varphi_k) \mid \mathbf{control}_n(\lambda k. \varphi_b, \lambda x. \varphi_k)$$

$$\frac{\text{SSHIFT} \quad R = \mathbf{shift}_n(\lambda k. \varphi_b, \mathbf{id})}{\mathcal{E}, h, h, R \models \mathbf{shift}_n k. \varphi_b}$$

$$\frac{\text{SCONTROL} \quad R = \mathbf{control}_n(\lambda k. \varphi_b, \mathbf{id})}{\mathcal{E}, h, h, R \models \mathbf{control}_n k. \varphi_b}$$

$$\frac{\text{SBINDVAL} \quad \begin{array}{l} \mathcal{E}, h_1, h_3, v \models \varphi_1 \\ \mathcal{E}, h_3, h_2, R \models \varphi_2[v/r] \end{array}}{\mathcal{E}, h_1, h_2, R \models \varphi_1; r. \varphi_2}$$

$$\frac{\text{SBINDSHIFT} \quad \begin{array}{l} \mathcal{E}, h_1, h_2, \mathbf{shift}_n(\lambda k. \varphi_b, \lambda x. \varphi_k) \models \varphi_1 \\ R = \mathbf{shift}_n(\lambda k. \varphi_b, \lambda x. \varphi_k; r. \varphi_2) \end{array}}{\mathcal{E}, h_1, h_2, R \models \varphi_1; r. \varphi_2}$$

$$\frac{\text{SBINDCONTROL} \quad \begin{array}{l} \mathcal{E}, h_1, h_2, \mathbf{control}_n(\lambda k. \varphi_b, \lambda x. \varphi_k) \models \varphi_1 \\ R = \mathbf{control}_n(\lambda k. \varphi_b, \lambda x. \varphi_k; r. \varphi_2) \end{array}}{\mathcal{E}, h_1, h_2, R \models \varphi_1; r. \varphi_2}$$

$$\frac{\text{SDELIMVAL} \quad \mathcal{E}, h_1, h_2, v \models \varphi}{\mathcal{E}, h_1, h_2, v \models \langle \varphi \rangle}$$

$$\frac{\text{SDELIMSHIFT} \quad \begin{array}{l} \mathcal{E}, h_1, h_3, \mathbf{shift}(\lambda k. \varphi_b, \lambda x. \varphi_k) \models \varphi \\ \mathcal{E}[k := \lambda x. \langle \varphi_k \rangle], h_3, h_2, r \models \langle \varphi_b \rangle \end{array}}{\mathcal{E}, h_1, h_2, R \models \langle \varphi \rangle}$$

$$\frac{\text{SDELIMSHIFT0} \quad \begin{array}{l} \mathcal{E}, h_1, h_3, \mathbf{shift}_0(\lambda k. \varphi_b, \lambda x. \varphi_k) \models \varphi \\ \mathcal{E}[k := \lambda x. \langle \varphi_k \rangle], h_3, h_2, r \models \varphi_b \end{array}}{\mathcal{E}, h_1, h_2, R \models \langle \varphi \rangle}$$

$$\frac{\text{SDELIMCONTROL} \quad \begin{array}{l} R_0 = \mathbf{control}(\lambda k. \varphi_b, \lambda x. \varphi_k) \\ \mathcal{E}, h_1, h_3, R_0 \models \varphi \\ \mathcal{E}[k := \lambda x. \varphi_k], h_3, h_2, r \models \langle \varphi_b \rangle \end{array}}{\mathcal{E}, h_1, h_2, R \models \langle \varphi \rangle}$$

$$\frac{\text{SDELIMCONTROL0} \quad \begin{array}{l} R_0 = \mathbf{control}_0(\lambda k. \varphi_b, \lambda x. \varphi_k) \\ \mathcal{E}, h_1, h_3, R_0 \models \varphi \\ \mathcal{E}[k := \lambda x. \varphi_k], h_3, h_2, r \models \varphi_b \end{array}}{\mathcal{E}, h_1, h_2, R \models \langle \varphi \rangle}$$

Fig. 4.10. Semantics of control operators in staged logic, extending Fig. 3.8

tics. Practical systems often also make concessions here: for example, Racket uses an implicit top-level delimiter with an identity continuation [121].

SBINDVAL is the same as in Fig. 3.8, generalising sequencing to binding. **SBINDSHIFT** and **SBINDCONTROL** extend the continuation, pushing the “current continuation” into the outcome. **SDELIMVAL** serves as a base case for when φ is a value, in which case the delimiter is removed. **SDELIMSHIFT** applies when φ reduces to a shift; reduction continues with the shift body, with the the continuation appropriately bound, and delimiters around both. The remaining rules are similar, but directly implement the semantics of each operator by varying where the delimiters are placed, as in Fig. 4.4.

4.3 Reduction

Control operators can be reasoned about via the same semantics-directed *reduction* first introduced in [Section 3.3](#). In this section, we focus first on **shift**, as it gives the strongest reasoning rules, noting along the way where other operators differ. We begin with a few auxiliary definitions.

Definition 4.1 (Shift-free). φ is *shift-free*, denoted $\varphi^\#$, iff it can never result in a shift outcome.

$$\varphi^\# \triangleq \mathcal{E}_1, h_1, h_2, R \models \varphi \Rightarrow \neg \exists k \varphi_b x \varphi_k. R = \mathbf{shift}(\lambda k. \varphi_b, \lambda x. \varphi_k)$$

Shift-freeness can be proved compositionally ([Fig. 4.11](#)).

$$\boxed{\varphi^\#}$$

$$\begin{array}{cccc} \text{SFENS} & \text{SFREQ} & \text{SFSEQ} & \text{SFRESET} \\ \frac{}{(\mathbf{ens} Q)^\#} & \frac{\varphi^\#}{(\mathbf{req} H \varphi)^\#} & \frac{\varphi_1^\# \quad \varphi_2^\#}{(\varphi_1; \varphi_2)^\#} & \frac{}{\langle \varphi \rangle^\#} \end{array}$$

Fig. 4.11. Proving shift-freeness (select rules)

ens is always shift-free, **req** and sequencing are shift-free if their subexpressions are, and most importantly, a reset is always shift-free; unlike an effect handler, it is not possible for a shift to “escape” an enclosing reset. Intuitively, this is because the shift body is always placed under another reset. Crucially, this last rule is not true for the zero operators.

We may now define the reduction rules ([Fig. 4.12](#)). Again, the goal is to minimise the scopes of resets, trying to move things out to the left if they are shift-free, before eliminating resets by matching against a shift in its body. **RNORMAL** serves as a base case – if φ is shift-free, we can simply remove an enclosing reset. **RSKIP** allows moving a shift-free prefix φ_1 out from under a reset. The next two rules work in tandem to determine the extent of the continuation. **RINIT** begins the process, introducing a **shift**^c with an identity

$\boxed{\varphi \rightsquigarrow \varphi}$		
$\frac{\text{RNORMAL} \quad \varphi^\#}{\langle \varphi \rangle \rightsquigarrow \varphi}$	$\frac{\text{RSKIP} \quad \varphi_1^\#}{\langle \varphi_1; \varphi_2 \rangle \rightsquigarrow \varphi_1; \langle \varphi_2 \rangle}$	$\frac{\text{RINIT}}{\mathbf{shift}^c k. \varphi_b \rightsquigarrow \mathbf{shift}^c k. \varphi_b \mathbf{id}}$
$\frac{\text{REXTEND}}{(\mathbf{shift}^c k. \varphi_b \lambda x. \varphi_k); r. \varphi \rightsquigarrow \mathbf{shift}^c k. \varphi_b (\lambda x. \varphi_k; r. \varphi)}$	$\frac{\text{RSHIFTELMIM}}{\langle \mathbf{shift}^c k. \varphi_b (\lambda x. \varphi_k) \rangle \rightsquigarrow \mathbf{ens} [k = \lambda x. \varphi_k]; \varphi_b}$	

Fig. 4.12. Shift/reset reduction

continuation – because of the definition of **shift**, this is just an unfolding. **REXTEND** picks up after this: when iterated, it extends the continuation φ_k with any φ after it. When a **shift**^c is the only thing remaining, the delimited continuation is completely determined. **RSHIFTELMIM** then ends the process, eliminating the shift and replacing it with its body, and also defining the continuation for use inside it.

The rules are written as entailments, and are thus sound by construction.

4.4 Evaluation

To demonstrate the effectiveness and applicability of our proposed methodology, we extended HEIFER with support for the shift and reset operators, as well as the reduction rules described in Section 4.3.

We then used HEIFER to verify a suite of benchmark programs involving nontrivial uses of shift and reset, mostly taken from the literature on delimited continuations. As our focus is broadening the applicability of automated verification and scaling it to handle more language features, we selected small programs using language features which are difficult for existing automated verifiers to handle, such as stateful higher-order functions and multishot continuations. To our knowledge, none of the programs have been verified automatically.

Table 4.1
Experimental results

Program	Desc.	LoC	LoS	Total(s)	SMT(s)
State monad [21]	HO	21	2	4.84	0.02
Alice-Cat [144]	Multi	3	1	0.33	0.02
Printf [21]	ATM	30	6	2.20	0.05
Flip [67]	Multi	9	1	3.89	0.04
Toss [186]	Imp	8	1	6.50	0.08
HTTcc-inc3 [74]	Imp	5	1	5.05	0.05
Times [21]	Ind	17	5	0.88	0.11
Total		93	17	23.69	0.37

Table 4.1 lists the benchmark programs, followed by a list of the interesting features of each program: whether it is higher-order, involves multi-shot continuations, whether it uses imperative state, whether induction is needed, and whether it uses answer-type modification (ATM), its size (lines of code and lines of specification), and finally the total time taken to verify the whole program and the amount of time spent in SMT solvers (both in seconds). All experiments were performed on Ubuntu 24.04.2 Linux using a 3201 MHz 6-Core AMD Ryzen 5 CPU with 16 GB of RAM. Z3 4.13.0 was used.

Insights These experiments are small in scale, and so mainly serve to validate the claims that staged logic is a novel and effective means of verifying programs containing delimited continuations, and is feasible to automate. The lines of specifications required were significantly less than the LoC (with a spec-to-code ratio of 0.18), owing to the use of biabduction; often only the specification needs to be given, and an induction hypothesis (where a loop invariant would normally be required in other systems) is inferred based on the structure of the verification condition. This suggests potential for being usable for automated verification method of larger, more realistic programs.

CASE STUDIES

5.1 Landin's Knot

Landin's Knot [128] is a classic example of a higher-order, effectful program. It illustrates how the addition of state to an otherwise terminating language leads to the ability to write nonterminating programs, via cycles through the memory.

A semantic account of Landin's Knot is rather involved. Typing the memory into which mutable references point historically required a step-indexed, possible-worlds model [14]. A modern specification of the Knot requires a way to uphold invariants on the contents of memory [42, Section 9].

Intuitively, however, the Knot is not a very complex program: it is easy to see informally how the recursive behaviour arises, by mentally executing it a few steps until the cycle becomes apparent. In this section I present a simple, syntactic proof in staged logic that follows this pattern of reasoning.

First, some definitions.

$$\begin{aligned}
 \text{read}(l) &\triangleq \forall v. \mathbf{req} \ l \mapsto v; \mathbf{ens} \ l \mapsto v; \mathbf{ret} \ v \\
 \text{write}(l, v) &\triangleq \forall v_0. \mathbf{req} \ l \mapsto v_0; \mathbf{ens} \ l \mapsto v \\
 \text{landin_rec}(f) &\triangleq \\
 &\quad \exists l \text{ knot}. \\
 &\quad \mathbf{ens} \ l \mapsto (); \\
 &\quad \mathbf{ens} \ \text{knot} = (\lambda n. \text{read}(l); f_1. f(f_1, n)); \\
 &\quad \text{write}(l, \text{knot}); \\
 &\quad \mathbf{ret} \ \text{knot}
 \end{aligned}$$

landin_rec uses the eponymous technique, presented as a fixed-point combinator. It allocates a reference l (with a dummy initial value, $()$) and writes a function value to it. This function in turn reads from l and calls f with whatever is there. We take this description of *landin_rec* as its specification and focus on how to verify a use of it. Following Birkedal and Bizjak [42],

we verify a recursive factorial function.

$$\begin{aligned}
& \text{fact}_f(\text{self}, n) \triangleq \\
& \quad \mathbf{ens} \ r.n \leq 0 \wedge r = 1 \\
& \quad \vee \mathbf{ens} \ n > 0; \text{self}(n-1); r_1. \mathbf{ret} \ n * r_1 \\
& \text{let fact } n \triangleq \text{if } n \leq 0 \text{ then } 1 \text{ else } n * \text{fact } (n-1)
\end{aligned}$$

We give an open-recursive version fact_f in staged logic, as well as a pure, mathematical version fact to serve as a specification.

We then prove the following entailment, which uses landin_rec to tie a knot before invoking it on n .

$$\begin{aligned}
& \text{landin_rec}(\text{fact}_f); f. f(n) \\
& \sqsubseteq \exists l v. \mathbf{ens} \ l \mapsto v; \mathbf{ret} \ (\text{fact } n)
\end{aligned}$$

The fact that a reference is allocated in the process of tying the knot is intentionally exposed. We quantify away the exact value, so it doesn't otherwise affect the proof.

The idea of the proof is to symbolically execute the knot until we surface how it is tied, at which point we can proceed by induction on n . This example has been mechanically verified in Rocq¹.

Proof. We start with the formula on the left of the entailment, and unfold and simplify it until we have a direct use of the knot.

$$\begin{aligned}
& \text{landin_rec}(\text{fact}_f); f. f(n) \\
& \sqsubseteq \exists l \text{ knot}. \mathbf{ens} \ l \mapsto (); \mathbf{ens} \ \text{knot} = (\lambda n. \text{read}(l); f_1. f(f_1, n)); \\
& \quad \text{write}(l, \text{knot}); \mathbf{ret} \ \text{knot}; f. f(n) \\
& \sqsubseteq \exists l \text{ knot}. \mathbf{ens} \ \text{knot} = (\lambda n. \text{read}(l); f_1. f(f_1, n)); \mathbf{ens} \ l \mapsto (); \\
& \quad \forall v_0. \mathbf{req} \ l \mapsto v_0; \mathbf{ens} \ l \mapsto \text{knot}; \mathbf{ret} \ \text{knot}; f. f(n) \\
& \sqsubseteq \exists l \text{ knot}. \mathbf{ens} \ \text{knot} = (\lambda n. \text{read}(l); f_1. f(f_1, n)); \mathbf{ens} \ l \mapsto \text{knot}; \mathbf{ret} \ \text{knot}; f. f(n) \\
& \sqsubseteq \exists l \text{ knot}. \mathbf{ens} \ \text{knot} = (\lambda n. \text{read}(l); f_1. f(f_1, n)); \mathbf{ens} \ l \mapsto \text{knot}; \text{knot}(n)
\end{aligned}$$

We then continue by induction on n .

¹https://github.com/dariusf/staged/blob/master/shiftreset/more_examples/Landin.v

Base case $\mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{knot}(0)$
 $\sqsubseteq \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ret\ (fact\ 0)}$

$\mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{knot}(0)$
 $\sqsubseteq \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{read}(l); f_1. \mathbf{fact}_f(f_1, 0)$
 $\sqsubseteq \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{fact}_f(\mathbf{knot}, 0)$
 $\sqsubseteq \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ens\ } r. n \leq 0 \wedge r = 1$
 $\sqsubseteq \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ret\ (fact\ 0)} \quad \square$

Inductive case $\mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{knot}(n_0 + 1)$
 $\sqsubseteq \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ret\ (fact\ } n_0 + 1)$

$\mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{knot}(n_0 + 1)$
 $\sqsubseteq \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{fact}_f(\mathbf{knot}, n_0 + 1)$
 $\sqsubseteq \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot};$
 $\quad \mathbf{ens\ } n_0 + 1 > 0; \mathbf{knot}((n_0 + 1) - 1); r_1. \mathbf{ret\ (} n_0 + 1) * r_1$
 $\sqsubseteq \mathbf{ens\ } n_0 + 1 > 0;$
 $\quad \mathbf{ens\ knot} = (\lambda n. \mathbf{read}(l); f_1. f(f_1, n)); \mathbf{ens\ } l \mapsto \mathbf{knot};$
 $\quad \mathbf{knot}(n_0); r_1. \mathbf{ret\ (} n_0 + 1) * r_1$
 $\sqsubseteq \mathbf{ens\ } n_0 + 1 > 0; \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ret\ fact\ } n_0; r_1. \mathbf{ret\ ((} n_0 + 1) * r_1)$
 $\sqsubseteq \mathbf{ens\ } n_0 + 1 > 0; \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{ret\ ((} n_0 + 1) * \mathbf{fact\ } n_0)$
 $\sqsubseteq \mathbf{ens\ } n_0 + 1 > 0; \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{fact\ (} n_0 + 1)$
 $\sqsubseteq \mathbf{ens\ } l \mapsto \mathbf{knot}; \mathbf{fact\ (} n_0 + 1) \quad \square$

□

5.2 Interpreting regular expressions

In this section, we verify a series of interpreters for a simple language of regular expressions, defined in OCaml as follows.

```
type regex =
  | Emp
  | Atom of int
  | Seq of regex * regex
  | Disj of regex * regex
```

The regular expression $(1|2)3$ would be represented as `Seq (Disj (Atom 1)(Atom 2))(Atom 3)`. The strings matching regular expressions are assumed to be lists of integers. `Emp` matches the empty string, an atom matches

```

let rec matches_prefix r ns : int list option =
  match r with
  | Emp -> Some ns
  | Atom a ->
    begin match ns with
    | b :: ns1 when a = b -> Some ns1
    | _ -> None
    end
  | Seq (r1, r2) ->
    begin match matches_prefix r1 ns with
    | None -> None
    | Some rest -> matches_prefix r2 rest
    end
  | Disj (_, _) -> None (* incomplete *)

let matches r ns : bool =
  match matches_prefix r ns with
  | Some [] -> true
  | _ -> false

```

Fig. 5.1. A regular expression interpreter

a string consisting of a single integer, and regular expressions may be sequenced and may use disjunction.

A first attempt at an interpreter for it is shown in [Fig. 5.1](#). `matches` is defined in terms of an auxiliary function, `matches_prefix`, which evaluates a regular expression `r` against a string of numbers `ns`. Its result is the suffix of the input remaining after removing the numbers satisfying `r`, or `None` if there is no match. `matches` then simply checks if the entire string is matched, i.e. if there is a match and the string remaining is empty.

The first three cases are easy to define. `Emp` always matches and leaves the string unchanged. `Atom a` matches only if the input string is nonempty and the first element is exactly `a`. `Seq` matches if the string remaining after matching `r1` can be used to match `r2`, and the result is the string remaining afterward.

The case for disjunction is trickier. Consider how we would interpret the example from earlier, `Seq (Disj (Atom 1)(Atom 2))(Atom 3)`. We can descend into the first argument of `Seq` as before, but once there, we can return

```

let rec matches_prefix_cps r ns k : int list option =
  match r with
  | Emp -> k (Some ns)
  | Atom a ->
    k (match ns with
      | b :: ns1 when a = b -> Some ns1
      | _ -> None)
  | Seq (r1, r2) ->
    matches_prefix_cps r1 ns (function
      | None -> None
      | Some rest -> matches_prefix_cps r2 rest k)
  | Disj (r1, r2) ->
    (match matches_prefix_cps r1 ns k with
      | None -> matches_prefix_cps r2 ns k
      | r -> r)

let matches_cps r ns : bool =
  match matches_prefix_cps r ns (fun x -> x) with
  | Some [] -> true
  | _ -> false

```

Fig. 5.2. A continuation-composing regular expression interpreter

two possible suffixes, depending on which disjunct we pick, as well which of the two eventually leads to success. In other words, we may have to back-track. However, the present return type does not provide a way to produce both suffixes. We sidestep this for now by simply making disjunction never match.

One solution is to generalise the type of the function to `int list list`, where we produce every possible choice. We consider instead two continuation-based solutions.

`matches_prefix_cps` (Fig. 5.2) generalises `maches_prefix` so that it takes a continuation `k`, with answer type as annotated. The first three cases now use the continuation to produce a result; the case for `Seq` uses continuations to receive intermediate results, with `k` getting only the final result. Disjunction can now be implemented by using the continuation twice, and examining the result of matching `r1` for failure (making the interpreter continuation-

```

let rec matches_prefix_sr r ns =
  match r with
  | Emp -> Some ns
  | Atom a ->
    (match ns with
    | b :: ns1 when a = b -> Some ns1
    | _ -> None)
  | Seq (r1, r2) ->
    let v = matches_prefix_sr r1 ns in
    (match v with
    | None -> None
    | Some rest -> matches_prefix_sr r2 rest)
  | Disj (r1, r2) ->
    shift (fun k ->
      match k (matches_prefix_sr r1 ns) with
      | None -> k (matches_prefix_sr r2 ns)
      | v -> v)

let matches_sr r ns : bool =
  match reset (matches_prefix_sr r ns) with
  | Some [] -> true
  | _ -> false

```

Fig. 5.3. A regular expression interpreter using shift/reset [67, 68]

composing).

A solution using shift/reset is shown in Fig. 5.3. The beauty of it is that it all the cases which do not rely on manipulating continuations are identical to the version in Fig. 5.1. However, in this version, disjunction can be implemented without any other changes, by using shift to access the continuation of that branch, peek into its outcome, and “restart” execution with the second disjunct if the first leads to failure. The only other change is the delimiter around the scrutinee of the match in matches_sr.

5.2.1 Verification

Some properties of the simple interpreter are shown in Fig. 5.4. For example, *Emp* is the left and right unit of sequencing, and sequencing is associative.

$$\begin{aligned}
& \text{matches Seq}(\text{Emp}, r) \text{ ns} = \text{matches } r \text{ ns} \\
& \text{matches Seq}(r, \text{Emp}) \text{ ns} = \text{matches } r \text{ ns} \\
& \text{matches Seq}(r_1, \text{Seq}(r_2, r_3)) \text{ ns} = \text{matches Seq}(\text{Seq}(r_1, r_2), r_3) \text{ ns} \\
& \frac{\text{matches } r \text{ ns} = \text{true}}{\text{matches_prefix } r \text{ ns} = \text{Some } []} \qquad \frac{\text{matches_prefix } r \text{ ns} = \text{Some } \text{rest}}{\exists \text{ ns}'. \text{ ns} = \text{ns}' ++ \text{rest}}
\end{aligned}$$

Fig. 5.4. Properties of the interpreter in Fig. 5.1

$$\begin{aligned}
& \text{matches_cps Disj}(r, r) \text{ ns} = \text{matches_cps } r \text{ ns} \\
& \text{matches_cps Seq}(\text{Disj}(r_1, r_2), r) \text{ ns} = \text{matches_cps Disj}(\text{Seq}(r_1, r), \text{Seq}(r_2, r)) \text{ ns} \\
& \text{matches_cps Disj}(\text{Emp}, r) \text{ ns} = \text{matches_cps Emp ns} \\
& \frac{\text{Disj_free } r}{k (\text{matches_prefix } r \text{ ns}) = \text{matches_prefix_cps } r \text{ ns } k}
\end{aligned}$$

Fig. 5.5. Properties of the continuation-composing interpreter in Fig. 5.2

matches has a *true* result only if *matches_prefix* produces an empty suffix, and the result of *matches_prefix* is indeed a suffix of the input it is given. Since the interpreter is pure and terminating, it can be thought of as a mathematical function and modelled that way in a proof assistant. These properties can then be proved using standard techniques: case analysis suffices for all except the last one, which requires induction.

It would be good to have similar assurances about the two continuation-based interpreters. We can treat the continuation-composing interpreter the same way, provided *k* is only instantiated with pure functions. It is easy to prove that disjunction is idempotent and distributes over sequencing, and *Emp* is absorbing on the left of disjunction.

Interestingly, one might assume that disjunction is commutative, but in our interpreter, it is not. A counterexample is the regular expression *Disj (Atom 1, (Seq (Atom 1, Atom 1)))* and the input *11*, for which we get different suffixes from *matches_prefix_cps*.

$$\frac{k^\#}{\langle \text{matches_prefix_sr } r \text{ ns}; r_1. k(r_1) \rangle \sqsubseteq \text{matches_prefix_cps } r \text{ ns } k}$$

Fig. 5.6. Relating the shift/reset (Fig. 5.3) and continuation-composing interpreters

It would be tedious to prove all the properties of sequencing and `Emp` again. We can get them all by proving a statement relating *matches* and *matches_cps*, and using transitivity. The proof comes down to the last lemma in Fig. 5.5, which can again be shown by induction on *r*. The *Disj_free* side condition simply ensures that there is syntactically no disjunction in *r*, as *matches_prefix* has a meaningless implementation of it.

All proofs up to this point have been mechanically verified in Lean².

Finally, we move on the last interpreter. As we saw in Section 2.1, the approach of treating the program as a mathematical function breaks down with the addition of control effects that cannot be expressed in a standard higher-order logic. Consequently, we switch to using staged logic.

To reuse our earlier work, we relate the last interpreter to the continuation-composing one using the lemma in Fig. 5.6. In this way, *matches_prefix_cps* serves as a specification, or reference implementation, of *matches_prefix_sr*. We generalise over the (shift-free) continuation of the reset in order to relate it to the explicit continuation, a technique first used in Section 4.1.2.

We first model both programs in staged logic, in Fig. 5.7. We then prove the following entailment.

$$\begin{aligned} & \langle \text{matches_prefix_sr}(r, \text{ns}) \rangle; r_1. k(r_1) \\ & \sqsubseteq \text{matches_prefix_cps } r, \text{ns}, k \end{aligned}$$

The proof goes by induction on *r*. We give a detailed account of the proof below, but it is otherwise routine. The point is that it is simplified by the use of staged logic, as there is no need to invent abstractions for the uses of shift.

²<https://github.com/dariusf/regex-interpreter/blob/master/Regex.lean>

$$\begin{aligned}
\text{matches_prefix_sr}(r, ns) &\triangleq \\
&\mathbf{ens} \ r = \mathbf{Emp}; \mathbf{ret} \ \mathbf{Some} \ ns \\
&\vee \exists a. \mathbf{ens} \ r = (\mathbf{Atom} \ a); \\
&\quad (\exists b \ ns_1. \mathbf{ens} \ ns = b :: ns_1 \wedge a = b; \mathbf{ret} \ (\mathbf{Some} \ ns_1) \vee \mathbf{ret} \ \mathbf{None}) \\
&\vee \exists r_1 \ r_2. \mathbf{ens} \ r = (\mathbf{Seq}(r_1, r_2)); \\
&\quad \text{matches_prefix_sr}(r_1, ns); v. \\
&\quad (\mathbf{ens} \ v = \mathbf{None}; \mathbf{ret} \ \mathbf{None} \vee \\
&\quad \quad (\exists \text{rest}. \mathbf{ens} \ v = (\mathbf{Some} \ \text{rest}); \text{matches_prefix_sr}(r_2, \text{rest}))) \\
&\vee \exists r_1 \ r_2. \mathbf{ens} \ r = (\mathbf{Disj}(r_1, r_2)); \\
&\quad \mathbf{shift} \ k. \text{matches_prefix_sr}(r_1, ns); v_1. \ k(v_1); r_3. \\
&\quad (\mathbf{ens} \ r_3 = \mathbf{None}; \text{matches_prefix_sr}(r_2, ns); a. \ k(a)) \vee \\
&\quad (\exists a. \mathbf{ens} \ r_3 = (\mathbf{Some} \ a); \mathbf{ret} \ r_3)
\end{aligned}$$

$$\begin{aligned}
\text{matches_prefix_cps}(r, ns, k) &\triangleq \\
&\mathbf{ens} \ r = \mathbf{Emp}; k(\mathbf{Some} \ ns) \\
&\vee \exists a. \mathbf{ens} \ r = (\mathbf{Atom} \ a); \\
&\quad (\exists b \ ns_1. \mathbf{ens} \ ns = b :: ns_1 \wedge a = b; k(\mathbf{Some} \ ns_1) \vee k(\mathbf{None})) \\
&\vee \exists r_1 \ r_2. \mathbf{ens} \ r = (\mathbf{Seq}(r_1, r_2)); \\
&\quad \text{matches_prefix_cps}(r_1, ns, \lambda v. \\
&\quad \quad (\mathbf{ens} \ v = \mathbf{None}; k(\mathbf{None}) \vee \\
&\quad \quad \quad (\exists \text{rest}. \mathbf{ens} \ v = (\mathbf{Some} \ \text{rest}); \text{matches_prefix_cps}(r_2, \text{rest}, k)))) \\
&\vee \exists r_1 \ r_2. \mathbf{ens} \ r = (\mathbf{Disj}(r_1, r_2)); \\
&\quad \text{matches_prefix_cps}(r_1, ns, k); r_3. \\
&\quad (\mathbf{ens} \ r_3 = \mathbf{None}; \text{matches_prefix_cps}(r_2, ns, k) \vee k(r_3))
\end{aligned}$$

Fig. 5.7. *matches_prefix_cps* and *matches_prefix_sr* in staged logic

$$\begin{aligned}
\text{Case } r = \text{Emp} \quad & \langle \mathbf{ens} \ r = \text{Emp}; \mathbf{ret} \ \text{Some } ns; r_1. k(r_1) \rangle \\
& \sqsubseteq \mathbf{ens} \ r = \text{Emp}; k(\text{Some } ns) \\
& \quad \langle \mathbf{ens} \ r = \text{Emp}; \mathbf{ret} \ \text{Some } ns; r_1. k(r_1) \rangle \\
& \sqsubseteq \langle \mathbf{ens} \ r = \text{Emp}; k(\text{Some } ns) \rangle \quad \text{Simplify} \\
& \sqsubseteq \mathbf{ens} \ r = \text{Emp}; k(\text{Some } ns) \quad \square \quad \text{Pure}
\end{aligned}$$

$$\begin{aligned}
\text{Case } r = \text{Atom } \dots \quad & \langle \exists a. \mathbf{ens} \ r = \text{Atom } a; \\
& \quad ((\exists b \ ns_1. \mathbf{ens} \ ns = b :: ns_1 \wedge a = b; \mathbf{ret} \ \text{Some } ns_1) \vee \\
& \quad \mathbf{ret} \ \text{None}); r_1. k(r_1) \rangle \\
& \sqsubseteq \exists a. \mathbf{ens} \ r = \text{Atom } a; \\
& \quad \exists b \ ns_1. \mathbf{ens} \ ns = b :: ns_1 \wedge a = b; k(\text{Some } ns_1) \vee \\
& \quad k(\text{None})
\end{aligned}$$

We eliminate the existentials and **ens** proof obligations, and match the disjuncts. This pattern is followed in the other cases.

$$\begin{aligned}
& \langle \mathbf{ret} \ (\text{Some } ns_1) \vee \mathbf{ret} \ \text{None}; r_1. k(r_1) \rangle \\
& \sqsubseteq k(\text{Some } ns) \vee k(\text{None})
\end{aligned}$$

The goal then follows from simplification and the fact that k is shift-free.

$$\begin{aligned}
& \langle \mathbf{ret} \ (\text{Some } ns_1) \vee \mathbf{ret} \ \text{None}; r_1. k(r_1) \rangle \\
& \sqsubseteq \langle k(\text{Some } ns_1) \vee k(\text{None}) \rangle \quad \text{Simplify} \\
& \sqsubseteq k(\text{Some } ns) \vee k(\text{None}) \quad \square \quad \text{Pure}
\end{aligned}$$

$$\begin{aligned}
\text{Case } r = \text{Seq } \dots \quad & \langle \exists r_1 \ r_2. \mathbf{ens} \ r = \text{Seq}(r_1, r_2); \\
& \quad \text{matches_prefix_sr}(r_1, ns); v. (\mathbf{ens} \ v = \text{None}; \mathbf{ret} \ \text{None} \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v = (\text{Some } \text{rest}); \\
& \quad \text{matches_prefix_sr}(r_2, \text{rest})); r_3. k(r_3) \rangle \\
& \sqsubseteq \exists r_1 \ r_2. \mathbf{ens} \ r = \text{Seq}(r_1, r_2); \\
& \quad \text{matches_prefix_cps}(r_1, ns, \lambda v. \mathbf{ens} \ v = \text{None}; k(\text{None}) \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v = (\text{Some } \text{rest}); \\
& \quad \text{matches_prefix_cps}(r_2, \text{rest}, k)))
\end{aligned}$$

The induction hypotheses are

$$\forall ns \ k. \langle \text{matches_prefix_sr}(r_i, ns); r_3. k(r_3) \rangle \sqsubseteq \text{matches_prefix_cps}(r_i, ns, k)$$

for $i \in \{1, 2\}$, assuming k is shift-free.

We first eliminate the existentials and cancel **ens** on both sides.

$$\begin{aligned}
& \langle \text{matches_prefix_sr}(r_1, ns); v. (\mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None} \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_sr}(r_2, \text{rest}))) \rangle; r_3. k(r_3) \rangle \\
& \sqsubseteq \text{matches_prefix_cps}(r_1, ns, \lambda v. \mathbf{ens} \ v=\text{None}; k(\text{None}) \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_cps}(r_2, \text{rest}, k)))
\end{aligned}$$

We need the lemma [112, Section 3.2]

Lemma 5.1 (Reset under bind). $\langle \varphi_1; r. \varphi_2 \rangle \equiv \langle \varphi_1; r. \langle \varphi_2 \rangle \rangle$

We then successively simplify the left side.

$$\begin{aligned}
& \langle \text{matches_prefix_sr}(r_1, ns); v. (\mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None} \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_sr}(r_2, \text{rest}))) \rangle; r_3. k(r_3) \rangle \\
& \sqsubseteq \langle \text{matches_prefix_sr}(r_1, ns); v. \langle (\mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None} \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_sr}(r_2, \text{rest}))) \rangle; r_3. k(r_3) \rangle \rangle \quad \text{Lemma 5.1} \\
& \sqsubseteq \text{matches_prefix_cps}(r_1, ns, \lambda v. \langle (\mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None} \vee \\
& \quad (\exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_sr}(r_2, \text{rest}))) \rangle; r_3. k(r_3) \rangle \rangle) \quad \text{Ind. hyp. 1} \\
& \sqsubseteq \text{matches_prefix_cps}(r_1, ns, \lambda v. \\
& \quad (\langle \mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None}; r_3. k(r_3) \rangle \vee \\
& \quad \langle \exists \text{rest}. \mathbf{ens} \ v=(\text{Some } \text{rest}); \\
& \quad \quad \text{matches_prefix_sr}(r_2, \text{rest}); r_3. k(r_3) \rangle))
\end{aligned}$$

The application of the induction hypothesis requires us to prove that what we instantiate k is with shift-free, which it is due to the reset. The last step follows from distributing disjunction over sequencing, and reset over disjunction.

Because $\sqsubseteq$ is a precongruence with respect to functions, it suffices to prove the following two entailments.

$$\begin{aligned}
& \langle \mathbf{ens} \ v=\text{None}; \mathbf{ret} \ \text{None}; r_3. k(r_3) \rangle \\
& \sqsubseteq \mathbf{ens} \ v=\text{None}; k(\text{None})
\end{aligned}$$

This can be proved by simplification.

$$\begin{aligned} & \langle \exists \text{rest}. \mathbf{ens} \ v = (\text{Some } \text{rest}); \text{matches_prefix_sr}(r_2, \text{rest}); r_3. k(r_3) \rangle \\ & \sqsubseteq \exists \text{rest}. \mathbf{ens} \ v = (\text{Some } \text{rest}); \text{matches_prefix_cps}(r_2, \text{rest}, k) \end{aligned}$$

This is exactly induction hypothesis 2 after simplification, and k is shift-free. $\square$

$$\begin{aligned} \text{Case } r = \text{Disj } \dots & \quad \langle \exists r_1 r_2. \mathbf{ens} \ r = \text{Disj}(r_1, r_2); \mathbf{shift} \ k_1. \\ & \quad \text{matches_prefix_sr}(r_1, ns); v_1. k_1(v_1); r_3. \\ & \quad ((\mathbf{ens} \ r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns); a. k_1(a)) \\ & \quad \vee (\exists a. \mathbf{ens} \ r_3 = (\text{Some } a); \mathbf{ret} \ r_3)); r_4. k(r_4) \rangle \\ & \sqsubseteq \exists r_1 r_2. \mathbf{ens} \ r = \text{Disj}(r_1, r_2); \text{matches_prefix_cps}(r_1, ns, k); r_3. \\ & \quad (\mathbf{ens} \ r_3 = \text{None}; \text{matches_prefix_cps}(r_2, ns, k) \vee k(r_3)) \end{aligned}$$

The induction hypotheses are

$$\forall ns \ k. \langle \text{matches_prefix_sr}(r_i, ns); r_3. k(r_3) \rangle \sqsubseteq \text{matches_prefix_cps}(r_i, ns, k)$$

for $i \in \{1, 2\}$, assuming k is shift-free.

First we eliminate existentials and proof obligations.

$$\begin{aligned} & \langle \mathbf{shift} \ k_1. \\ & \quad \text{matches_prefix_sr}(r_1, ns); v_1. k_1(v_1); r_3. \\ & \quad ((\mathbf{ens} \ r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns); a. k_1(a)) \\ & \quad \vee (\exists a. \mathbf{ens} \ r_3 = (\text{Some } a); \mathbf{ret} \ r_3)); r_4. k(r_4) \rangle \\ & \sqsubseteq \text{matches_prefix_cps}(r_1, ns, k); r_3. \\ & \quad (\mathbf{ens} \ r_3 = \text{None}; \text{matches_prefix_cps}(r_2, ns, k) \vee k(r_3)) \end{aligned}$$

Given the shift immediately inside the reset, we can perform reduction, followed by simplification and application of [Lemma 5.1](#), as in the previous case.

$$\begin{aligned}
& \langle \text{shift } k_1. \\
& \quad \text{matches_prefix_sr}(r_1, ns); v_1. k_1(v_1); r_3. \\
& \quad ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)); a. k_1(a)) \\
& \quad \vee (\exists a. \text{ens } r_3 = (\text{Some } a); \text{ret } r_3)); r_4. k(r_4) \rangle \quad \text{SR reduction} \\
\sqsubseteq & \text{ens } k_1 = \text{id}; \\
& \langle \text{matches_prefix_sr}(r_1, ns); v_1. k_1(v_1); r_3. \\
& \quad ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)); a. k_1(a)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \quad \text{Simplify} \\
\sqsubseteq & \langle \text{matches_prefix_sr}(r_1, ns); r_3. \\
& \quad ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \quad \text{Lemma 5.1} \\
\sqsubseteq & \langle \text{matches_prefix_sr}(r_1, ns); r_3. \\
& \quad \langle ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \rangle \quad \text{Ind. hyp. 1} \\
\sqsubseteq & \text{matches_prefix_cps}(r_1, ns, \lambda r_3. \\
& \quad \langle ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \rangle
\end{aligned}$$

Again, we use [Lemma 5.1](#) to enable the use of the induction hypothesis, as it gives us a second reset to rewrite with, and also ensures that the continuation we instantiate the IH with is shift-free.

$$\begin{aligned}
& \text{matches_prefix_cps}(r_1, ns, \lambda r_3. \\
& \quad \langle ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \rangle \\
\sqsubseteq & \text{matches_prefix_cps}(r_1, ns, k); r_3. \\
& (\text{ens } r_3 = \text{None}; \text{matches_prefix_cps}(r_2, ns, k) \vee k(r_3))
\end{aligned}$$

Using the fact that $\sqsubseteq$ is a precongruence,

$$\begin{aligned}
& \langle ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns)) \\
& \quad \vee (\exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3)); r_4. k(r_4) \rangle \\
\sqsubseteq & (\text{ens } r_3 = \text{None}; \text{matches_prefix_cps}(r_2, ns, k) \vee k(r_3))
\end{aligned}$$

Matching the disjuncts, the second is immediate.

$$\langle \exists a. \text{ens } r_3 = \text{Some } a; \text{ret } r_3; r_4. k(r_4) \rangle \sqsubseteq k(r_3)$$

In the first,

$$\begin{aligned}
& \langle ((\text{ens } r_3 = \text{None}; \text{matches_prefix_sr}(r_2, ns))); r_4. k(r_4) \rangle \\
\sqsubseteq & (\text{ens } r_3 = \text{None}; \text{matches_prefix_cps}(r_2, ns, k))
\end{aligned}$$

cancelling **ens** on both sides,

$$\begin{aligned} & \langle ((\text{matches_prefix_sr}(r_2, ns)); r_4. k(r_4)) \\ & \sqsubseteq (\text{matches_prefix_cps}(r_2, ns, k) \end{aligned}$$

we get exactly induction hypothesis 2. □

5.3 Program equivalence

<pre> int doubleSquare1(int x) { int y = 0; int z = 2 * x; while (z > 0) { y = y + x; z--; } return y; } </pre>	<pre> int doubleSquare2(int x) { int y = 0; int z = x; while (z > 0) { y = y + x; z--; } return 2 * y; } </pre>
--	--

Fig. 5.8. Two programs which, given x , compute $2x^2$ [178]

Consider the two programs in Fig. 5.8. `doubleSquare2` is a faster program, with half the number of loop iterations. We would like to prove `doubleSquare1` $\equiv$ `doubleSquare2`, e.g. to validate a compiler optimisation. As the two objects we are proving equivalence of are (stateful) programs, we can view this as a relational verification problem of a $\forall\forall$ property, of the form

$$\forall s_1 \forall s_2. s_1 \sim s_2 \Rightarrow \llbracket \text{doubleSquare1} \rrbracket s_1 \sim \llbracket \text{doubleSquare2} \rrbracket s_2$$

where $\sim$ is some kind of equivalence relation, and s_1 and s_2 are program states.

There are two families of approaches to proving relational properties of this form. The first, "product programs", reduces the relational verification problem to one in standard Hoare logic by constructing a new program such that its verification soundly implies that of the original program. The second

```

method doubleSquareSequenced (x1:int, x2:int)
  returns (y1: int, y2: int)
  requires x1 >= 0 && x2 >= 0
  requires x1 == x2
  ensures y1 == y2
{
  y1 := 0;
  var z1 := 2 * x1;
  while (z1 > 0)
    invariant y1 == x1 * (2 * x1 - z1)
    invariant z1 >= 0
    {
      y1 := y1 + x1;
      z1 := z1 - 1;
    }
  y2 := 0;
  var z2 := x2;
  while (z2 > 0)
    invariant y2 == x2 * (x2 - z2)
    invariant z2 >= 0
    {
      y2 := y2 + x2;
      z2 := z2 - 1;
    }
  y2 := 2 * y2;
}

```

Fig. 5.9. Alignment of programs by sequencing

is to use a relational Hoare logic [36], a system of rules for composing and constructing *quadruples* of the form $\{P\} e_1 \mid e_2 \{Q\}$.

I describe the first approach and briefly comment on the second, to illustrate the key issue of *alignment*. After that, I show in detail how staged logic can be used to carry out similar equivalence proofs.

Product programs The simplest way to construct a product program for this situation is append them together, and modify them to act on disjoint state (Fig. 5.9).

We have to provide two loop invariants which “align” the two programs:


```

method doubleSquareInterleaved (x1:int, x2:int)
  returns (y1:int, y2:int)
  requires x1 >= 0 && x2 >= 0
  requires x1 == x2
  ensures y1 == y2
{
  y1 := 0;
  y2 := 0;
  var z1 := 2 * x1;
  var z2 := x2;
  while (z1 > 0 && z2 > 0)
    invariant y1 == 2 * y2 // simpler
  {
    y1 := y1 + x1;
    y2 := y2 + x2;
    if (z1 > 0) { // unroll
      y1 := y1 + x1;
      z1 := z1 - 1;
    }
    z1 := z1 - 1;
    z2 := z2 - 1;
  }
  y2 := y2 * 2;
}

```

Fig. 5.10. Alignment of programs by interleaving statements

to say that one iteration of one program corresponds to two iterations in another, by relating their intermediate states. This lets us prove that the states at the end are equal.

We can construct a better alignment by interleaving the statements of the two programs (Fig. 5.10). Note that we have to explicitly unroll the loop of `doubleSquare1` once. The loop invariant then becomes simpler³.

Relational Hoare logic In the second approach, using a relational Hoare logic, the issue of alignment is internalised in its proof rules. For example, in Relational Separation Logic [208], the **WHILE** rule only applies to two pro-

³Both programs were inspired by Dickerson et al. [77] and are verified with Dafny.

$$\begin{aligned}
\text{loop}_1(x, y, z) &\triangleq \\
&\quad \mathbf{ens} \ z > 0; \text{loop}_1(y+x, z-1) \\
&\quad \vee \mathbf{ens} \ r. z \leq 0 \wedge r=y \\
\text{doubleSquare}_1(x) &\triangleq \text{loop}_1(x, 0, 2 * x) \\
\text{doubleSquare}_2(x) &\triangleq \text{loop}_1(x, 0, x); r_1. \mathbf{ens} \ r. r = r_1 * 2
\end{aligned}$$

Fig. 5.11. Functional presentation of the loop

grams with top-level while loops, similar to the second example above.

$$\frac{\text{Inv} \Rightarrow (B_1 \Leftrightarrow B_2) \quad \{ \text{Inv} \wedge B_1 \} C_1 \mid C_2 \{ \text{Inv} \}}{\{ \text{Inv} \} \text{while } B_1 \text{ do } C_1 \mid \text{while } B_2 \text{ do } C_2 \{ \text{Inv} \wedge \neg B_1 \}} \text{WHILE}$$

In other words, the user can only verify programs with similar control flow structure, and have to get programs into the right form either by rewriting them or using things like one-sided rules.

5.3.1 Staged logic

As staged logic is able to model programs faithfully, even stateful and effectful ones, equivalence can be directly stated and proved. The issue of alignment does recur.

This section shows two proofs using staged logic, one using a functional presentation, the other using a more faithful imperative one.

Functional One view of the loop is as a tail-recursive functional program. We model it in staged logic in Fig. 5.11. For simplicity, z is assumed to be a natural number.

We would like to prove $\forall x. \text{doubleSquare}_1(x) \equiv \text{doubleSquare}_2(x)$. To do this, we first prove a more general lemma:

Lemma 5.2 (Alignment between loop iterations).

$$\text{loop}_1(x, y + y, 2 * z) \equiv \text{loop}_1(x, y, z); r_1. \mathbf{ens} \ r. r = 2 * r_1$$

This is where the alignment occurs: saying how one iteration of the loop corresponds to two. The proof goes by induction on z .

Base case $z = 0$

$$\text{loop}_1(x, y + y, 2 * 0) \equiv \text{loop}_1(x, y, 0); r_1. \mathbf{ens} \ r. r = 2 * r_1$$

Simplifying,

$$\mathbf{ens} \ r. r = y + y \equiv \mathbf{ens} \ r. r = 2 * y \quad \square$$

Inductive case $z = z_0 + 1$

$$\text{loop}_1(x, y + y, 2 * (z_0 + 1)) \equiv \text{loop}_1(x, y, z_0 + 1); r_1. \mathbf{ens} \ r. r = 2 * r_1$$

$$\begin{aligned} & \text{loop}_1(x, y + y, 2 * (z_0 + 1)) \\ & \equiv \text{loop}_1(x, y + y, 2 + 2 * z_0) \\ & \equiv \text{loop}_1(x, y + y + x + x, 2 * z_0) \\ & \equiv \text{loop}_1(x, (x + y) + (x + y), 2 * z_0) \\ & \equiv \text{loop}_1(x, x + y, z_0); r_1. \mathbf{ens} \ r. r = 2 * r_1 \quad \text{Ind. hyp.} \\ & \equiv \text{loop}_1(r, x, 1 + z_0); r_1. \mathbf{ens} \ r. r = 2 * r_1 \quad \square \end{aligned}$$

The original goal follows easily from this lemma.

$$\begin{aligned} & \text{doubleSquare}_1(x) \\ & \equiv \text{loop}_1(x, 0, 2 * x) \\ & \equiv \text{loop}_1(x, 0 + 0, 2 * x) \\ & \equiv \text{loop}_1(x, 0, x); r_1. \mathbf{ens} \ r. r = 2 * r_1 \quad \text{Lemma 5.2} \\ & \equiv \text{doubleSquare}_2(x) \quad \square \end{aligned}$$

Imperative The imperative behaviours of the loop can also be modelled imperatively, and an advantage of staged logic is that the same proof methods can be used.

In Fig. 5.12, we first define stateful operations in terms of **req** and **ens**. loop_2 is defined using these, and used in the two programs below. This time, the *value* of z is assumed to be a natural number; note that it is this value which decreases in loop_2 , whose arguments otherwise remain unchanged.

This time, it is difficult to align the programs by the number of loop iterations, as that is not exposed as one of its parameters. Instead, we align

$$\begin{aligned}
incr(l, n) &\triangleq \forall a. \mathbf{req} \ l \mapsto a; \mathbf{ens} \ l \mapsto a + n \\
decr(l, n) &\triangleq \forall a. \mathbf{req} \ l \mapsto a; \mathbf{ens} \ l \mapsto a - n \\
read(l) &\triangleq \forall a. \mathbf{req} \ l \mapsto a; \mathbf{ens} \ r. l \mapsto a * [r = a] \\
\\
loop_2(x, y, z) &\triangleq \forall a. \\
&\quad \mathbf{req} \ z \mapsto a \wedge a > 0; \mathbf{ens} \ z \mapsto a; \\
&\quad incr(y, x); decr(z, 1); loop_2(x, y, z) \\
&\quad \wedge \mathbf{req} \ z \mapsto a \wedge a \leq 0; \mathbf{ens} \ z \mapsto a; \mathbf{ret} \ () \\
\\
doubleSquare_1(x) &\triangleq \exists y z. \mathbf{ens} \ y \mapsto 0 * z \mapsto 2 * x; loop_2(x, y, z); read(y) \\
doubleSquare_2(x) &\triangleq \exists y z. \mathbf{ens} \ y \mapsto 0 * z \mapsto x; loop_2(x, y, z); read(y); r. \mathbf{ret} \ r * 2
\end{aligned}$$

Fig. 5.12. Imperative presentation of the loop

them by loop structure, by writing a lemma that describes the net effect of the loop. It increments the value of y by $z * x$ and sets z to 0. Hence, the following specification for it:

Lemma 5.3 (Net effect of imperative loop).

$$loop_2(x, y, z) \sqsubseteq \forall a b. \mathbf{req} \ y \mapsto a * z \mapsto b; \mathbf{ens} \ y \mapsto (a + b * x) * z \mapsto 0$$

Let b_0 be the initial value of z . The lemma can be proved by well-founded induction on b_0 .

The induction hypothesis is:

$$\begin{aligned}
&\forall b. b < b_0 \Rightarrow \\
&\quad loop_2(x, y, z) \\
&\quad \sqsubseteq \forall a. \mathbf{req} \ y \mapsto a * z \mapsto b; \mathbf{ens} \ y \mapsto (a + b(x)) * z \mapsto 0
\end{aligned}$$

Unfolding and simplifying quantifiers,

$$\begin{aligned}
&\mathbf{req} \ z \mapsto b_0 \wedge b_0 > 0; \mathbf{ens} \ z \mapsto b_0; \\
&\quad incr(y, x); decr(z, 1); loop_2(x, y, z) \\
&\quad \wedge \mathbf{req} \ z \mapsto b_0 \wedge b_0 \leq 0; \mathbf{ens} \ z \mapsto b_0; \mathbf{ret} \ () \\
&\sqsubseteq \mathbf{req} \ y \mapsto a * z \mapsto b_0; \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0
\end{aligned}$$

To use one of the two conjuncts, we proceed by case analysis on b_0 .

Case $b_0 > 0$

$$\begin{aligned} & \mathbf{req} \ z \mapsto b_0; \mathbf{ens} \ z \mapsto b_0; \\ & \quad \mathit{incr}(y, x); \mathit{decr}(z, 1); \mathit{loop}_2(x, y, z) \\ \sqsubseteq & \mathbf{req} \ y \mapsto a * z \mapsto b_0; \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

The first step is symbolically execute the stateful operations. We do this by moving the **reqs** to the left,

$$\begin{aligned} & \mathbf{ens} \ y \mapsto a * z \mapsto b_0; \\ & \mathbf{req} \ z \mapsto b_0; \mathbf{ens} \ z \mapsto b_0; \\ & \quad \mathit{incr}(y, x); \mathit{decr}(z, 1); \mathit{loop}_2(x, y, z) \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

then using biabduction:

$$\begin{aligned} & \mathbf{ens} \ y \mapsto a; \\ & \mathbf{ens} \ z \mapsto b_0; \\ & \quad \mathit{incr}(y, x); \mathit{decr}(z, 1); \mathit{loop}_2(x, y, z) \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

We can repeat this to symbolically execute the stateful operations in the obvious way.

$$\begin{aligned} & \mathbf{ens} \ y \mapsto (a + x); \\ & \mathbf{ens} \ z \mapsto (b_0 - 1); \\ & \quad \mathit{loop}_2(x, y, z) \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

We can then use the induction hypothesis for $b_0 - 1$.

$$\begin{aligned} & \mathbf{ens} \ y \mapsto (a + x); \\ & \mathbf{ens} \ z \mapsto (b_0 - 1); \\ & \quad \forall a'. \mathbf{req} \ y \mapsto a' * z \mapsto (b_0 - 1); \mathbf{ens} \ y \mapsto (a' + (b_0 - 1) * x) * z \mapsto 0 \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a' + b_0 * x) * z \mapsto 0 \end{aligned}$$

Letting a' be $a + x$, we can simplify this to the following entailment, which follows from reflexivity.

$$\begin{aligned} & \mathbf{ens} \ y \mapsto ((a + x) + (b_0 - 1) * x) * z \mapsto 0 \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a' + b_0 * x) * z \mapsto 0 \end{aligned}$$

Case $b_0 \leq 0$

$$\begin{aligned} & \mathbf{req} \ z \mapsto b_0; \mathbf{ens} \ z \mapsto b_0; \mathbf{ret} \ () \\ \sqsubseteq & \mathbf{req} \ y \mapsto a * z \mapsto b_0; \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

We use the same idea to move **req** to the left.

$$\begin{aligned} & \mathbf{ens} \ y \mapsto a * z \mapsto b_0; \mathbf{req} \ z \mapsto b_0; \mathbf{ens} \ z \mapsto b_0; \mathbf{ret} \ () \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

Using biabduction,

$$\begin{aligned} & \mathbf{ens} \ y \mapsto a * z \mapsto b_0; \mathbf{ret} \ () \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (a + b_0 * x) * z \mapsto 0 \end{aligned}$$

Since b_0 is a natural number, $b_0 * x = 0$. □

Having proved [Lemma 5.3](#), we can use it to show the equivalence of $\mathit{doubleSquare}_1$ and $\mathit{doubleSquare}_2$.

$$\begin{aligned} & \mathit{doubleSquare}_1(x) \\ \sqsubseteq & \exists y \ z. \mathbf{ens} \ y \mapsto 0 * z \mapsto 2 * x; \mathit{loop}_2(x, y, z); \mathit{read}(y) \\ \sqsubseteq & \mathbf{ens} \ y \mapsto 0 * z \mapsto 2 * x; \\ & \forall a \ b. \mathbf{req} \ y \mapsto a * z \mapsto b; \mathbf{ens} \ y \mapsto (a + b * x) * z \mapsto 0; \mathit{read}(y) \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (0 + (2 * x) * x) * z \mapsto 0; \mathit{read}(y) \\ \sqsubseteq & \mathbf{ret} \ 2 * (x * x) \end{aligned}$$

$$\begin{aligned} & \mathit{doubleSquare}_2(x) \\ \sqsubseteq & \exists y \ z. \mathbf{ens} \ y \mapsto 0 * z \mapsto x; \mathit{loop}_2(x, y, z); \mathit{read}(y); r. \mathbf{ret} \ r * 2 \\ \sqsubseteq & \mathbf{ens} \ y \mapsto 0 * z \mapsto x; \\ & \forall a \ b. \mathbf{req} \ y \mapsto a * z \mapsto b; \mathbf{ens} \ y \mapsto (a + b * x) * z \mapsto 0; \\ & \mathit{read}(y); r. \mathbf{ret} \ r * 2 \\ \sqsubseteq & \mathbf{ens} \ y \mapsto (0 + x * x) * z \mapsto 0; \\ & \mathit{read}(y); r. \mathbf{ret} \ r * 2 \\ \sqsubseteq & \mathbf{ret} \ 2 * (x * x) \end{aligned}$$

□

Conclusion Staged logic is very much geared towards proofs of refinement and equivalence, so it handles this particular kind of relational property well. Nontrivial generalisation may in general be required to come up with the alignment, but there is some flexibility in how to do so, as shown by the different routes we took for the functional and imperative versions of the proofs.

MECHANISING STAGED LOGIC

Chapters 2 to 4 gave an introduction to the use of staged logic, using an intuitive and idealised model. While this was convenient for exposition (and sufficient for the implementation of an automated verifier), it is not the full story. This chapter gives more precise definitions for implementation in a proof assistant. To that end, I formalise the theory presented in Chapter 4 (which builds on Chapter 2) in Rocq and prove its soundness.

6.1 A model using simple behavioural refinement

6.1.1 Staged formulae

```

Inductive flow : Type :=
| req : hprop → flow → flow
| ens : (val → hprop) → flow
| bind : flow → (val → flow) → flow
| fex : forall (A:Type), (A → flow) → flow
| fall : forall (A:Type), (A → flow) → flow
| unk : var → val → flow
| intersect : flow → flow → flow
| disj : flow → flow → flow
| shc : var → flow → (val → flow) → flow
| rs : flow → flow
| defun : var → (val → flow) → flow
| discard : var → flow.

```

Fig. 6.1. Mixed embedding of the syntax of staged logic

Syntax The syntax of staged formulae (Fig. 2.9) is represented using an inductive datatype (Fig. 6.1). The embedding is mixed. The overall structure is deeply embedded, to sidestep both impredicativity and the positivity restriction in representing uninterpreted functions (Section 6.1.3). However, it contains shallowly embedded elements, with higher-order abstract syntax

(HOAS) used for binders, due to the shallow embedding of the underlying separation logic, whose propositions are represented in a standard way [56] as functions of type $\text{heap} \rightarrow \text{Prop}$.

To highlight a number of subtleties and differences:

- `unk` (“unknown”) represents uninterpreted functions, which take a type of names `var`.
- `val` is a type of (dynamically-typed) values which also includes the constructor `vfptr : var → val` for *function pointers*.
- `shc` (representing **shift**^c, from Section 4.2) and `rs` represent shift and reset. The shift body is deeply embedded, but the continuation is shallowly embedded; the reason that the shift body allows its argument (a continuation) to “escape” via a function pointer value, as in **shift** k . **ret** k , while the continuation is only built up during reduction and never inspects its argument.
- `defun k (fun x ⇒ f)` represents (**ens** $k=(\lambda x. \varphi)$), which cannot be directly implemented, as lambdas are not in `flow` due to Rocq’s positivity restriction on definitions of inductive data types. Instead, `defun` associates a definition with the name `k` in the *specification environment* (Section 2.2). `discard` undoes the effect of a `defun` and is required for technical reasons, as the changes to the model made by `defun` must be undone in refinement proofs where the specification does not make a corresponding change. The model is also extended to have *two* specification environments, in order to capture the effects of `defun`. More on this in Section 6.1.3.

Semantics The semantics of staged formulae is defined inductively via a separate big-step interpretation relation. This follows the presentation in

Fig. 2.10 and 4.10 closely. The new cases for `defun` are shown in Fig. 6.2 to give a flavour for how things are encoded.

```

Inductive satisfies : senv → senv →
  heap → heap → result → flow → Prop :=

| s_defun s1 s2 h1 x uf :
  ~Fmap.indom s1 x →
  s2 = Fmap.update s1 x uf →
  satisfies s1 s2 h1 h1 (norm vunit) (defun x uf)

| s_discard s1 s2 h (f:var) :
  s2 = Fmap.remove s1 f →
  satisfies s1 s2 h h (norm vunit) (discard f)

```

Fig. 6.2. Semantics of staged logic (select cases)

`defun` adds a new name to the specification environment, while `discard` removes it. The name must be new so `defun` followed by `discard` is the identity.

Due to the `Inductive` interpretation of the semantics, it only includes terminating behaviours, if staged logic is seen as an abstract programming language. If it is seen as a logic, this means that derivations are finite: it is not possible to compromise soundness by creating an infinite derivation.

6.1.2 Entailment

We work entirely with an extensional account of entailment, following Section 2.3.

$$\varphi_1 \sqsubseteq \varphi_2 \triangleq \forall \mathcal{E} h_1 h_2 R. (\mathcal{E}, h_1, h_2, R \models \varphi_1 \Rightarrow \mathcal{E}, h_1, h_2, R \models \varphi_2)$$

The benefits of this encoding are twofold: (1) it ensures that all the various entailment rules (Fig. 2.17, 2.19, 3.11 and 4.12) are sound by construction, and (2) interpreting entailment directly in terms of Rocq’s implication lets us reuse its proof context and associated machinery for interactive proofs. Properties such as the contravariance of `req` also come for free.

Entailment does double duty in interactive proofs, serving as a means for stating rewriting rules (Fig. 2.17) as well as a *sequent* (Fig. 2.19), appearing “below the line” in the Rocq proof state display as a representation of both the staged logic context and goal. This in turn supports the view of staged logic proofs as performing a sequence of entailment-preserving rewrites, leading to a trivial goal that can be solved using an automated tactic.

Theorem 2.2.1 (Soundness of Entailment Rules). *Given an entailment $\varphi_a \sqsubseteq \varphi_c$, the consequent overapproximates the antecedent, i.e. if $\mathcal{E}, h_1, h_2, R \models \varphi_a$, then $\mathcal{E}, h_1, h_2, R \models \varphi_c$.*

Proof. By construction. Each rule is proved against the semantics directly. □

6.1.3 Avoiding impredicativity

As staged logic is designed for reasoning about higher-order programs, it admits many higher-order constructions:

- Lambda values which may be stored in the heap, yet may be annotated with specifications
- Lambda expressions at the specification level
- First-class continuations, used in constructions such as **shift** k . k , yet whose definition is only known in the course of reduction

These are convenient to have on pen and paper, as well as in the implementation of an automated verifier, but they are difficult to encode in Rocq, as they create a cycle between the definitions of values and the interpretation of staged formulae.

We use a combination of syntactic techniques to support these use cases.

- To give specifications to functions, the user can prove a specification for an identical expression as a separate Rocq [Lemma](#), then instantiate it as needed.
- The specification environment $\mathcal{E}$ links such specifications to their occurrences.
- To allow functions in the heap, we reify the names of function stages as *function pointer values* [152, 59] which refer to definitions in the specification environment.
- To support dynamic definition of continuations, we allow the specification environment to be updated by two new staged formula primitives, **defun** and **discard**. We extend the model of [Section 6.1.1](#) with a second specification environment, resulting in the interpretation relation $\mathcal{E}_1, \mathcal{E}_2, h_1, h_2, R \models \varphi$. **defun** $f (\lambda x. \varphi)$ is morally equivalent to the pen-and-paper **ens** $f = (\lambda x. \varphi)$, and extends $\mathcal{E}$ with a binding $[f \leftarrow (\lambda x. \varphi)]$, while **discard** simply removes a name that is already present – its purpose is mainly technical ([Fig. 6.2](#)).

The end result is that we can justify our pen-and-paper and automated proofs, while using a sound (but lower-level) representation when building proofs within Rocq.

6.1.4 Rewriting

Staged logic proofs primarily consist of rewrites. The mechanisation makes heavy use of Rocq’s *generalized rewriting* [190] to support this, which allows rewriting to be performed not just with equalities, but with user-defined equivalence relations or *preorders* (which need not be symmetric).

The entailment relation $\sqsubseteq$ is such a preorder, and may be used for rewriting given supporting lemmas demonstrating that each constructor of staged

logic is *monotonic* [26] or *proper* [190] with respect to it. These lemmas are provided as instances of a `Proper` typeclass. For example, the following key lemma (left), which states that entailment is monotonic with respect to itself, is given as the typeclass instance on the right. This allows a goal $\varphi_1 \sqsubseteq \varphi_2$ to be replaced with $\varphi_3 \sqsubseteq \varphi_4$, allowing the antecedent to be weakened and the consequent to be strengthened.

$$\begin{array}{c}
 \text{PROPERENTAILS} \\
 \hline
 \varphi_3 \sqsupseteq \varphi_1 \quad \varphi_4 \sqsubseteq \varphi_2 \quad \varphi_3 \sqsubseteq \varphi_4 \\
 \hline
 \varphi_1 \sqsubseteq \varphi_2
 \end{array}
 \qquad
 \begin{array}{l}
 \text{Instance Proper_entails :} \\
 \text{Proper (flip entails ==>} \\
 \text{entails ==> impl) entails.}
 \end{array}$$

For another example, suppose the antecedent is $C[\varphi]$ for some context C . To rewrite it to $C[\varphi']$, we would need a `Proper` instance for every enclosing staged logic constructor in C , demonstrating that all of them preserve the entailment relation, or that they respect the context they are rewriting under. Examples of these lemmas are given in Fig. 6.3.

$$\begin{array}{c}
 \text{PROPERSATISFIES} \\
 \hline
 \varphi_1 \sqsubseteq \varphi_2 \quad \mathcal{E}, h_1, h_2, v \models \varphi_1 \\
 \hline
 \mathcal{E}, h_1, h_2, v \models \varphi_2
 \end{array}
 \qquad
 \begin{array}{c}
 \text{PROPERSEQ} \\
 \hline
 \varphi_1 \sqsubseteq \varphi_2 \quad \varphi_3 \sqsubseteq \varphi_4 \\
 \hline
 \varphi_1; \varphi_3 \sqsubseteq \varphi_2; \varphi_4
 \end{array}$$

$$\begin{array}{c}
 \text{PROPERREQ} \\
 \hline
 \varphi_1 \sqsubseteq \varphi_2 \\
 \hline
 \text{req } H \varphi_1 \sqsubseteq \text{req } H \varphi_2
 \end{array}
 \qquad
 \begin{array}{c}
 \text{RRESET} \\
 \hline
 \varphi_1 \sqsubseteq \varphi_2 \\
 \hline
 \langle \varphi_1 \rangle \sqsubseteq \langle \varphi_2 \rangle
 \end{array}$$

Fig. 6.3. Select rewriting lemmas

Depending on C , Rocq’s rewriting machinery uses typeclasses to piece together a proof using these lemmas to descend into it, justifying a rewrite all the way down.

6.1.5 Simplification and reduction

With the ability to rewrite inside arbitrary contexts (Section 6.1.4), we are able to state the reduction rules of Section 4.3 simply as entailment lemmas, and rely on rewriting to justify them.

Theorem 3.3.1 (Soundness of Reduction Rules).

$$\varphi_1 \leadsto \varphi_2 \text{ iff } \mathcal{E}, h_1, h_2, R \models \varphi_1 \Rightarrow \mathcal{E}, h_1, h_2, R \models \varphi_2.$$

Proof. The soundness of reduction follows directly from the soundness of entailment. □

6.1.6 Limitations of the model

While the model formalised so far in this chapter is sufficient to prove properties of many programs discussed in this dissertation, it does not work for all of them. The main issue is that the model contains *syntax* – specification environments $\mathcal{E}$ are maps containing values of type `flow` – which must be subject to a further operational interpretation. This prevents some statements from being proved using vanilla entailment ([Section 2.3](#)). For example, consider [Lemma 6.1](#).

Lemma 6.1 (Associativity of seq. – untrue!). $(\varphi_1; \varphi_2); \varphi_3 \Leftrightarrow \varphi_1; (\varphi_2; \varphi_3)$.

To see intuitively why this lemma cannot be proved, suppose that φ_1 contained a shift. According to the semantics, we would get shift outcomes on both sides, with continuations $(\Box; \varphi_2); \varphi_3$ and $\Box; (\varphi_2; \varphi_3)$. The logical next step would be reassociate the continuations, perhaps inductively. Unfortunately, the definition of entailment ([Definition 2.1](#)) requires the outcomes to be *syntactically equal*, and does not allow for this.

Intuitively, however, [Lemma 6.1](#) should certainly be true. We demonstrate this formally by strengthening the definition of entailment to allow for this inductive reassociation.

Definition 6.1 (Generalised entailment).

$$\begin{array}{c}
\text{GENTAILSSHIFT} \\
\forall \mathcal{E}_1 \mathcal{E}_2 h_1 h_2 \varphi_b \varphi_k. \\
\text{GENTAILSBASE} \\
\forall \mathcal{E}_1 \mathcal{E}_2 h_1 h_2 v. \quad \mathcal{E}_1, h_1 \models \mathcal{E}_2, h_2, \varphi_1 \Rightarrow \\
\mathcal{E}_1, h_1 \models \mathcal{E}_2, h_2, v \varphi_1 \Rightarrow \quad \exists \varphi'_b \varphi'_k. \mathcal{E}_1, h_1 \models \mathcal{E}_2, h_2, \varphi_2 \wedge \\
\mathcal{E}_1, h_1 \models \mathcal{E}_2, h_2, v \varphi_2 \quad (\varphi_b \subseteq_n \varphi'_b) \wedge (\forall v. \varphi_k[r/v] \subseteq_n \varphi'_k[r/v]) \\
\hline
\varphi_1 \subseteq_0 \varphi_2 \quad \varphi_1 \subseteq_{(n+1)} \varphi_2
\end{array}$$

A statement of generalised entailment $\varphi_1 \subseteq_n \varphi_2$ is indexed by the height of its derivations. At index 0, there are no shifts, and this is simply standard entailment for values. At index $n + 1$, there is at least one more shift, and we inductively require the resulting continuations to be related by generalised entailment at index n . Intuitively, an index of n allows us to talk about the behaviour of continuations *after n future shifts*. With this, associativity can be proved as we would expect, with $\varphi_1 \equiv_n \varphi_2 \triangleq \varphi_1 \subseteq_n \varphi_2 \wedge \varphi_2 \subseteq_n \varphi_1$.

Lemma 6.2 (Generalised associativity). $(\varphi_1; \varphi_2); \varphi_3 \equiv_n \varphi_1; (\varphi_2; \varphi_3)$

Proof. By induction on n . □

Unfortunately, when used as a sequent, the index exposed in generalised entailment is too constraining.

Another way to sidestep the issue is to ignore the cases where φ_1 and φ_2 have shifts.

Lemma 6.3 (Associativity of seq. – shift-free).

$$\text{If } \varphi_1^\# \text{ and } \varphi_2^\#, \text{ then } (\varphi_1; \varphi_2); \varphi_3 \Leftrightarrow \varphi_1; (\varphi_2; \varphi_3).$$

This can also be proved, but it is less than ideal, as we cannot reason completely about programs containing shifts.

Despite the shortcomings of the current model, it does validate the soundness of the framework, at least sufficiently to construct an automated tool.

A more expressive model is described in [Section 6.2](#).

6.1.7 Evaluation

Table 6.1
Case studies verified in Rocq

Program	Features	Foundational	LoC
<i>foldr</i> (Section 2.1)	HO, ind., imp.	✓	512
Landin’s knot (Section 5.1)	HO, ind., imp.	✓	436
Multi [186]	Multi-shot	✗	50
Times [21]	Ind.	✗	201
Toss [186]	Ind., imp.	✗	342
Total			593

The mechanisation of the metatheory of staged logic, as well as basic automation, spans a development of 6392 LoC in Rocq. It is built on top of a textbook mechanisation of separation logic [56].

As mentioned in Section 6.1.6, the model proposed is not sufficiently expressive to be able to verify some case studies without additional axioms – which are verified with respect to a different, incompatible notion of entailment (Definition 6.1). Nevertheless, to evaluate the feasibility of the theory, I attempted a number of case studies (Table 6.1) to get a sense of the effort and issues involved. Each example is shown with a list of its interesting features (use of higher-order functions, induction, imperative state, or multi-shot continuations), followed by the LoC (comprising all of program, specification, and lemma definitions, and proof script).

Included with the mechanisation is a library of tactics which automate applying the rules of Section 2.3, but with fewer heuristics, optimising instead for predictability in interactive use. This significantly reduces the length of proof scripts.

6.2 Towards a model using contextual refinement

In Section 6.1, I described a mechanisation with a simple form of behavioural refinement as the notion of entailment. It is complete enough to be able to

handle a number of case studies interactively, with tactics for working at the level of the logic.

However, as shown in [Section 6.1.6](#), the underlying model is not sufficiently expressive to be able to give a full treatment of the logic, leading to odd limitations, like being unable to rewrite on the right side of a shift. It is also an encoding of the logic, and does not model every feature in it exactly, e.g. using function pointers instead of general lambda expressions.

In this section, I sketch an alternative interpretation of the logic, outlining the path towards a complete mechanisation.

6.2.1 A more expressive notion of refinement

First, a solution to the problem mentioned in [Section 6.1.6](#). It arose because of the use of shift results, which contain syntax representing a shift body or continuation. The simple definition of entailment described in [Section 6.1.2](#) relies on the semantics ([Fig. 6.2](#)), and because the latter has nothing more to say about shift results other than that they are values, we ended up with semantically equal objects that were syntactically different, without a notion of refinement expressive enough to relate them.

It turns out that a version of this problem also occurs in the plain lambda calculus with lambda abstraction values. The missing piece is thus the notion of *contextual refinement*, which abstracts away syntactic detail by enclosing formulae in an arbitrary context, which, intuitively, captures all possible ways we can continue interpretation on the shift result/lambda abstraction. Any such suspended values which cannot be distinguished by arbitrary further use will be contained in the relation.

Contextual refinement is difficult to prove directly [\[27\]](#). An alternative is to use a binary *logical relation*, which provides a definition that is sound and complete with respect to contextual refinement, yet is easier to work with. Logical relations are typically defined inductively over the structure

$$\begin{aligned}
\mathcal{L} &\triangleq \{ (e_1, e_2) \mid (\forall v. e_1 = v \Rightarrow e_2 \Downarrow) \wedge (\forall e. e_1 \longrightarrow e \Rightarrow (e, e_2) \in \triangleright \mathcal{L}) \} \\
\mathcal{O} &\triangleq \{ (e_1, e_2) \mid (e_1, e_2) \in \mathcal{L} \wedge (e_2, e_1) \in \mathcal{L} \} \\
\mathcal{V} &\triangleq \{ (v_1, v_2) \mid \forall v'_1 v'_2 \in \triangleright \mathcal{V}. (v_1 v'_1, v_2 v'_2) \in \mathcal{R} \} \\
\mathcal{K} &\triangleq \{ (K_1, K_2) \mid \forall v_1 v_2 \in \mathcal{V}, M_1 M_2 \in \mathcal{M}. (M_1[K_1[v_1]], M_2[K_2[v_2]]) \in \mathcal{O} \} \\
\mathcal{E} &\triangleq \{ (e_1, e_2) \mid \forall K_1 K_2 \in \mathcal{K}, M_1 M_2 \in \mathcal{M}. (M_1[K_1[e_1]], M_2[K_2[e_2]]) \in \mathcal{O} \} \\
\mathcal{R} &\triangleq \{ (e_1, e_2) \mid \forall K_1 K_2 \in \triangleright \mathcal{K}, M_1 M_2 \in \triangleright \mathcal{M}. (M_1[K_1[e_1]], M_2[K_2[e_2]]) \in \mathcal{O} \} \\
\mathcal{M} &\triangleq \{ (M_1, M_2) \mid \forall v_1 v_2 \in \triangleright \mathcal{V}. (M_1[v_1], M_2[v_2]) \in \mathcal{O} \}
\end{aligned}$$

Fig. 6.4. Logical relation for a lambda calculus with shift/reset

of types, and are often *step-indexed* to cope with circularities that cannot be resolved by the structure of types alone. I use a step-indexed, but *untyped* treatment [103, 27], in the interest of both generality (types were not hitherto mentioned in this work) and compactness (a typed treatment would have to handle things like answer-type modification). The technique of *biorthogonality* [161], which is to define logical relations for contexts as well as expressions/values in a mutually recursive manner, is also used.

The resulting binary logical relation is shown in Fig. 6.4. It is defined using contextual equivalence (contextual refinement in both directions), and for a lambda calculus with shift/reset¹². It assumes the underlying logic is step-indexed, using the later modality to make definitions well-founded.

The different relations are named mnemonically. The relation $\mathcal{L}$ is one side of the observational equivalence relation $\mathcal{O}$ for closed programs. $\longrightarrow$ is a *meta-contextual step*: a small-step operational semantics for shift/reset with context and meta-context [67]³. $\mathcal{V}$ is for values; since we are dealing with a lambda calculus, the only values are lambda abstractions, which are in the relation if applying them to values in the relation after one step produces results in the relation. $\mathcal{K}$ is for contexts, which are related if plugged with and into related values and meta-contexts, respectively, if the result is observationally equivalent. $\mathcal{E}$ is for expressions, which are related if plugged

¹The missing parts of staged logic are addressed in Section 6.2.3.

²As staged logic is now directly encoded as a lambda calculus, there is no longer a need for specification environments or function pointers.

³Notably, the bubble-up semantics and shift results are no longer needed.

into related contexts and meta-contexts and observationally equivalent. $\mathcal{R}$ is the same as $\mathcal{E}$ with the exception of the later modalities; it is only used in $\mathcal{V}$ and is there for technical reasons [27]. An increment over Balik et al. [27], where continuations are undelimited, is the addition of meta-contexts to the definitions and to the $\mathcal{M}$ relation.

6.2.2 Encoding in Rocq

The addition of contexts and contextual semantics necessitates a deep embedding of staged logic, so the syntax is defined as an inductive datatype. As staged logic is a lambda calculus at heart, this is in keeping with observations made in the literature that it is more appropriate to deeply-embed “dynamic” objects (such as programs, involving state transitions) and shallowly-embed “static” objects (such as assertions, which describe sets of states) [203].

Given the deep embedding, binding is handled via the functorial approach [166], using well-scoped de Bruijn indices.

The next salient feature of the encoding is the use of `IxFree` [165] for a step-indexed base logic with a later modality [79], which hides the explicit index arithmetic. `Iris` [110] could equally be used as a base logic; indeed, I used it for an early prototype, but it was generally easier to proceed with `IxFree` due to its simpler core and focus.

Finally, contextual equivalence is a congruence (and contextual refinement a *precongruence*), so the Proper instances required (Section 6.1.4) are easy to define using the completeness theorem.

6.2.3 Remaining work

Encoding all of staged logic, as well as tackling case studies, remains work in progress. Many of the ideas from Section 6.1 can be reused, but must be adapted to the logical relations setting.

- Specifications remain. **req** can be encoded in a similar way, **ens** induces (may-)nondeterminism, which must be handled by may-observational equivalence [43].
- Heaps also remain. They must be step-indexed, like the CMRAs [110] of Iris, as they are able to continue functions.
- Since both the languages of programs and specifications are deeply embedded, triples can be defined as cross-language logical relations.
- Effect handlers also admit a logical relations account [209].

6.3 Summary

Table 6.2
Summary of both mechanisations

		Simple	Contextual refinement
Syntax	Separation logic	Shallow	Deep
	Staged logic	Deep + shallow	
Semantics	Programs	Big-step	Contextual small-step
	Delimited continuations	Bubble-up	List of contexts
Entailment		Refinement	Contextual refinement
Sound		✓	✓
Complete		✗	✓
Higher-order functions		✓	✓
req/ens		✓	✗
Heaps		✓	✗
shift/reset		✓	✓
Tactics		✓	✗
LoC		6392	5931+
Case studies		5	-

Table 6.2 gives a summary of both mechanisations. The main differences between them are 1. the definition of entailment, and 2. how the syntax of staged logic is represented.

The simple one uses a shallow embedding for the underlying (separation) logic, which is conventional [203] and allows the reuse of existing proof

automation [56]. Its syntax is deeply embedded, but with anything involving values shallowly embedded (using HOAS), as necessitated by the shallow embedding of the underlying logic. Limitations of shallow embedding, such as the inability to perform induction over syntax or have function values due to negative occurrences, are worked around or suffered.

The one based on contextual refinement uses a deep embedding for everything. This is more general but significantly more complex, necessitating things like an intrinsically-typed de Bruijn representation, as well as definitions for evaluation and general program contexts.

The simple mechanisation is a relatively practical one, with a small number of case studies foundationally verified, using a number of specialised tactics. It also supports the whole language of staged logic. However, there are entailments which cannot be proved in it – this is what is meant by lack of completeness.

To overcome this problem, the second one is based on contextual refinement, which is defined using a step-indexed, biorthogonal logical relation. It is unfinished at the time of writing and does not cover the entire language of staged logic; there are also no tactics or case studies using it yet. However, its size is already comparable to the simple one, reflecting the significant increase in complexity.

RELATED WORK

7.1 Program logics

Deductive verification is typically carried out with the help of *program logics*, formal systems which give programs an axiomatic semantics. In this section, we give a brief history of the development of program logics, leading up to the modern developments that we build on.

7.1.1 Hoare logics

The field of program logics could be said to have begun with *Hoare logic* [100], which introduced the *Hoare triple* $\{P\}e\{Q\}$ as a way of characterising the behaviours of programs. Dijkstra [78] developed the idea of automating the verification of Hoare triples using *weakest preconditions*, which half a century later forms the basis of many mature tools, such as Dafny [137] and Why3 [83]. As these tools matured, they came to be structured as *verification platforms*, supporting *intermediate verification languages* [28, 149]: shared infrastructure to which the verification of programs in different languages could be reduced. Cameleer [158, 183, 181] is one such extension of the Why3 platform, enabling automated verification of OCaml programs. It has a backend-agnostic specification language, GOSPEL [184], which can also be interpreted via Ortac [104] for more lightweight runtime checking.

While tools based on standard Hoare logic are highly mature, they have limited support for more dynamic language features, such as heap manipulation, higher-order programs, and effects; there is not yet consensus on how best to support them. Staged logic is one answer, bridging Hoare logic with its sister methodology of refinement. Seen as an extension of Hoare logic, staged logic uses triples in much the same way, and can be seen as enriching

them with an underlying logic that is *behavioural*, satisfied by traces rather than single states.

Separation logic Reynolds [173] and O’Hearn [154] proposed *separation logic*, a substructural logic of resources, as a way to solve the frame problem of Hoare logic and enable reasoning about heap-manipulating programs. It became the basis of a new generation of automated verification tools, such as VeriFast [109, 108], HIP/SLEEK [58], and Viper [149], which automate program proofs using *symbolic execution* [37]. Separation logic is also the foundation of the Infer tool, which pioneered the use of *biabduction* [50] for *begin-anywhere* static analysis.

Staged logic adopts separation logic as a means of describing program states, while handling the behavioural aspects. It also relies on biabduction to simplify formulae, in a manner similar to symbolic execution.

Mechanisation While there are many automated program verifiers in continued and successful use, a major alternative direction has been to encode program/separation logics in proof assistants, to inherit both their interactive facilities as well as their foundational soundness guarantees.

A key design choice when mechanising a logic is whether to embed each aspect of it *shallowly* or *deeply* [203]. A shallow embedding allows for maximal reuse of the proof assistant’s features, but limits metatheoretical reasoning, as well as what can be encoded, if there is no precisely-matching feature in the metalogic – an example is in the use of higher-order abstract syntax (HOAS), where binding is represented by metalogic functions, and negative occurrences limit what can be expressed and require nontrivial workarounds. Dually, a deep embedding results in no semantic mismatch and full control over every aspect of the object logic, with the tradeoff that every feature of the logic has to be reimplemented – to continue the example of binding, a first-order encoding requires necessitates representations such as de Bruijn

indices, as well as theorems to say that terms are closed.

Modern separation logics mostly shallowly embed the language of propositions and differ on whether to deeply embed the program. These two correspond roughly to whether the logic is seen as something “extrinsic” [125] to the program, and related to it only by a separate relation like a Hoare triple, or “intrinsically” part of it, e.g. encoded in its type.

CFML [54, 55] is based on the notion of *characteristic formulae*, relations between pre- and postconditions that can be computed from the (deeply-embedded) syntax of programs, such that the syntax no longer features in verification conditions. It is simple and elegant, and is used for teaching in the *Separation Logic Foundations* [56] textbook. It is common in Iris [110] to deeply embed programs and their contexts [90, 198]. The logic of Iris is shallowly embedded, but atop a sophisticated step-indexed model of higher-order logic, in order to support impredicative quantification and higher-order state. The Verified Software Toolchain (VST) [17] makes similar choices with respect to programs, but implements a step-indexed model using *indirection theory* [102], a general method of implementing stratified models. These logics mechanise the process of symbolic execution using magic wands, weakest preconditions, and the ramified frame rule [101].

Hoare Type Theory (HTT) [150] pioneered the approach of using dependent types to embed specifications into the types of (shallowly-embedded) monadic programs. This approach was followed by Swamy et al. [193], who proposed *Dijkstra monads* and the use of weakest preconditions instead to automate the computation of specifications; this was later generalised to work for any *specification monad* [142, 205] and more combinations of effects. Pulse [89, 194, 81] is a separation logic embedded in $F^\star$, based on these ideas, which also marries them with indirection theory for reasoning about concurrency.

In the first mechanisation (Section 6.1), staged logic takes a variation of

the former position: staged formulae are deeply embedded but use HOAS and a shallow embedding of the underlying (separation) logic. The result is an incomplete but more practical mechanisation, suitable for interactive use. In the second (Section 6.2), because of the dynamic nature of the logic and its ability to manipulate higher-order values, it is completely deeply embedded, as a program would be. This mechanisation is more faithful but is significantly more complex.

7.1.2 Refinement

Temporal logics and Hoare logics Refinement is a foundational concept in program verification, serving as a means of relating descriptions of systems. Perhaps the most well-known and widely-used temporal logic, TLA^+ [127], uses it to relate state machines to protocols or temporal properties, which are all expressed in the same logic. Refinement is also used in Hoare logics, typically to justify that an effectful program implements some pure function, by including the latter directly in the postcondition; we rely on this sort of use in our work.

Contextual refinement A related property is *contextual refinement*, a relation between *open* programs which quantifies over their contexts:

$$\forall C. C[e] \sqsubseteq C[e']$$

This intuitively means that an arbitrary client C cannot distinguish e from e' .

Proving contextual refinement directly by induction over contexts is known to be difficult [27]. It is instead usually proved by via the technique of logical relations [27]. The idea is to define relations which are sound and complete with respect to contextual refinement, but for which it is easier to prove membership.

Logical relations were classically defined inductively over the structure

of types. However, semantically-cyclic language features such as recursive types [19] and general references [14] necessitated induction over some other index; this led to the technique of *step-indexed* logical relations. Step-indexing can be treated logically [79], hidden behind a *later* modality. Modern logics such as IxFree [165] and Iris [110] take this approach, and provide facilities for defining predicates that are well-founded based on the index. These can then be used to construct logics for reasoning about contextual refinement.

One approach is to reason about contextual refinements directly. Earlier approaches, such as Timany and Birkedal [198], worked with the logical relation directly, which was defined in terms of weakest preconditions; they introduced the idea of context-local triples to simplify reasoning when contexts were not manipulated, but ultimately unfold the logical relation and work with its definition. ReLoC Reloaded [90] is a newer approach which uses a *typed* relation of the form $e_1 \lesssim e_2 : \tau$, and structural rules to manipulate this judgment, remaining at the level of refinements.

An alternative is to use a relational program logic to reason compositionally about it. Simuliris [91] takes this approach, with a simulation relation (similar to the judgment of Relational Hoare Logic [36]) $\{P\} e_1 \preceq e_2 \{r_1 r_2. Q\}$.

The approach I take in the latest mechanisation (Section 6.2) is similar to the one used in Timany and Birkedal [198], working with the logical relation directly – a simple, baseline approach – with a view towards eventually adapting the approach used in ReLoC Reloaded (to the untyped setting, and to programs with control operators), to realise the goal of reasoning at the level of the entailment relation.

Refinement calculi Refinement calculi [24, 148, 25, 52] are another family of program logics, originating in the methodology of *stepwise refinement*, a

deductive program synthesis method for deriving programs from specifications. The distinguishing features of refinement calculi are that they embed *specification statements* in programs, allowing both to be expressed in the same abstract programming language, and a binary *refinement* relation which orders abstract programs, serving as a means of giving them specifications. This allows working and reasoning entirely at the level of programs.

Staged logic bears superficial resemblance to refinement calculi, given that it mixes specifications and program elements in a single language. It is, however, a logic with constructs for *representing* programs, rather than a programming language, and is designed for verification rather than synthesis.

Modern routes to mechanisation for refinement calculi commonly involve shallow, monadic embeddings of effectful programs. For example, Boulmé [45] uses a state monad equipped with a least fixed point operator to verify higher-order imperative programs. Another line of work [15, 23, 195] from Swierstra and Baanen [195] uses free monads to uniformly implement a variety of effects. None of these tackle delimited continuations, which is not an algebraic effect [106, 162] and cannot be expressed as a free monad. It is unclear how to shallowly embed effect handlers or delimited continuations alongside e.g. a weakest precondition semantics.

7.1.3 Monadic encodings of effects

Monads are a useful semantic device for shallowly embedding effects, and another line of work using them for reasoning about programs is *monadic equational reasoning* [93, 13]. Staged logic similarly advocates for the idea of treating effects equationally, a simple and effective approach. However, staged logic also deals with specifications, and, consequently, inequations (refinement). The semantics of specifications can be shallowly embedded using a weakest precondition semantics, as in the encodings of refinement calculi mentioned in [Refinement calculi](#). However, it is unclear how to extend

this to non-algebraic effects. Staged logic thus uses a deep embedding for the semantics of specifications.

Another program verification approach leveraging monads is the ordered *specification monads* of the Dijkstra monad line of work [142, 205]. This approach differs from staged logic in that specifications are given intrinsically, rather than extrinsically – the monad is only used to encode the effectful program, and the specifications are given in a standard weakest precondition style.

7.1.4 Relational verification

Much of the program verification literature is dedicated to *trace properties*, predicates on program executions of programs. *Relational properties* or *hyperproperties* generalise this to relations between executions.

In the relational verification literature, *refinement* refers to a relational $\forall\exists$ property which copes with a nondeterminism reference program [35, Definition 2]. This is not the sense in which it is used in this dissertation, which uses the classic sense of behavioural inclusion.

Relational verification is mainly carried out in two ways [30].

The first is to use a *relational program logic* [36], a bespoke formal system for proving specific kinds of relational properties. Some examples are RHLE [76] ($\forall\exists$ properties), CHL [188] (*k-safety* or $\forall^*$ properties), and Hyper Hoare Logic [69], which allows arbitrary quantifier alternations.

The second way is to encode relational verification problems into standard Hoare logic, to allow reuse of existing tools. This requires different encodings depending on the type of property. *Self-composition*, or more generally, *product programs* [31] for $\forall\forall$ properties. *Asymmetric product programs* [32] extend this to $\forall\exists$ properties. This can also be done at the level of logics [207].

Regardless of the verification method, the issue of *alignment* is essential.

Intuitively, it is the act of bringing parts of the programs being related into correspondence, to make verification easier. For example, relational program logics often contain “lockstep” rules which only apply to two structurally-similar programs. Alignment appears in different forms depending on the verification approach being considered. In the self-composition approach, it may require duplicating loop bodies, unrolling loops, and so on, while in relational program logics, relational invariants allow loop states to, and there may be rules for executing one program, given that it does not change the state relevant to the relation. For some formulations, it is possible to automatically search for alignments [200, 77, 16].

This dissertation focuses mostly on trace properties. A case study on encoding program equivalence (a $\forall\forall$ property) using staged logic is given in [Section 5.3](#). Other kinds of $\forall\forall$ properties, as well as other relational properties, are left to the future work.

7.2 Higher-order imperative programs

7.2.1 Specifications

Before we can verify higher-order programs, we must first specify what it means for them to be correct. There are two broad families of approaches for doing so, with contemporary approaches fusing them to combine their benefits.

Nested Hoare triples and invariants The use of a Hoare triple as a proposition to give a function parameter a specification is a natural and “obvious” [174] way to treat higher-order programs. Such a Hoare triple is typically coupled with an *invariant*, a predicate of the metalogic, to describe the effect of a function; an introduction to this pattern was given in [Section 2.1](#). For this reason it requires higher-order quantification [132, Section 2.1], and

so works well in program logics designed for interactive verification, such as iHTT [192], Iris [110, 42], and CFML [54].

It can also be adapted to automated verification, either directly [57] or by first-order axiomatisation: using axioms to associate specifications and invariants with otherwise uninterpreted higher-order functions. The latter approach was pioneered by Régis-Gianas and Pottier [170], extended to the imperative setting by Kanig and Filliâtre [114], and remains commonly used in other verifiers [181] in the Why3 [83] lineage, as well as newer tools like Verus [129]. *Refinement types*, such as in LiquidHaskell [201], can also be seen as an intrinsic encoding of nested triples.

Greybox specifications Büchi and Weck [49] proposed the idea of *greybox specifications*, observing that refinement was a good way to specify higher-order programs with side effects, due to the difficulty of abstracting them in general; Boulmé [45] also makes this observation in the context of OCaml programs. Shaner et al. [177] adapted the greybox approach to JML in the form of *model programs*, which extends [132] to aspect-oriented and context-oriented language features.

Kanig and Filliâtre [114] noted that “one is often confronted with the problem of specifying the behaviour of a program where the most natural description of the behaviour is the program itself!” In Why3 [83] and Dafny [137], pure functions require no further specification, as they are already mathematical objects, and so can be given directly to SMT solvers, even in higher-order contexts. This technique can be seen as an instance of (and, retrospectively, motivation for) the greybox approach as well.

Prusti [206] primarily uses a nested triple approach (*specification entailments*), but also proposes *call descriptions*, which specify the *uses* of closures within higher-order functions, something which would be otherwise opaque to clients, in order to inform clients about their effects. Pen-

ninckx et al. [157] uses *abstract* nested triples to specify I/O in a similar way.

Coma [156] is a recent addition to the Why3 platform, with facilities for enforcing abstraction barriers, allowing the user to vary how much of a program is specified, and which parts are subject to the greybox approach.

The approach I describe in this dissertation also fuses both paradigms, combining the symbolic execution of separation logic triples with the flexibility of refinement as the means of specification.

7.2.2 Syntactic methods

Given the right specification, (syntactic) verification is often natural. With a nested triple, one must instantiate any higher-order quantifiers (which typically requires human input) before using the standard function application rule. Model programs are equally simple to verify and instantiate for use, though symbolic execution is not considered there, something we remedy.

In this section we survey additional syntactic techniques useful for verification.

Defunctionalization Defunctionalization [172] is a classic technique for compiling higher-order programs to first-order ones, and it can similarly be used to reduce the verification of a higher-order program to a first-order setting. It was first proposed by Soares [182] for Cameleer, using axiomatised specifications parameterised over states to handle effectful programs, as Régis-Gianas and Pottier [171] do. Creusot [75] models closures as stateful objects, following the *closure conversion* Rust uses, a closely related concept to defunctionalization.

Type system guarantees An expressive type system can significantly simplify how higher-order state is specified and managed. For example, in Rust, closures have exclusive ownership over mutable locations that they capture;

Prusti [206] exploits the fact that they can maintain *history invariants* to provide stronger guarantees about captured state. Creusot [75] uses *prophecies*, similar in spirit to ghost variables allowing one to talk about the values of mutable locations at the end of borrows, to specify mutable state; it then treats closures as if they have constant prophecies relating to their captured state.

7.2.3 Semantic methods

In the earlier section on [Mechanisation](#), we briefly surveyed the techniques generally used to implement higher-order program logics in interactive proof assistants. Here we focus more specifically on ways in which they support the verification of higher-order imperative programs and the issues encountered therein.

As today’s mature proof assistants are based on type theories or higher-order logic, they do not natively support closures or imperative updates in their metalanguages. Embedding a program logic thus begins with an embedding of programs, which may be *deep* or *shallow*. Deep embeddings are generally handled in an *extrinsic* way, with (nested) Hoare triple propositions associating specifications with programs, while shallow embeddings are handled in an *intrinsic* manner, with their syntax represented as a value of a monadic type indexed with a specification.

Nested triples Hoare triples are the usual way of specifying higher-order programs in proof assistants. Due to higher-order quantification and the impredicativity of, e.g., Rocq’s Prop, they are naturally usable in a nested manner, e.g., in CFML [54, 56].

Dijkstra monads Dijkstra monads [193, 142] were originally introduced to automate the computation of verification conditions, but their use case was

to verify higher-order, stateful programs. The intrinsic style of verification with dependently-typed monads lends itself very naturally to higher-order programs, with state naturally handled by computing weakest preconditions (which transform predicates over states), and the effectful monadic layer taking care of the effects.

Impredicativity A problem that plagues both extrinsic and intrinsic approaches alike is that of impredicativity. It shows up in higher-order programs in semantically-cyclic features such as *higher-order store*. This necessitates constructing models stratified by some notion of step, typically a small step in the operational semantics, and using a *later modality* to guard recursive occurrences of predicates, ensuring they take place at a smaller step to keep definitions well-founded. Iris [110] is perhaps the most influential and well-known example of this approach. *Indirection theory* [102] is a general approach for building stratified models and viewing them through the lens of approximation; VST [17] and PulseCore [81] are recent examples building on it.

Syntactic indirection [18, 59, Chapter 35] is a means of working around the issue of impredicativity without a later modality, typically using a syntactic approach at the cost of some expressiveness. VeriFast [108] is a recent example of program logic with this approach. XCAP [152] was an early example, doing this to verify low-level programs with function pointers. It works around the problem using both syntactic assertions, and a modular approach where programs are verified under local assumptions and deferring the check for whether assumptions are all globally consistent. This can also be seen as deeply-embedding part of the assertion language in order to defer what would have otherwise been a cyclic interpretation, breaking the cycle. Chapter 6 illustrates how we use a similar approach to work around the issues with impredicativity.

7.3 Effect handlers

7.3.1 In practice

Effect handlers originated in experimental programming languages such as Eff [33, 34], Effekt [47, 46], and Koka [135, 133], but have been steadily finding their way into mainstream languages; natively in OCaml 5 [179] and WebAssembly [159], and in the form of libraries in Scala 3 [199] and Haskell [167].

Effect handlers have many applications [7], including concurrency [179], control inversion [6], automatic differentiation [73], reactivity [4], model-checking [2], and probabilistic programming [151, 3].

7.3.2 Reasoning

Protocols The dominant paradigm for reasoning about effects today is the protocol-based approach, pioneered by Hazel [72, 71, 73] and Maze [71], and implemented in Cameleer [183, 181]. An extensive introduction to it was given in Section 3.1.1. In summary, the user provides two pieces of specification: a *protocol*, a contract for an effect that a client can rely on and that a handler must implement; and a *handler invariant*, a means of reasoning about the recursive nature of deep handlers. This allows modular verification in Hoare logic, but at the cost of requiring a global specification that commits to an interpretation of effects. Hazel and Maze define a protocol as a predicate transformer in Iris’ logic, while Cameleer builds it in primitively, encoding an effect as a WhyML exception and using defunctionalization and axiomatisation to represent continuations and their specifications. Only Maze supports multishot continuations, with restrictions that resources cannot be framed around them.

Refinement An emerging approach for reasoning about effect handlers is to internalise them and talk directly about them using refinement. This is the basis of the approach introduced in [Section 3.1.2](#) in this dissertation. The idea was pioneered by Song et al. [\[185\]](#), focusing on temporal verification for pure functional programs. It was extended with heap-manipulation by Foo et al. [\[87\]](#) and to effects by Song et al. [\[187\]](#). The key difference compared to the protocol approach are that the interpretations of effects are deferred until handlers are known and yet admit a modular, *greybox* specification; consequently, no novel specification mechanisms are required. I believe it is possible to marry both approaches, as the interpreted and uninterpreted approaches are complimentary and are each useful for different applications.

Type-and-effect systems Traditionally, effects have been specified via type-and-effect systems, which refine type systems with annotations, for example in the form of a set of effect labels [\[134\]](#). Recent approaches are based on refinement types [\[115\]](#) and modal types [\[210, 196\]](#). These approaches typically aim to verify less expressive properties, such as the absence of unhandled effects. Functional correctness is supported to an extent, with limitations on language features; heap-manipulation is often not handled, and advanced language features such as modules continue to be open [\[179\]](#).

Monads While fixing the composition issues with monads and transformers was a key motivation for effect handlers, monads continue to be used to encode effects in proof assistants for reasoning. There are two distinct approaches: directly working with a monadic program and proving properties via equational reasoning [\[93, 13\]](#) or refinement [\[23, 195, 15, 45\]](#); or having a computation monad (which specifies the effects) indexed by a specification monad [\[193, 141\]](#), to build a program logic for the former. In both cases, it is not common to internalise handlers in the object language, unlike what we do; effect handlers are defined as folds over monadic types at the meta

level instead.

7.4 Delimited continuations

7.4.1 In practice

Delimited continuation operators were classically implemented in research languages, most prominently Racket, which supports the widest variety [8], but like effect handlers, they have recently begun to enter the mainstream, most recently in Scala 3 [147] (via a program transformation), as (untyped) primitives in GHC [116], where they have replaced monads as the basis for modern effect systems [167], and as libraries in OCaml [118, 180].

It is common to implement either **control**₀ (as in GHC), or a set of four lower-level operations *newPrompt*, *pushPrompt*, *takeSubCont*, and *pushSubCont* for manipulating the control stack [180, 118, 120, 80, 96] with named prompts for generality.

Many of the use cases of effects apply equally to delimited continuations. Additional ones include partial evaluation [130, 65], web interactions [94], operating systems [122], automatic differentiation [202], probabilistic programming [60], and linguistics [176].

7.4.2 Reasoning

Equational reasoning and refinement Reasoning about delimited continuations has classically been carried out either by direct appeal to their semantics in CPS [39, Section 3], or by the use of equations [111]. Encodings of this approach in proof assistants are possible using the *generalised monad* [22], which also takes an input parameter. It has been used as a means of implementing a continuation monad admitting ATM [117, 189] and can support proofs by equational reasoning [99], at the cost of having to annotate the

types of subterms, and find encodings for other kinds of effects, such as state. The approach we take with refinement sits in this category, generalising the equational approach to allow the use of specifications, and using a separation logic for stateful behaviours.

Program logics HTT_{cc} [74] is a mechanised program logic for (algebraic) `callcc` and `throw`. They use nested triples and focus on reconciling state changes with continuations, using so *large-footprint assertions*, which do not admit a general frame rule and must quantify over heaps to indicate framing.

Timany and Birkedal [198] develop a relational program logic which proposes the idea of *context-local* and *non-context-local* (Section 7.4) triples: the former are used for programs free of control operators and allow modular reasoning, while the latter are for programs with control changes and allow global reasoning reflecting the operational semantics. They use contextual refinement to show that a cooperatively-concurrent program implemented using delimited control refines a specification program with primitive concurrency.

Effect handlers and (the 0 variants of) delimited continuation operators are known to be interderivable [88, 160]. de Vilhena [71] exploits this to adapt their protocol-based program logic for effect handlers to `callcc` and `throw`; more on this in [Protocols](#). The converse has been done by Cong and Asai [63], but for a type system.

Type systems Danvy and Filinski [66] gave a type system supporting the ATM of `shift` and `reset` soon after they were proposed. It turns the typical typing judgment $\Gamma \vdash e : \tau$ into a 5-place version $\Gamma, \alpha \vdash e : \tau, \beta$, where α and β track the (input and output) *answer types* of an expression e (i.e. the type of an enclosing `reset`), which may be modified by the execution of the e . Asai and Kameyama [20] generalise it with *answer type polymorphism*, and illustrate it lucidly in OchaCaml [143], a variant of OCaml which tracks the

answer types of functions using an arrow of the form $\tau_1/\alpha \rightarrow \tau_2/\beta$. Cong and Asai [62] extend this to a dependently typed system.

Sekiyama and Unno [175] propose *answer effect modification*, which generalises answer types with refinements, allowing their use for reasoning about delimited control operators. Kawamata et al. [115] adapts this idea to effect handlers. This approach is currently limited to pure programs.

Protocols The protocol approach can be adapted to handle (un)delimited control operators. Rules for *callcc* and *throw* can be found in de Vilhena [71, Section 6.3.2]. They are reproduced here to illustrate the key ideas.

$$\begin{array}{c}
\text{CALLCC} \\
\frac{\forall k. \text{isCont } k \ \Phi \multimap \text{ewp } e \ \langle CT \rangle \{\Phi\}}{\text{ewp } (\text{callcc } k \ e) \ \langle CT \rangle \{\Phi\}}
\end{array}
\qquad
\begin{array}{c}
\text{THROW} \\
\frac{\text{isCont } k \ \Phi \quad \Phi(w)}{\text{ewp } (\text{throw } k \ w) \ \langle CT \rangle \{\Phi\}}
\end{array}$$

To prove that a use of *callcc* satisfies a postcondition Φ , we must prove that the body satisfies Φ under the assumption that k is a continuation whose *input* satisfies Φ . This is because the input to *throw* k in e will also be the result of e , and also the result of *callcc*. *CT* is a protocol allowing the use of the *callcc* and *throw* effects.

The rule for *throw* simply requires that k is a continuation whose input w satisfies Φ .

The extension to shift/reset [11, pages 87-88] builds on these ideas.

$$\begin{array}{c}
\text{SHIFT} \\
\frac{\Box (\forall w. \Phi'(w) \multimap \text{wp } (k \ w) \ \langle _ \rangle \{\Phi\}) \multimap \text{wp } e \ \langle \text{Some } \Phi \rangle \{\Phi'\}}{\text{wp } (\text{shift } k \ e) \ \langle \text{Some } \Phi \rangle \{\Phi'\}}
\end{array}
\qquad
\begin{array}{c}
\text{RESET} \\
\frac{\text{wp } e \ \langle \text{Some } \Phi \rangle \{\Phi\}}{\text{wp } \text{reset } e \ \langle _ \rangle \{\Phi\}}
\end{array}$$

Φ is the “answer postcondition”, which describes 1. the postcondition of the enclosing reset, or the input to the metacontinuation, and 2. the postcondition of k . Φ' is the postcondition of shift and the shift body, which also constrains the input to k . “Answer postcondition modification” is possible as

Φ and Φ' can be different. k has no effects and can be used many times.

The rule for *reset* handles all “effects” and supplies the “answer postcondition” Φ .

Comparing the approach once again with staged logic, the main and fundamental difficulty with using protocols is that one has to come up with the Φ and Φ' upfront, similar to how one has to state invariants upfront ([Section 3.1](#)).

Secondly, the extension to **control** not obvious. It is not just a matter of adding an operational semantics for it, as in staged logic; we would also need a protocol, which seems nontrivial to write. I am not aware of any program logics for **control**, so it is unclear what the protocol should be parameterised over.

CONCLUSION

In this dissertation, I have described *staged logic*, a comprehensive and novel approach to using refinement to reason about effectful higher-order programs. Staged logic underlies the implementation of a new automated verifier for OCaml programs, HEIFER, and is mechanised in Rocq. I have shown that it is feasible to automate, by verifying a number of challenging programs with it, as well as that it is sound, via the mechanisation.

8.1 Future work

There are several avenues for future work.

- [Section 6.2](#) details the current state of the mechanisation, and [Section 6.2.3](#) gives the next steps for completing it, with the goal of enabling interactive use in Rocq.
- A certification back-end for HEIFER, to allow it to generate certificates for the Rocq mechanisation, would usefully integrate the two tools, and also confirm the soundness of the former.
- Exploring the use of staged logic in new domains, such as relational verification and concurrency. This combination is particularly interesting due to the use of effect handlers to model concurrency, and the difficulty of verifying such libraries built in this way today.

BIBLIOGRAPHY

- [1] Add effect syntax. <https://github.com/ocaml/ocaml/pull/12309>.
- [2] DSCheck - tool for testing concurrent OCaml programs. <https://github.com/ocaml-multicore/dscheck>.
- [3] EffPPL. <https://github.com/Arnhav-Datar/EffPPL>.
- [4] Friendship ended with monads: Testing out algebraic effects in OCaml for animations. <https://gopiandcode.uk/logs/log-bye-bye-monads-algebraic-effects.html>.
- [5] Multicont: Continuations with multi-shot semantics in OCaml. <https://github.com/dhil/ocaml-multicont>.
- [6] The OCaml manual - language extensions - effect handlers. <https://ocaml.org/manual/5.3/effects.html>, .
- [7] OCaml multicore examples. <https://github.com/ocaml-multicore/effects-examples>, .
- [8] Continuations. <https://docs.racket-lang.org/reference/cont.html>.
- [9] Implementing simple generators with effect handlers. <https://discuss.ocaml.org/t/implementing-simple-generators-with-effect-handlers/16020/7>.
- [10] Modular specification and verification of closures in Rust (artefact). <https://zenodo.org/records/5482557>, 2021.
- [11] Proof of programs with effect handlers, 2022. URL <https://devilhena-paulo.github.io/files/phd-defense-slides.pdf>.

- [12] Documentation of closures. <https://github.com/viperproject/prusti-dev/issues/1431>, 2024.
- [13] Reynald Affeldt, Jacques Garrigue, and Takafumi Saikawa. A practical formalization of monadic equational reasoning in dependent-type theory. *J. Funct. Program.*, 35, 2025. doi: 10.1017/S0956796824000157. URL <https://doi.org/10.1017/s0956796824000157>.
- [14] Amal Jamil Ahmed. *Semantics of types for mutable state*. Princeton University, 2004.
- [15] João Alpuim and Wouter Swierstra. Embedding the refinement calculus in coq. *Sci. Comput. Program.*, 164:37–48, 2018. doi: 10.1016/J.SCICO.2017.04.003. URL <https://doi.org/10.1016/j.scico.2017.04.003>.
- [16] Timos Antonopoulos, Eric Koskinen, Ton Chanh Le, Ramana Nagasamudram, David A. Naumann, and Minh Ngo. An algebra of alignment for relational verification. *Proc. ACM Program. Lang.*, 7(POPL): 573–603, 2023. doi: 10.1145/3571213. URL <https://doi.org/10.1145/3571213>.
- [17] Andrew W. Appel. Verified Software Toolchain. In Alwyn Goodloe and Suzette Person, editors, *NASA Formal Methods - 4th International Symposium, NFM 2012, Norfolk, VA, USA, April 3-5, 2012. Proceedings*, volume 7226 of *Lecture Notes in Computer Science*, page 2. Springer, 2012. doi: 10.1007/978-3-642-28891-3_2. URL https://doi.org/10.1007/978-3-642-28891-3_2.
- [18] Andrew W. Appel. *Program Logics for Certified Compilers*. Cambridge University Press, 2014. ISBN 978-1-10-704801-0. URL <http://www.cambridge.org/de/academic/subjects/>

computer-science/programming-languages-and-applied-logic/
program-logics-certified-compilers?format=HB.

- [19] Andrew W. Appel and David A. McAllester. An indexed model of recursive types for foundational proof-carrying code. *ACM Trans. Program. Lang. Syst.*, 23(5):657–683, 2001. doi: 10.1145/504709.504712. URL <https://doi.org/10.1145/504709.504712>.
- [20] Kenichi Asai and Yuki Yoshi Kameyama. Polymorphic delimited continuations. In Zhong Shao, editor, *Programming Languages and Systems, 5th Asian Symposium, APLAS 2007, Singapore, November 29–December 1, 2007, Proceedings*, volume 4807 of *Lecture Notes in Computer Science*, pages 239–254. Springer, 2007. doi: 10.1007/978-3-540-76637-7_16. URL https://doi.org/10.1007/978-3-540-76637-7_16.
- [21] Kenichi Asai and Oleg Kiselyov. Introduction to programming with shift and reset. 2011. URL <https://api.semanticscholar.org/CorpusID:62358634>.
- [22] Robert Atkey. Parameterised notions of computation. *J. Funct. Program.*, 19(3-4):335–376, 2009. doi: 10.1017/S095679680900728X. URL <https://doi.org/10.1017/S095679680900728X>.
- [23] Tim Baanen. Algebraic effects, specification and refinement. Master’s thesis, 2019.
- [24] Ralph-Johan Back. *On the correctness of refinement steps in program development*. Department of Computer Science, University of Helsinki Helsinki, Finland, 1978.
- [25] Ralph-Johan Back. A calculus of refinements for program derivations. *Acta Informatica*, 25(6):593–624, 1988. doi: 10.1007/BF00291051. URL <https://doi.org/10.1007/BF00291051>.

- [26] Ralph-Johan Back and Joakim von Wright. *Refinement Calculus - A Systematic Introduction*. Graduate Texts in Computer Science. Springer, 1998. ISBN 978-0-387-98417-9. doi: 10.1007/978-1-4612-1674-2. URL <https://doi.org/10.1007/978-1-4612-1674-2>.
- [27] Patrycja Balik, Dariusz Biernacki, and Piotr Polesiuk. Untyped logical relations at work: Control operators, contextual equivalence and full abstraction. In *Proceedings of the Workshop Dedicated to Olivier Danvy on the Occasion of His 64th Birthday*, OLIVIERFEST '25, pages 128–141, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400721502. doi: 10.1145/3759427.3760374. URL <https://doi.org/10.1145/3759427.3760374>.
- [28] Michael Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem P. de Roever, editors, *Formal Methods for Components and Objects, 4th International Symposium, FMCO 2005, Amsterdam, The Netherlands, November 1-4, 2005, Revised Lectures*, volume 4111 of *Lecture Notes in Computer Science*, pages 364–387. Springer, 2005. doi: 10.1007/11804192_17. URL https://doi.org/10.1007/11804192_17.
- [29] Clark W. Barrett, Christopher L. Conway, Morgan Deters, Liana Hadarean, Dejan Jovanovic, Tim King, Andrew Reynolds, and Cesare Tinelli. CVC4. In Ganesh Gopalakrishnan and Shaz Qadeer, editors, *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*, volume 6806 of *Lecture Notes in Computer Science*, pages 171–177. Springer, 2011. doi: 10.1007/978-3-642-22110-1_14. URL https://doi.org/10.1007/978-3-642-22110-1_14.

- [30] Gilles Barthe. An introduction to relational program verification, 2020.
- [31] Gilles Barthe, Juan Manuel Crespo, and César Kunz. Relational verification using product programs. In Michael J. Butler and Wolfram Schulte, editors, *FM 2011: Formal Methods - 17th International Symposium on Formal Methods, Limerick, Ireland, June 20-24, 2011. Proceedings*, volume 6664 of *Lecture Notes in Computer Science*, pages 200–214. Springer, 2011. doi: 10.1007/978-3-642-21437-0_17. URL https://doi.org/10.1007/978-3-642-21437-0_17.
- [32] Gilles Barthe, Juan Manuel Crespo, and César Kunz. Beyond 2-safety: Asymmetric product programs for relational program verification. In Sergei N. Artëmov and Anil Nerode, editors, *Logical Foundations of Computer Science, International Symposium, LFCS 2013, San Diego, CA, USA, January 6-8, 2013. Proceedings*, volume 7734 of *Lecture Notes in Computer Science*, pages 29–43. Springer, 2013. doi: 10.1007/978-3-642-35722-0_3. URL https://doi.org/10.1007/978-3-642-35722-0_3.
- [33] Andrej Bauer and Matija Pretnar. Programming with algebraic effects and handlers. *J. Log. Algebraic Methods Program.*, 84(1):108–123, 2015. doi: 10.1016/j.jlamp.2014.02.001. URL <https://doi.org/10.1016/j.jlamp.2014.02.001>.
- [34] Andrej Bauer and Matija Pretnar. Eff. <http://www.eff-lang.org/>, 2020.
- [35] Bernhard Beckert and Mattias Ulbrich. Trends in relational program verification. In Peter Müller and Ina Schaefer, editors, *Principled Software Development - Essays Dedicated to Arnd Poetzsch-Heffter on the Occasion of his 60th Birthday*, pages 41–58. Springer, 2018. doi:

10.1007/978-3-319-98047-8_3. URL https://doi.org/10.1007/978-3-319-98047-8_3.

- [36] Nick Benton. Simple relational correctness proofs for static analyses and program transformations. In Neil D. Jones and Xavier Leroy, editors, *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2004, Venice, Italy, January 14-16, 2004*, pages 14–25. ACM, 2004. doi: 10.1145/964001.964003. URL <https://doi.org/10.1145/964001.964003>.
- [37] Josh Berdine, Cristiano Calcagno, and Peter W. O’Hearn. Small-foot: Modular automatic assertion checking with separation logic. In Frank S. de Boer, Marcello M. Bonsangue, Susanne Graf, and Willem P. de Roever, editors, *Formal Methods for Components and Objects, 4th International Symposium, FMCO 2005, Amsterdam, The Netherlands, November 1-4, 2005, Revised Lectures*, volume 4111 of *Lecture Notes in Computer Science*, pages 115–137. Springer, 2005. doi: 10.1007/11804192_6. URL https://doi.org/10.1007/11804192_6.
- [38] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, Catalin Hritcu, Samin Ishtiaq, Markulf Kohlweiss, K. Rustan M. Leino, Jay R. Lorch, Kenji Maillard, Jianyang Pan, Bryan Parno, Jonathan Protzenko, Tahina Ramananandro, Ashay Rane, Aseem Rastogi, Nikhil Swamy, Laure Thompson, Peng Wang, Santiago Zanella Béguelin, and Jean Karim Zinzindohoue. Everest: Towards a verified, drop-in replacement of HTTPS. In Benjamin S. Lerner, Rastislav Bodík, and Shriram Krishnamurthi, editors, *2nd Summit on Advances in Programming Languages, SNAPL 2017, May 7-10, 2017, Asilomar, CA, USA*, volume 71 of *LIPICs*, pages 1:1–1:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi:

10.4230/LIPICS.SNAPL.2017.1. URL <https://doi.org/10.4230/LIPICS.SNAPL.2017.1>.

- [39] Małgorzata Biernacka, Dariusz Biernacki, and Olivier Danvy. An operational foundation for delimited continuations in the CPS hierarchy. *Logical Methods in Computer Science*, 1(2:5):1–39, November 2005.
- [40] Dariusz Biernacki and Olivier Danvy. Theoretical pearl: A simple proof of a folklore theorem about delimited control. *J. Funct. Program.*, 16(3):269–280, 2006. doi: 10.1017/S0956796805005782. URL <https://doi.org/10.1017/S0956796805005782>.
- [41] Dariusz Biernacki, Olivier Danvy, and Chung-chieh Shan. On the static and dynamic extents of delimited continuations. *Sci. Comput. Program.*, 60(3):274–297, 2006. doi: 10.1016/J.SCICO.2006.01.002. URL <https://doi.org/10.1016/j.scico.2006.01.002>.
- [42] Lars Birkedal and Aleš Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic. *Lectures notes, September*, 2023. URL <https://iris-project.org/tutorial-material.html>.
- [43] Lars Birkedal, Ales Bizjak, and Jan Schwinghammer. Step-indexed relational reasoning for countable nondeterminism. *Log. Methods Comput. Sci.*, 9(4), 2013. doi: 10.2168/LMCS-9(4:4)2013. URL [https://doi.org/10.2168/LMCS-9\(4:4\)2013](https://doi.org/10.2168/LMCS-9(4:4)2013).
- [44] Olivier Bouissou, Eric Conquet, Patrick Cousot, Radhia Cousot, Jérôme Feret, Khalil Ghorbal, Eric Goubault, David Lesens, Laurent Mauborgne, Antoine Miné, et al. Space software validation using abstract interpretation. In *The International Space System Engineering Conference: Data Systems in Aerospace-DASIA 2009*, volume 1, pages 1–7. European Space Agency, 2009.

- [45] Sylvain Boulmé. Intuitionistic refinement calculus. In Simona Ronchi Della Rocca, editor, *Typed Lambda Calculi and Applications, 8th International Conference, TLCA 2007, Paris, France, June 26-28, 2007, Proceedings*, volume 4583 of *Lecture Notes in Computer Science*, pages 54–69. Springer, 2007. doi: 10.1007/978-3-540-73228-0_6. URL https://doi.org/10.1007/978-3-540-73228-0_6.
- [46] Jonathan Immanuel Brachthäuser and Philipp Schuster. Effekt: extensible algebraic effects in scala (short paper). In Heather Miller, Philipp Haller, and Ondrej Lhoták, editors, *Proceedings of the 8th ACM SIGPLAN International Symposium on Scala, SCALA@SPLASH 2017, Vancouver, BC, Canada, October 22-23, 2017*, pages 67–72. ACM, 2017. doi: 10.1145/3136000.3136007. URL <https://doi.org/10.1145/3136000.3136007>.
- [47] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. Effects as capabilities: effect handlers and lightweight effect polymorphism. *Proc. ACM Program. Lang.*, 4(OOPSLA):126:1–126:30, 2020. doi: 10.1145/3428194. URL <https://doi.org/10.1145/3428194>.
- [48] Martin Brain and Elizabeth Polgreen. A pyramid of (formal) software verification. In André Platzer, Kristin Yvonne Rozier, Matteo Pradella, and Matteo Rossi, editors, *Formal Methods - 26th International Symposium, FM 2024, Milan, Italy, September 9-13, 2024, Proceedings, Part II*, volume 14934 of *Lecture Notes in Computer Science*, pages 393–419. Springer, 2024. doi: 10.1007/978-3-031-71177-0_24. URL https://doi.org/10.1007/978-3-031-71177-0_24.
- [49] Martin Büchi and Wolfgang Weck. *The Greybox Approach: When Black-box Specification Hide too much*. Turku Centre for Computer Science, 1999.

- [50] Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. Compositional shape analysis by means of bi-abduction. In Zhong Shao and Benjamin C. Pierce, editors, *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, pages 289–300. ACM, 2009. doi: 10.1145/1480881.1480917. URL <https://doi.org/10.1145/1480881.1480917>.
- [51] Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. Compositional shape analysis by means of bi-abduction. *J. ACM*, 58(6):26:1–26:66, 2011. doi: 10.1145/2049697.2049700. URL <https://doi.org/10.1145/2049697.2049700>.
- [52] Ana Cavalcanti, Augusto Sampaio, and Jim Woodcock. Refinement: An overview. In Ana Cavalcanti, Augusto Sampaio, and Jim Woodcock, editors, *Refinement Techniques in Software Engineering, First Pernambuco Summer School on Software Engineering, PSSE 2004, Recife, Brazil, November 23-December 5, 2004, Revised Lectures*, volume 3167 of *Lecture Notes in Computer Science*, pages 1–17. Springer, 2004. doi: 10.1007/11889229_1. URL https://doi.org/10.1007/11889229_1.
- [53] Aleks Chakarov, Jaco Geldenhuys, Matthew Heck, Mike Hicks, Sam Huang, Georges Axel Jaloyan, Anjali Joshi, Rustan Leino, Mikael Mayer, Sean McLaughlin, Akhilesh Mritunjai, Clément Pit Claudel, Sorawee Porncharoenwase, Florian Rabe, Marianna Rapoport, Giles Reger, Cody Roux, Neha Rungta, Robin Salkeld, Matthias Schlaipfer, Daniel Schoepe, Johanna Schwartzentruher, Serdar Tasiran, Aaron Tomb, Emina Torlak, John Tristan, Lucas Wagner, Mike Whalen, Remy Willems, Jenny Xiang, Tae Joon Byun, Joshua Cohen, Ruijie Wang, Junyoung Jang, Jakob Rath, Hira Taqdees Syeda, Dominik

- Wagner, and Yongwei Yuan. Formally verified cloud-scale authorization. 2025. URL <https://www.amazon.science/publications/formally-verified-cloud-scale-authorization>.
- [54] Arthur Charguéraud. Characteristic formulae for the verification of imperative programs. In Manuel M. T. Chakravarty, Zhenjiang Hu, and Olivier Danvy, editors, *Proceeding of the 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011, Tokyo, Japan, September 19-21, 2011*, pages 418–430. ACM, 2011. doi: 10.1145/2034773.2034828. URL <https://doi.org/10.1145/2034773.2034828>.
- [55] Arthur Charguéraud. *A Modern Eye on Separation Logic for Sequential Programs. (Un nouveau regard sur la Logique de Séparation pour les programmes séquentiels)*. 2023. URL <https://tel.archives-ouvertes.fr/tel-04076725>.
- [56] Arthur Charguéraud. *Separation Logic Foundations*. 2024.
- [57] Nathaniel Charlton, Ben Horsfall, and Bernhard Reus. Crowfoot: A verifier for higher-order store programs. In Viktor Kuncak and Andrey Rybalchenko, editors, *Verification, Model Checking, and Abstract Interpretation - 13th International Conference, VMCAI 2012, Philadelphia, PA, USA, January 22-24, 2012. Proceedings*, volume 7148 of *Lecture Notes in Computer Science*, pages 136–151. Springer, 2012. doi: 10.1007/978-3-642-27940-9_10. URL https://doi.org/10.1007/978-3-642-27940-9_10.
- [58] Wei-Ngan Chin, Cristina David, Huu Hai Nguyen, and Shengchao Qin. Automated verification of shape, size and bag properties via user-defined predicates in separation logic. *Sci. Comput. Program.*, 77(9):1006–1036, 2012. doi: 10.1016/J.SCICO.2010.07.004. URL <https://doi.org/10.1016/j.scico.2010.07.004>.

- [59] Adam Chlipala. A verified compiler for an impure functional language. In Manuel V. Hermenegildo and Jens Palsberg, editors, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, pages 93–106. ACM, 2010. doi: 10.1145/1706299.1706312. URL <https://doi.org/10.1145/1706299.1706312>.
- [60] Alexander Collins and Vinod Grover. Probabilistic programming with cuppl. *CoRR*, abs/2010.08454, 2020. URL <https://arxiv.org/abs/2010.08454>.
- [61] Commenters. Multi-shot continuations gone forever? <https://discuss.ocaml.org/t/multi-shot-continuations-gone-forever/9072>, 2022.
- [62] Youyou Cong and Kenichi Asai. Handling delimited continuations with dependent types. *Proc. ACM Program. Lang.*, 2(ICFP):69:1–69:31, 2018. doi: 10.1145/3236764. URL <https://doi.org/10.1145/3236764>.
- [63] Youyou Cong and Kenichi Asai. Understanding algebraic effect handlers via delimited control operators. In Wouter Swierstra and Nicolas Wu, editors, *Trends in Functional Programming - 23rd International Symposium, TFP 2022, Virtual Event, March 17-18, 2022, Revised Selected Papers*, volume 13401 of *Lecture Notes in Computer Science*, pages 59–79. Springer, 2022. doi: 10.1007/978-3-031-21314-4_4. URL https://doi.org/10.1007/978-3-031-21314-4_4.
- [64] Olivier Danvy. Programming with tighter control. *Special issue of the BIGRE journal Putting the Scheme Language to Work, Brest, France (July 1989)*, pages 10–29, 1989.
- [65] Olivier Danvy. Type-directed partial evaluation. In Hans-Juergen

- Boehm and Guy L. Steele Jr., editors, *Conference Record of POPL96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*, pages 242–257. ACM Press, 1996. doi: 10.1145/237721.237784. URL <https://doi.org/10.1145/237721.237784>.
- [66] Olivier Danvy and Andrzej Filinski. A functional abstraction of typed contexts. DIKU Rapport 89/12, DIKU, Computer Science Department, University of Copenhagen, Copenhagen, Denmark, July 1989.
- [67] Olivier Danvy and Andrzej Filinski. Abstracting control. In Gilles Kahn, editor, *Proceedings of the 1990 ACM Conference on LISP and Functional Programming, LFP 1990, Nice, France, 27-29 June 1990*, pages 151–160. ACM, 1990. doi: 10.1145/91556.91622. URL <https://doi.org/10.1145/91556.91622>.
- [68] Olivier Danvy and Lasse R. Nielsen. Defunctionalization at work. In *Proceedings of the 3rd international ACM SIGPLAN conference on Principles and practice of declarative programming, September 5-7, 2001, Florence, Italy*, pages 162–174. ACM, 2001. doi: 10.1145/773184.773202. URL <https://doi.org/10.1145/773184.773202>.
- [69] Thibault Dardinier and Peter Müller. Hyper Hoare logic: (dis-)proving program hyperproperties. *Proc. ACM Program. Lang.*, 8(PLDI):1485–1509, 2024. doi: 10.1145/3656437. URL <https://doi.org/10.1145/3656437>.
- [70] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. Z3: an efficient SMT solver. In C. R. Ramakrishnan and Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of*

the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24. URL https://doi.org/10.1007/978-3-540-78800-3_24.

- [71] Paulo de Vilhena. *Proof of Programs with Effect Handlers. (Preuve de Programmes avec Effect Handlers)*. PhD thesis, Paris Cité University, France, 2022. URL <https://tel.archives-ouvertes.fr/tel-03891381>.
- [72] Paulo Emílio de Vilhena and François Pottier. A separation logic for effect handlers. *Proc. ACM Program. Lang.*, 5(POPL):1–28, 2021. doi: 10.1145/3434314. URL <https://doi.org/10.1145/3434314>.
- [73] Paulo Emílio de Vilhena and François Pottier. Verifying an effect-handler-based define-by-run reverse-mode AD library. *Log. Methods Comput. Sci.*, 19(4), 2023. doi: 10.46298/LMCS-19(4:5)2023. URL [https://doi.org/10.46298/lmcs-19\(4:5\)2023](https://doi.org/10.46298/lmcs-19(4:5)2023).
- [74] Germán Andrés Delbianco and Aleksandar Nanevski. Hoare-style reasoning with (algebraic) continuations. In Greg Morrisett and Tarmo Uustalu, editors, *ACM SIGPLAN International Conference on Functional Programming, ICFP’13, Boston, MA, USA - September 25 - 27, 2013*, pages 363–376. ACM, 2013. doi: 10.1145/2500365.2500593. URL <https://doi.org/10.1145/2500365.2500593>.
- [75] Xavier Denis and Jacques-Henri Jourdan. Specifying and verifying higher-order Rust iterators. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 29th International Conference, TACAS 2023, Held as Part of the European Joint Conferences on Theory and Practice of*

- Software, ETAPS 2022, Paris, France, April 22-27, 2023, Proceedings, Part II*, volume 13994 of *Lecture Notes in Computer Science*, pages 93–110. Springer, 2023. doi: 10.1007/978-3-031-30820-8_9. URL https://doi.org/10.1007/978-3-031-30820-8_9.
- [76] Robert Dickerson, Qianchuan Ye, Michael K. Zhang, and Benjamin Delaware. RHLE: modular deductive verification of relational $\forall \exists$ properties. In Ilya Sergey, editor, *Programming Languages and Systems - 20th Asian Symposium, APLAS 2022, Auckland, New Zealand, December 5, 2022, Proceedings*, volume 13658 of *Lecture Notes in Computer Science*, pages 67–87. Springer, 2022. doi: 10.1007/978-3-031-21037-2_4. URL https://doi.org/10.1007/978-3-031-21037-2_4.
- [77] Robert Dickerson, Prasita Mukherjee, and Benjamin Delaware. Kestrel: Relational verification using e-graphs for program alignment. *Proc. ACM Program. Lang.*, 9(OOPSLA1):1073–1100, 2025. doi: 10.1145/3720474. URL <https://doi.org/10.1145/3720474>.
- [78] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Commun. ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975. URL <https://doi.org/10.1145/360933.360975>.
- [79] Derek Dreyer, Amal Ahmed, and Lars Birkedal. Logical step-indexed logical relations. In *Proceedings of the 24th Annual IEEE Symposium on Logic in Computer Science, LICS 2009, 11-14 August 2009, Los Angeles, CA, USA*, pages 71–80. IEEE Computer Society, 2009. doi: 10.1109/LICS.2009.34. URL <https://doi.org/10.1109/LICS.2009.34>.
- [80] R. Kent Dybvig, Simon L. Peyton Jones, and Amr Sabry. A monadic framework for delimited continuations. *J. Funct. Program.*, 17(6):

- 687–730, 2007. doi: 10.1017/S0956796807006259. URL <https://doi.org/10.1017/S0956796807006259>.
- [81] Gabriel Ebner, Guido Martínez, Aseem Rastogi, Thibault Dardinier, Megan Frisella, Tahina Ramananandro, and Nikhil Swamy. Pulsecore: An impredicative concurrent separation logic for dependently typed programs. In *2025 Programming Language Design and Implementation*. ACM, June 2025. URL <https://www.microsoft.com/en-us/research/publication/pulsecore-an-impredicative-concurrent-separation-logic-for-dependently-typed>
Accepted for publication, to appear.
- [82] Matthias Felleisen. The theory and practice of first-class prompts. In Jeanne Ferrante and Peter Mager, editors, *Conference Record of the Fifteenth Annual ACM Symposium on Principles of Programming Languages, San Diego, California, USA, January 10-13, 1988*, pages 180–190. ACM Press, 1988. doi: 10.1145/73560.73576. URL <https://doi.org/10.1145/73560.73576>.
- [83] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 - where programs meet provers. In *European Symposium on Programming*, 2013. URL <https://api.semanticscholar.org/CorpusID:14572425>.
- [84] Robert Bruce Findler and Matthias Felleisen. Contracts for Higher-order Functions. In Mitchell Wand and Simon L. Peyton Jones, editors, *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP '02), Pittsburgh, Pennsylvania, USA, October 4-6, 2002*, pages 48–59. ACM, 2002. doi: 10.1145/581478.581484. URL <https://doi.org/10.1145/581478.581484>.
- [85] Darius Foo and Wei-Ngan Chin. Tracing OCaml programs. *CoRR*, abs/2304.04937, 2023. doi: 10.48550/ARXIV.2304.04937. URL <https://doi.org/10.48550/arXiv.2304.04937>.

- [86] Darius Foo, Andreea Costea, and Wei-Ngan Chin. Protocol conformance with Choreographic PlusCal. In Cristina David and Meng Sun, editors, *Theoretical Aspects of Software Engineering - 17th International Symposium, TASE 2023, Bristol, UK, July 4-6, 2023, Proceedings*, volume 13931 of *Lecture Notes in Computer Science*, pages 126–145. Springer, 2023. doi: 10.1007/978-3-031-35257-7_8. URL https://doi.org/10.1007/978-3-031-35257-7_8.
- [87] Darius Foo, Yahui Song, and Wei-Ngan Chin. Staged specification logic for verifying higher-order imperative programs. In *FM 2024: Formal Methods - 17th International Symposium on Formal Methods, Milan, Italy, Sept 9-13, 2024. Proceedings*, *Lecture Notes in Computer Science*. Springer, 2024.
- [88] Yannick Forster, Ohad Kammar, Sam Lindley, and Matija Pretnar. On the expressive power of user-defined effects: effect handlers, monadic reflection, delimited control. *Proc. ACM Program. Lang.*, 1(ICFP): 13:1–13:29, 2017. doi: 10.1145/3110257. URL <https://doi.org/10.1145/3110257>.
- [89] Aymeric Fromherz, Aseem Rastogi, Nikhil Swamy, Sydney Gibson, Guido Martínez, Denis Merigoux, and Tahina Ramananandro. Steel: proof-oriented programming in a dependently typed concurrent separation logic. *Proceedings of the ACM on Programming Languages*, 5: 1 – 30, 2021. URL <https://api.semanticscholar.org/CorpusID:233474595>.
- [90] Dan Frumin, Robbert Krebbers, and Lars Birkedal. Reloc reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. *Log. Methods Comput. Sci.*, 17(3), 2021. doi: 10.46298/LMCS-17(3:9)2021. URL [https://doi.org/10.46298/lmcs-17\(3:9\)2021](https://doi.org/10.46298/lmcs-17(3:9)2021).

- [91] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. Simuliris: a separation logic framework for verifying concurrent program optimizations. *Proc. ACM Program. Lang.*, 6(POPL):1–31, 2022. doi: 10.1145/3498689. URL <https://doi.org/10.1145/3498689>.
- [92] Cristian Gherghina, Cristina David, Shengchao Qin, and Wei-Ngan Chin. Expressive program verification via structured specifications. *Int. J. Softw. Tools Technol. Transf.*, 16(4):363–380, 2014. doi: 10.1007/S10009-014-0306-5. URL <https://doi.org/10.1007/s10009-014-0306-5>.
- [93] Jeremy Gibbons and Ralf Hinze. Just do it: simple monadic equational reasoning. In Manuel M. T. Chakravarty, Zhenjiang Hu, and Olivier Danvy, editors, *Proceeding of the 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011, Tokyo, Japan, September 19-21, 2011*, pages 2–14. ACM, 2011. doi: 10.1145/2034773.2034777. URL <https://doi.org/10.1145/2034773.2034777>.
- [94] Paul T. Graunke, Robert Bruce Findler, Shriram Krishnamurthi, and Matthias Felleisen. Modeling web interactions. In Pierpaolo Degano, editor, *Programming Languages and Systems, 12th European Symposium on Programming, ESOP 2003, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2003, Warsaw, Poland, April 7-11, 2003, Proceedings*, volume 2618 of *Lecture Notes in Computer Science*, pages 238–252. Springer, 2003. doi: 10.1007/3-540-36575-3_17. URL https://doi.org/10.1007/3-540-36575-3_17.
- [95] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An extensible architecture for building certified concurrent os kernels. In Kim-

- berly Keeton and Timothy Roscoe, editors, *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, pages 653–669. USENIX Association, 2016. URL <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/gu>.
- [96] Carl A. Gunter, Didier Rémy, and Jon G. Riecke. A generalization of exceptions and control in ML-like languages. In John Williams, editor, *Proceedings of the seventh international conference on Functional programming languages and computer architecture, FPCA 1995, La Jolla, California, USA, June 25-28, 1995*, pages 12–23. ACM, 1995. doi: 10.1145/224164.224173. URL <https://doi.org/10.1145/224164.224173>.
- [97] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael Lowell Roberts, Srinath T. V. Setty, and Brian Zill. Ironfleet: proving safety and liveness of practical distributed systems. *Commun. ACM*, 60(7):83–92, 2017. doi: 10.1145/3068608. URL <https://doi.org/10.1145/3068608>.
- [98] Daniel Hillerström and Sam Lindley. Shallow effect handlers. In Sukyoung Ryu, editor, *Programming Languages and Systems - 16th Asian Symposium, APLAS 2018, Wellington, New Zealand, December 2-6, 2018, Proceedings*, volume 11275 of *Lecture Notes in Computer Science*, pages 415–435. Springer, 2018. doi: 10.1007/978-3-030-02768-1_22. URL https://doi.org/10.1007/978-3-030-02768-1_22.
- [99] Noriko Hirota and Kenichi Asai. Correctness of functions with shift and reset. In *Proceedings of the 2011 ACM SIGPLAN Continuation Workshop*, pages 23–25, 2011. URL <https://www.cs.tsukuba.ac.jp/~kam/cw2011/book.pdf>.

- [100] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259. URL <https://doi.org/10.1145/363235.363259>.
- [101] Aquinas Hobor and Jules Villard. The ramifications of sharing in data structures. In Roberto Giacobazzi and Radhia Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, pages 523–536. ACM, 2013. doi: 10.1145/2429069.2429131. URL <https://doi.org/10.1145/2429069.2429131>.
- [102] Aquinas Hobor, Robert Dockins, and Andrew W. Appel. A theory of indirection via approximation. In Manuel V. Hermenegildo and Jens Palsberg, editors, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, pages 171–184. ACM, 2010. doi: 10.1145/1706299.1706322. URL <https://doi.org/10.1145/1706299.1706322>.
- [103] Dániel Horpácsi, Péter Bereczky, and Simon J. Thompson. Program equivalence in an untyped, call-by-value functional language with uncurried functions. *J. Log. Algebraic Methods Program.*, 132:100857, 2023. doi: 10.1016/J.JLAMP.2023.100857. URL <https://doi.org/10.1016/j.jlamp.2023.100857>.
- [104] Nikolaus Huber, Naomi Spargo, Nicolas Osborne, Samuel Hym, and Jan Midtgaard. Dynamic verification of OCaml software with gospel and ortac/qcheck-stm. In Arie Gurfinkel and Marijn Heule, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings*,

- Part III*, volume 15698 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2025. doi: 10.1007/978-3-031-90660-2_1. URL https://doi.org/10.1007/978-3-031-90660-2_1.
- [105] R. John M. Hughes. A novel representation of lists and its application to the function "reverse". *Inf. Process. Lett.*, 22(3):141–144, 1986. doi: 10.1016/0020-0190(86)90059-1. URL [https://doi.org/10.1016/0020-0190\(86\)90059-1](https://doi.org/10.1016/0020-0190(86)90059-1).
- [106] Martin Hyland, Paul Blain Levy, Gordon D. Plotkin, and John Power. Combining algebraic effects with continuations. *Theor. Comput. Sci.*, 375(1-3):20–40, 2007. doi: 10.1016/J.TCS.2006.12.026. URL <https://doi.org/10.1016/j.tcs.2006.12.026>.
- [107] Chiaki Ishio and Kenichi Asai. Type system for four delimited control operators. In Bernhard Scholz and Yuki Yoshi Kameyama, editors, *Proceedings of the 21st ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences, GPCE 2022, Auckland, New Zealand, December 6-7, 2022*, pages 45–58. ACM, 2022. doi: 10.1145/3564719.3568691. URL <https://doi.org/10.1145/3564719.3568691>.
- [108] Bart Jacobs. Verifast’s separation logic: a higher-order(ish) logic without later for modular verification of fine-grained concurrent programs. *CoRR*, abs/2505.04500, 2025. doi: 10.48550/ARXIV.2505.04500. URL <https://doi.org/10.48550/arXiv.2505.04500>.
- [109] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. Verifast: A powerful, sound, predictable, fast verifier for C and java. In Mihaela Gheorghiu Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi, editors, *NASA Formal Methods - Third International Symposium, NFM*

- 2011, Pasadena, CA, USA, April 18-20, 2011. *Proceedings*, volume 6617 of *Lecture Notes in Computer Science*, pages 41–55. Springer, 2011. doi: 10.1007/978-3-642-20398-5_4. URL https://doi.org/10.1007/978-3-642-20398-5_4.
- [110] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.*, 28:e20, 2018. doi: 10.1017/S0956796818000151. URL <https://doi.org/10.1017/S0956796818000151>.
- [111] Yuki Yoshi Kameyama. Axioms for delimited continuations in the CPS hierarchy. In Jerzy Marcinkowski and Andrzej Tarlecki, editors, *Computer Science Logic, 18th International Workshop, CSL 2004, 13th Annual Conference of the EACSL, Karpacz, Poland, September 20-24, 2004, Proceedings*, volume 3210 of *Lecture Notes in Computer Science*, pages 442–457. Springer, 2004. doi: 10.1007/978-3-540-30124-0_34. URL https://doi.org/10.1007/978-3-540-30124-0_34.
- [112] Yuki Yoshi Kameyama and Masahito Hasegawa. A sound and complete axiomatization of delimited continuations. In Colin Runciman and Olin Shivers, editors, *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming, ICFP 2003, Uppsala, Sweden, August 25-29, 2003*, pages 177–188. ACM, 2003. doi: 10.1145/944705.944722. URL <https://doi.org/10.1145/944705.944722>.
- [113] Yuki Yoshi Kameyama and Takuo Yonezawa. Typed dynamic control operators for delimited continuations. In Jacques Garrigue and Manuel V. Hermenegildo, editors, *Functional and Logic Programming, 9th International Symposium, FLOPS 2008, Ise, Japan, April 14-16, 2008. Proceedings*, volume 4989 of *Lecture Notes in Computer Science*,

- pages 239–254. Springer, 2008. doi: 10.1007/978-3-540-78969-7_18. URL https://doi.org/10.1007/978-3-540-78969-7_18.
- [114] Johannes Kanig and Jean-Christophe Filliâtre. Who: a verifier for effectful higher-order programs. In *Proceedings of the 2009 ACM SIG-PLAN workshop on ML*, pages 39–48, 2009.
- [115] Fuga Kawamata, Hiroshi Unno, Taro Sekiyama, and Tachio Terauchi. Answer refinement modification: Refinement type system for algebraic effects and handlers. *Proc. ACM Program. Lang.*, 8(POPL):115–147, 2024. doi: 10.1145/3633280. URL <https://doi.org/10.1145/3633280>.
- [116] Alexis King. Delimited continuation primops. https://gitlab.haskell.org/ghc/ghc/-/merge_requests/7942, 2022.
- [117] Oleg Kiselyov. Genuine shift/reset in haskell98. <https://okmij.org/ftp/continuations/implementations.html>, 2007.
- [118] Oleg Kiselyov. Delimited control in OCaml, abstractly and concretely: System description. In Matthias Blume, Naoki Kobayashi, and Germán Vidal, editors, *Functional and Logic Programming, 10th International Symposium, FLOPS 2010, Sendai, Japan, April 19-21, 2010. Proceedings*, volume 6009 of *Lecture Notes in Computer Science*, pages 304–320. Springer, 2010. doi: 10.1007/978-3-642-12251-4_22. URL https://doi.org/10.1007/978-3-642-12251-4_22.
- [119] Oleg Kiselyov. Three new monad transformers for multi-prompt delimited control. <https://okmij.org/ftp/continuations/implementations.html#CC-monads>, 2010.
- [120] Oleg Kiselyov. Delimited control in OCaml, abstractly and concretely. *Theor. Comput. Sci.*, 435:56–76, 2012. doi: 10.1016/J.TCS.2012.02.025. URL <https://doi.org/10.1016/j.tcs.2012.02.025>.

- [121] Oleg Kiselyov. Undelimited continuations do not occur in practice. <https://okmij.org/ftp/continuations/against-callcc.html#illusion>, 2012.
- [122] Oleg Kiselyov and Chung-chieh Shan. Delimited continuations in operating systems. In Boicho N. Kokinov, Daniel C. Richardson, Thomas Roth-Berghofer, and Laure Vieu, editors, *Modeling and Using Context, 6th International and Interdisciplinary Conference, CONTEXT 2007, Roskilde, Denmark, August 20-24, 2007, Proceedings*, volume 4635 of *Lecture Notes in Computer Science*, pages 291–302. Springer, 2007. doi: 10.1007/978-3-540-74255-5_22. URL https://doi.org/10.1007/978-3-540-74255-5_22.
- [123] Oleg Kiselyov and Chung-chieh Shan. Embedded probabilistic programming. In Walid Mohamed Taha, editor, *Domain-Specific Languages, IFIP TC 2 Working Conference, DSL 2009, Oxford, UK, July 15-17, 2009, Proceedings*, volume 5658 of *Lecture Notes in Computer Science*, pages 360–384. Springer, 2009. doi: 10.1007/978-3-642-03034-5_17. URL https://doi.org/10.1007/978-3-642-03034-5_17.
- [124] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David A. Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: formal verification of an os kernel. In Jeanna Neeffe Matthews and Thomas E. Anderson, editors, *Proceedings of the 22nd ACM Symposium on Operating Systems Principles 2009, SOSP 2009, Big Sky, Montana, USA, October 11-14, 2009*, pages 207–220. ACM, 2009. doi: 10.1145/1629575.1629596. URL <https://doi.org/10.1145/1629575.1629596>.
- [125] Joomy Korkut. Intrinsic vs. extrinsic verifica-

- tion. <https://joomy.korkutblech.com/posts/2024-10-30-intrinsic-vs-extrinsic-verification.html>, 2024.
- [126] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In Giuseppe Castagna and Andrew D. Gordon, editors, *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, pages 205–217. ACM, 2017. doi: 10.1145/3009837.3009855. URL <https://doi.org/10.1145/3009837.3009855>.
 - [127] Leslie Lamport. *Specifying systems*, volume 388. Addison-Wesley Boston, 2002.
 - [128] P. J. Landin. The mechanical evaluation of expressions. *Comput. J.*, 6(4):308–320, 1964. doi: 10.1093/COMJNL/6.4.308. URL <https://doi.org/10.1093/comjnl/6.4.308>.
 - [129] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proc. ACM Program. Lang.*, 7(OOPSLA1):286–315, 2023. doi: 10.1145/3586037. URL <https://doi.org/10.1145/3586037>.
 - [130] Julia L. Lawall and Olivier Danvy. Continuation-based partial evaluation. In Robert R. Kessler, editor, *Proceedings of the 1994 ACM Conference on LISP and Functional Programming, Orlando, Florida, USA, 27-29 June 1994*, pages 227–238. ACM, 1994. doi: 10.1145/182409.182483. URL <https://doi.org/10.1145/182409.182483>.
 - [131] Gary T. Leavens and David A. Naumann. Behavioral subtyping, specification inheritance, and modular reasoning. *ACM Trans. Program.*

- Lang. Syst.*, 37(4):13:1–13:88, 2015. doi: 10.1145/2766446. URL <https://doi.org/10.1145/2766446>.
- [132] Gary T. Leavens, David A. Naumann, Hridayesh Rajan, and Tomoyuki Aotani. Specifying and verifying advanced control features. In Tiziana Margaria and Bernhard Steffen, editors, *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications - 7th International Symposium, ISoLA 2016, Imperial, Corfu, Greece, October 10-14, 2016, Proceedings, Part II*, volume 9953 of *Lecture Notes in Computer Science*, pages 80–96, 2016. doi: 10.1007/978-3-319-47169-3_7. URL https://doi.org/10.1007/978-3-319-47169-3_7.
- [133] Daan Leijen. Koka. <https://www.microsoft.com/en-us/research/project/koka/>, 2012.
- [134] Daan Leijen. Koka: Programming with row polymorphic effect types. In Paul Blain Levy and Neel Krishnaswami, editors, *Proceedings 5th Workshop on Mathematically Structured Functional Programming, MSFP@ETAPS 2014, Grenoble, France, 12 April 2014*, volume 153 of *EPTCS*, pages 100–126, 2014. doi: 10.4204/EPTCS.153.8. URL <https://doi.org/10.4204/EPTCS.153.8>.
- [135] Daan Leijen. Algebraic effects for functional programming. Technical report, Microsoft, 2016.
- [136] Daan Leijen. Structured asynchrony with algebraic effects. In Sam Lindley and Brent A. Yorgey, editors, *Proceedings of the 2nd ACM SIGPLAN International Workshop on Type-Driven Development, TyDe@ICFP 2017, Oxford, UK, September 3, 2017*, pages 16–29. ACM, 2017. doi: 10.1145/3122975.3122977. URL <https://doi.org/10.1145/3122975.3122977>.

- [137] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In Edmund M. Clarke and Andrei Voronkov, editors, *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25-May 1, 2010, Revised Selected Papers*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20. URL https://doi.org/10.1007/978-3-642-17511-4_20.
- [138] Xavier Leroy, Sandrine Blazy, Daniel Kästner, Bernhard Schommer, Markus Pister, and Christian Ferdinand. CompCert – a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems*. SEE, 2016.
- [139] Stéphane Lescuyer. *Formalizing and Implementing a Reflexive Tactic for Automated Deduction in Coq. (Formalisation et développement d’une tactique reflexive pour la demonstration automatique en coq)*. PhD thesis, University of Paris-Sud, Orsay, France, 2011. URL <https://tel.archives-ouvertes.fr/tel-00713668>.
- [140] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondrej Lhoták, José Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundness: a manifesto. *Commun. ACM*, 58(2):44–46, 2015. doi: 10.1145/2644805. URL <https://doi.org/10.1145/2644805>.
- [141] Kenji Maillard. Principles of program verification for arbitrary monadic effects. (principes de la vérification de programmes à effets monadiques arbitraires). 2019.
- [142] Kenji Maillard, Danel Ahman, Robert Atkey, Guido Martínez, Catalin Hritcu, Exequiel Rivas, and Éric Tanter. Dijkstra monads for all. *Proc.*

- ACM Program. Lang.*, 3(ICFP):104:1–104:29, 2019. doi: 10.1145/3341708. URL <https://doi.org/10.1145/3341708>.
- [143] Moe Masuko and Kenichi Asai. Caml light+ shift/reset= caml shift. *Theory and Practice of Delimited Continuations (TPDC 2011)*, pages 33–46, 2011.
- [144] Marek Materzok and Dariusz Biernacki. Subtyping delimited continuations. In Manuel M. T. Chakravarty, Zhenjiang Hu, and Olivier Danvy, editors, *Proceeding of the 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011, Tokyo, Japan, September 19-21, 2011*, pages 81–93. ACM, 2011. doi: 10.1145/2034773.2034786. URL <https://doi.org/10.1145/2034773.2034786>.
- [145] Philip McGrath. Racket comes with a continuation-based web server framework that makes extensive use of resuming continuations multiple times. <https://github.com/WebAssembly/stack-switching/issues/110#issuecomment-2927824629>, 2025.
- [146] Eugenio Moggi. Notions of computation and monads. *Inf. Comput.*, 93(1):55–92, 1991. doi: 10.1016/0890-5401(91)90052-4. URL [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).
- [147] Guillem Bartrina I Moreno. *Lexical Delimited Continuations for Scala 3*. PhD thesis, École Polytechnique Fédérale de Lausanne, 2025.
- [148] Carroll Morgan. The refinement calculus. In *NATO ASI PDC*, 1994. URL <https://api.semanticscholar.org/CorpusID:10923264>.
- [149] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. Viper: A verification infrastructure for permission-based reasoning. In Alexander Pretschner, Doron Peled, and Thomas Hutzelmann, editors, *Dependable Software Systems Engineering*, volume 50 of *NATO*

- Science for Peace and Security Series - D: Information and Communication Security*, pages 104–125. IOS Press, 2017. doi: 10.3233/978-1-61499-810-5-104. URL <https://doi.org/10.3233/978-1-61499-810-5-104>.
- [150] Aleksandar Nanevski, J. Gregory Morrisett, and Lars Birkedal. Hoare type theory, polymorphism and separation. *J. Funct. Program.*, 18(5-6):865–911, 2008. doi: 10.1017/S0956796808006953. URL <https://doi.org/10.1017/S0956796808006953>.
- [151] Minh Nguyen, Roly Perera, Meng Wang, and Nicolas Wu. Modular probabilistic models via algebraic effects. *Proc. ACM Program. Lang.*, 6(ICFP):381–410, 2022. doi: 10.1145/3547635. URL <https://doi.org/10.1145/3547635>.
- [152] Zhaozhong Ni and Zhong Shao. Certified assembly programming with embedded code pointers. In J. Gregory Morrisett and Simon L. Peyton Jones, editors, *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2006, Charleston, South Carolina, USA, January 11-13, 2006*, pages 320–333. ACM, 2006. doi: 10.1145/1111037.1111066. URL <https://doi.org/10.1145/1111037.1111066>.
- [153] Liam O'Connor. Lightweight formal methods for programming. https://liamoc.net/work_with_me.html, 2024.
- [154] Peter W. O'Hearn. A primer on separation logic (and automatic program verification and analysis). In Tobias Nipkow, Orna Grumberg, and Benedikt Hauptmann, editors, *Software Safety and Security - Tools for Analysis and Verification*, volume 33 of *NATO Science for Peace and Security Series - D: Information and Communication Security*, pages 286–318. IOS Press, 2012. doi: 10.3233/978-1-61499-028-4-286. URL <https://doi.org/10.3233/978-1-61499-028-4-286>.

- [155] Peter W. O’Hearn. Incorrectness logic. *Proc. ACM Program. Lang.*, 4(POPL):10:1–10:32, 2020. doi: 10.1145/3371078. URL <https://doi.org/10.1145/3371078>.
- [156] Andrei Paskevich, Paul Patault, and Jean-Christophe Filliâtre. Coma, an intermediate verification language with explicit abstraction barriers. In Viktor Vafeiadis, editor, *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part II*, volume 15695 of *Lecture Notes in Computer Science*, pages 175–201. Springer, 2025. doi: 10.1007/978-3-031-91121-7_8. URL https://doi.org/10.1007/978-3-031-91121-7_8.
- [157] Willem Penninckx, Amin Timany, and Bart Jacobs. Specifying I/O using abstract nested Hoare triples in separation logic. In Toby Murray and Gidon Ernst, editors, *Proceedings of the 21st Workshop on Formal Techniques for Java-like Programs, FTfJP@ECOOP 2019, London, United Kingdom, July 15, 2019*, pages 5:1–5:7. ACM, 2019. doi: 10.1145/3340672.3341118. URL <https://doi.org/10.1145/3340672.3341118>.
- [158] Mário Pereira and António Ravara. Cameleer: A deductive verification tool for OCaml. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II*, volume 12760 of *Lecture Notes in Computer Science*, pages 677–689. Springer, 2021. doi: 10.1007/978-3-030-81688-9_31. URL https://doi.org/10.1007/978-3-030-81688-9_31.
- [159] Donald Pinckney, Arjun Guha, and Yuriy Brun. Wasm/k: delimited continuations for webassembly. In Matthew Flat, editor, *DLS*

- 2020: *Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages, Virtual Event, USA, November 17, 2020*, pages 16–28. ACM, 2020. doi: 10.1145/3426422.3426978. URL <https://doi.org/10.1145/3426422.3426978>.
- [160] Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. Typed equivalence of effect handlers and delimited control. In Herman Geuvers, editor, *4th International Conference on Formal Structures for Computation and Deduction, FSCD 2019, Dortmund, Germany, June 24-30, 2019*, volume 131 of *LIPICs*, pages 30:1–30:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi: 10.4230/LIPICS.FSCD.2019.30. URL <https://doi.org/10.4230/LIPICS.FSCD.2019.30>.
- [161] Andrew Pitts and Ian Stark. Operational reasoning for functions with local state. In Andrew Gordon and Andrew Pitts, editors, *Higher Order Operational Techniques in Semantics*, pages 227–273. Publications of the Newton Institute, Cambridge University Press, 1998. URL <http://www.inf.ed.ac.uk/~stark/operfl.html>.
- [162] Gordon D. Plotkin and John Power. Notions of computation determine monads. In Mogens Nielsen and Uffe Engberg, editors, *Foundations of Software Science and Computation Structures, 5th International Conference, FOSSACS 2002. Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2002 Grenoble, France, April 8-12, 2002, Proceedings*, volume 2303 of *Lecture Notes in Computer Science*, pages 342–356. Springer, 2002. doi: 10.1007/3-540-45931-6_24. URL https://doi.org/10.1007/3-540-45931-6_24.
- [163] Gordon D. Plotkin and John Power. Algebraic operations and generic effects. *Appl. Categorical Struct.*, 11(1):69–94, 2003. doi: 10.1023/A:1023064908962. URL <https://doi.org/10.1023/A:1023064908962>.

- [164] Gordon D. Plotkin and Matija Pretnar. Handlers of algebraic effects. In Giuseppe Castagna, editor, *Programming Languages and Systems, 18th European Symposium on Programming, ESOP 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, York, UK, March 22-29, 2009. Proceedings*, volume 5502 of *Lecture Notes in Computer Science*, pages 80–94. Springer, 2009. doi: 10.1007/978-3-642-00590-9_7. URL https://doi.org/10.1007/978-3-642-00590-9_7.
- [165] Piotr Polesiuk. Ixfree: Step-indexed logical relations in coq. In *3rd International Workshop on Coq for Programming Languages (CoqPL)*, 2017.
- [166] Piotr Polesiuk and Filip Sieczkowski. Functorial syntax for all, 2024.
- [167] prophet. Why are there so many libraries for algebraic effects? (comment). <https://discourse.haskell.org/t/why-are-there-so-many-libraries-for-algebraic-effects/11844/29>, 2025.
- [168] Olivia Proust and Frédéric Loulergue. Verified scalable parallel computing with why3. In Carla Ferreira and Tim A. C. Willemse, editors, *Software Engineering and Formal Methods - 21st International Conference, SEFM 2023, Eindhoven, The Netherlands, November 6-10, 2023, Proceedings*, volume 14323 of *Lecture Notes in Computer Science*, pages 246–262. Springer, 2023. doi: 10.1007/978-3-031-47115-5_14. URL https://doi.org/10.1007/978-3-031-47115-5_14.
- [169] Brian Randell. The 1968/69 NATO software engineering reports. *History of Software Engineering*, 37, 1996.
- [170] Yann Régis-Gianas and François Pottier. A Hoare logic for call-by-value functional programs. In Philippe Audebaud and Christine Paulin-

- Mohring, editors, *Mathematics of Program Construction, 9th International Conference, MPC 2008, Marseille, France, July 15-18, 2008. Proceedings*, volume 5133 of *Lecture Notes in Computer Science*, pages 305–335. Springer, 2008. doi: 10.1007/978-3-540-70594-9_17. URL https://doi.org/10.1007/978-3-540-70594-9_17.
- [171] Yann Régis-Gianas and François Pottier. A Hoare logic for call-by-value functional programs. In *International Conference on Mathematics of Program Construction*, 2008. URL <https://api.semanticscholar.org/CorpusID:1095142>.
- [172] John C. Reynolds. Definitional interpreters for higher-order programming languages. In John J. Donovan and Rosemary Shields, editors, *Proceedings of the ACM annual conference, ACM 1972, 1972, Volume 2*, pages 717–740. ACM, 1972. doi: 10.1145/800194.805852. URL <https://doi.org/10.1145/800194.805852>.
- [173] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*, pages 55–74. IEEE Computer Society, 2002. doi: 10.1109/LICS.2002.1029817. URL <https://doi.org/10.1109/LICS.2002.1029817>.
- [174] Jan Schwinghammer, Lars Birkedal, Bernhard Reus, and Hongseok Yang. Nested Hoare triples and frame rules for higher-order store. In Erich Grädel and Reinhard Kahle, editors, *Computer Science Logic, 23rd international Workshop, CSL 2009, 18th Annual Conference of the EACSL, Coimbra, Portugal, September 7-11, 2009. Proceedings*, volume 5771 of *Lecture Notes in Computer Science*, pages 440–454. Springer, 2009. doi: 10.1007/978-3-642-04027-6_32. URL https://doi.org/10.1007/978-3-642-04027-6_32.

- [175] Taro Sekiyama and Hiroshi Unno. Temporal verification with answer-effect modification: Dependent temporal type-and-effect system with delimited continuations. *Proc. ACM Program. Lang.*, 7(POPL):2079–2110, 2023. doi: 10.1145/3571264. URL <https://doi.org/10.1145/3571264>.
- [176] Chung-chieh Shan. Delimited continuations in natural language: quantification and polarity sensitivity. *CoRR*, cs.CL/0404006, 2004. URL <http://arxiv.org/abs/cs/0404006>.
- [177] Steve M. Shaner, Gary T. Leavens, and David A. Naumann. Modular verification of higher-order methods with mandatory calls specified by model programs. *Proceedings of the 22nd annual ACM SIGPLAN conference on Object-oriented programming systems, languages and applications*, 2007.
- [178] Rahul Sharma, Saurabh Gupta, Bharath Hariharan, Alex Aiken, and Aditya V. Nori. Verification as learning geometric concepts. In Francesco Logozzo and Manuel Fähndrich, editors, *Static Analysis - 20th International Symposium, SAS 2013, Seattle, WA, USA, June 20-22, 2013. Proceedings*, volume 7935 of *Lecture Notes in Computer Science*, pages 388–411. Springer, 2013. doi: 10.1007/978-3-642-38856-9_21. URL https://doi.org/10.1007/978-3-642-38856-9_21.
- [179] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. Retrofitting effect handlers onto OCaml. In Stephen N. Freund and Eran Yahav, editors, *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, pages 206–221. ACM, 2021. doi: 10.1145/3453483.3454039. URL <https://doi.org/10.1145/3453483.3454039>.

- [180] KC Sivaramakrishnan. `delimcc_of_fxhandler` - `delimcc` primitives from OCaml 5 effect handlers. https://github.com/kayceesrk/delimcc_of_fxhandler, 2022.
- [181] Tiago Soares and Mário Pereira. A framework for the automated verification of algebraic effects and handlers (extended version). *CoRR*, abs/2302.01265, 2023. doi: 10.48550/ARXIV2302.01265. URL <https://doi.org/10.48550/arXiv.2302.01265>.
- [182] Tiago Lopes Soares. A deductive verification framework for higher order programs. *CoRR*, abs/2011.14044, 2020. URL <https://arxiv.org/abs/2011.14044>.
- [183] Tiago Lopes Soares. Handle with care and confidence - extending Cameleer with algebraic effects and effect handlers. 2022.
- [184] Tiago Lopes Soares, Ion Chirica, and Mário Pereira. Static and dynamic verification of OCaml programs: The gospel ecosystem. In Tiziana Margaria and Bernhard Steffen, editors, *Leveraging Applications of Formal Methods, Verification and Validation. Specification and Verification - 12th International Symposium, ISoLA 2024, Crete, Greece, October 27-31, 2024, Proceedings, Part III*, volume 15221 of *Lecture Notes in Computer Science*, pages 247–265. Springer, 2024. doi: 10.1007/978-3-031-75380-0_14. URL https://doi.org/10.1007/978-3-031-75380-0_14.
- [185] Yahui Song, Darius Foo, and Wei-Ngan Chin. Automated temporal verification for algebraic effects. In Ilya Sergey, editor, *Programming Languages and Systems - 20th Asian Symposium, APLAS 2022, Auckland, New Zealand, December 5, 2022, Proceedings*, volume 13658 of *Lecture Notes in Computer Science*, pages 88–109. Springer, 2022. doi: 10.1007/978-3-031-21037-2_5. URL https://doi.org/10.1007/978-3-031-21037-2_5.

- [186] Yahui Song, Darius Foo, and Wei-Ngan Chin. Specification and verification for unrestricted algebraic effects and handling. *Proc. ACM Program. Lang.*, (ICFP), 2024.
- [187] Yahui Song, Darius Foo, and Wei-Ngan Chin. Specification and verification for unrestricted algebraic effects and handling. *Proc. ACM Program. Lang.*, (ICFP), 2024.
- [188] Marcelo Sousa and Isil Dillig. Cartesian hoare logic for verifying k-safety properties. In Chandra Krintz and Emery D. Berger, editors, *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2016, Santa Barbara, CA, USA, June 13-17, 2016*, pages 57–69. ACM, 2016. doi: 10.1145/2908080.2908092. URL <https://doi.org/10.1145/2908080.2908092>.
- [189] Matthieu Sozeau. The proved program of the month - type-safe printf via delimited continuations. <https://web.archive.org/web/20080513223632/http://www.lri.fr/perso/~sozeau/repos/coq/misc/shiftreset/GenuineShiftReset.html>, 2008.
- [190] Matthieu Sozeau. A new look at generalized rewriting in type theory. *J. Formaliz. Reason.*, 2(1):41–62, 2009. doi: 10.6092/ISSN.1972-5787/1574. URL <https://doi.org/10.6092/issn.1972-5787/1574>.
- [191] Kasper Svendsen. *Modular specification and verification for higher-order languages with state*. IT-Universitetet i København, 2013.
- [192] Kasper Svendsen, Lars Birkedal, and Matthew J. Parkinson. Verifying generics and delegates. In Theo D’Hondt, editor, *ECOOP 2010 - Object-Oriented Programming, 24th European Conference, Maribor, Slovenia, June 21-25, 2010. Proceedings*, volume 6183 of *Lecture Notes in Computer Science*, pages 175–199. Springer, 2010. doi:

10.1007/978-3-642-14107-2_9. URL https://doi.org/10.1007/978-3-642-14107-2_9.

- [193] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. Verifying higher-order programs with the Dijkstra monad. In Hans-Juergen Boehm and Cormac Flanagan, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 387–398. ACM, 2013. doi: 10.1145/2491956.2491978. URL <https://doi.org/10.1145/2491956.2491978>.
- [194] Nikhil Swamy, Aseem Rastogi, Aymeric Fromherz, Denis Merigoux, Danel Ahman, and Guido Martínez. Steelcore: an extensible concurrent separation logic for effectful dependently typed programs. *Proc. ACM Program. Lang.*, 4(ICFP):121:1–121:30, 2020. doi: 10.1145/3409003. URL <https://doi.org/10.1145/3409003>.
- [195] Wouter Swierstra and Tim Baanen. A predicate transformer semantics for effects (functional pearl). *Proc. ACM Program. Lang.*, 3(ICFP):103:1–103:26, 2019. doi: 10.1145/3341707. URL <https://doi.org/10.1145/3341707>.
- [196] Wenhao Tang, Leo White, Stephen Dolan, Daniel Hillerström, Sam Lindley, and Anton Lorenzen. Modal effect types. *Proc. ACM Program. Lang.*, 9(OOPSLA1):1130–1157, 2025. doi: 10.1145/3720476. URL <https://doi.org/10.1145/3720476>.
- [197] Joe Tidy. CrowdStrike IT outage affected 8.5 million Windows devices, microsoft says. <https://www.bbc.com/news/articles/cpe3zgznwjno>, 2024.
- [198] Amin Timany and Lars Birkedal. Mechanized relational verification of concurrent programs with continuations. *Proc. ACM Program. Lang.*,

- 3(ICFP):105:1–105:28, 2019. doi: 10.1145/3341709. URL <https://doi.org/10.1145/3341709>.
- [199] Turbolift. Turbolift. <https://marcinzh.github.io/turbolift/>, 2024.
- [200] Hiroshi Unno, Tachio Terauchi, and Eric Koskinen. Constraint-based relational verification. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I*, volume 12759 of *Lecture Notes in Computer Science*, pages 742–766. Springer, 2021. doi: 10.1007/978-3-030-81685-8_35. URL https://doi.org/10.1007/978-3-030-81685-8_35.
- [201] Niki Vazou, Eric L. Seidel, and Ranjit Jhala. LiquidHaskell: experience with refinement types in the real world. In Wouter Swierstra, editor, *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, pages 39–51. ACM, 2014. doi: 10.1145/2633357.2633366. URL <https://doi.org/10.1145/2633357.2633366>.
- [202] Fei Wang, Daniel Zheng, James M. Decker, Xilun Wu, Grégory M. Essertel, and Tiark Rompf. Demystifying differentiable programming: shift/reset the penultimate backpropagator. *Proc. ACM Program. Lang.*, 3(ICFP):96:1–96:31, 2019. doi: 10.1145/3341700. URL <https://doi.org/10.1145/3341700>.
- [203] Zhongye Wang, Qinxiang Cao, and Yichen Tao. Verifying programs with logic and extended proof rules: Deep embedding v.s. shallow embedding. *CoRR*, abs/2310.17616, 2023. doi: 10.48550/ARXIV.2310.17616. URL <https://doi.org/10.48550/arXiv.2310.17616>.
- [204] Patrick Wardle. Initial details about why CrowdStrike’s CSAgent.sys

- crashed. <https://news.ycombinator.com/item?id=41021366>, 2024.
- [205] Théo Winterhalter, Cezar-Constantin Andrici, Cătălin Hrițcu, Kenji Maillard, Guido Martínez, and Exequiel Rivas. Partial Dijkstra monads for all. In *Proceedings of the International Conference on Types for Proofs and Programs (TYPES)*, 2022.
- [206] Fabian Wolff, Aurel Bily, Christoph Matheja, Peter Müller, and Alexander J. Summers. Modular specification and verification of closures in Rust. *Proc. ACM Program. Lang.*, 5(OOPSLA):1–29, 2021. doi: 10.1145/3485522. URL <https://doi.org/10.1145/3485522>.
- [207] Shushu WU, Xiwei WU, and Qinxiang Cao. Encode the $\forall\exists$ relational hoare logic into standard hoare logic. *CoRR*, abs/2504.17444, 2025. doi: 10.48550/ARXIV.2504.17444. URL <https://doi.org/10.48550/arXiv.2504.17444>.
- [208] Hongseok Yang. Relational separation logic. *Theoretical Computer Science*, 375(1-3):308–334, 2007.
- [209] Yizhou Zhang, Guido Salvaneschi, and Andrew C. Myers. Handling bidirectional control flow. *Proc. ACM Program. Lang.*, 4(OOPSLA):139:1–139:30, 2020. doi: 10.1145/3428207. URL <https://doi.org/10.1145/3428207>.
- [210] Nikita Zyuzin and Aleksandar Nanevski. Contextual modal types for algebraic effects and handlers. *Proc. ACM Program. Lang.*, 5(ICFP):1–29, 2021. doi: 10.1145/3473580. URL <https://doi.org/10.1145/3473580>.

HIGHER-ORDER FUNCTIONS SPECIFIED IN CAMELEER

```

let rec map (f:'a -> 'b) (xs:'a list) =
  match xs with
  | [] -> []
  | x :: xs1 -> f x :: map f xs1
(*@ ys = map f xs
   variant xs
   ensures length ys = length xs
   ensures forall i. 0 <= i < length ys ->
     ys[i] = f (xs[i]) *)

(*@ lemma index_shift: forall x:'a, xs:'a list, i:int.
  1 <= i /\ i < length (Cons x xs) ->
    (Cons x xs)[i] = xs[i-1] *)

let rec foldr ((inv : 'b -> 'a seq -> bool) [@ghost])
  (f : 'a -> 'b -> 'b) (xs : 'a list) (acc : 'b)
= match xs with
  | [] -> acc
  | x :: t -> f x (foldr inv f t acc)
(*@ r = foldr inv f xs acc
   requires inv acc []
   requires forall acc x ys.
     inv acc ys -> inv (f x acc) (cons x
     ys)
   variant xs
   ensures inv r xs *)

```

Fig. A.1. foldr and map in Cameleer [158]

Cameleer specifications for map and foldr are shown in Fig. A.1. These may be compared to the specifications written for HEIFER in Section 2.1.

Because of the need to summarize the effects of f , map's postcondition uses a quantifier (over sequence indices) to talk about the elements of the input and output lists. This then necessitates lemmas such as `index_shift` for relating indices to list destructuring, finally requiring more lines of specification per line of code.

The `foldr` specification is similar to the one for Iris ([Section 2.1](#)), but does not use a higher-order triple, instead requiring a ghost argument for an invariant that must be preserved between calls to `f`. This approach is representative of many verifiers, including Dafny, WhyML, and vanilla F*. As mentioned before, this parametrisation of the specification with a summary of `f` is nontrivial, in that it cannot be mechanically done for every higher-order function, as this pair of examples shows.

